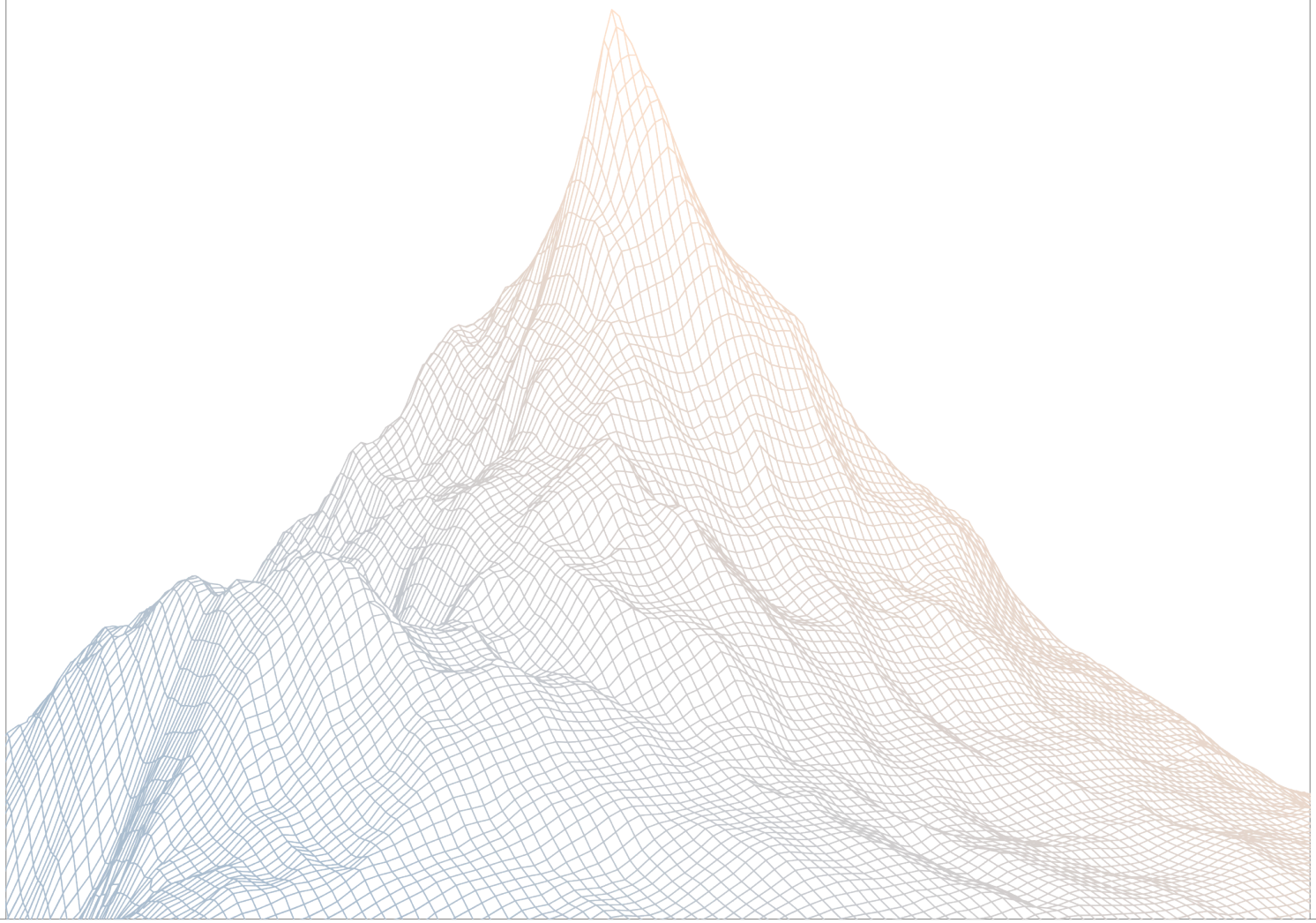
Zenith

Interfold

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: July 22nd to August 10th, 2026

AUDITED BY: Dravee
Mario Poneder

Contents

1 Introduction 3

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 4

2.1 About Interfold 4

2.2 Scope 4

2.3 Audit Timeline 6

2.4 Issues Found 6

2.5 Security Note 6

3 Findings Summary 7

4 Findings 12

4.1 Critical Risk 12

4.2 High Risk 17

4.3 Medium Risk 41

4.4 Low Risk 99

4.5 Informational 151

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Interfold

The Interfold enables applications that coordinate computation across independent parties without exposing their private inputs. Instead of pooling sensitive data or delegating execution authority to a centralized operator, the protocol executes encrypted workflows across a distributed network of ciphernodes.

2.2 Scope

The engagement involved a review of the following targets:

Target	interfold
Repository	https://github.com/theinterfold/interfold
Commit Hash	`c2097da61b4d07c4ce83840393ff4e9f171eefb4`

Files	`E3RefundManager.sol`
	`Interfold.sol`
	`lib/ExitQueueLib.sol`
	`lib/InterfoldPricing.sol`
	`registry/BondingRegistry.sol`
	`registry/CiphernodeRegistryOwnable.sol`

Target	Mitigation Review
Repository	https://github.com/theinterfold/interfold
Commit Hash	`c64bcfb890b596e626ea6578c5fbd53f808c3b43`

Files	`E3RefundManager.sol`
	`Interfold.sol`
	`lib/ExitQueueLib.sol`
	`lib/InterfoldPricing.sol`
	`registry/BondingRegistry.sol`
	`registry/CiphernodeRegistryOwnable.sol`

2.3 Audit Timeline

July 22, 2026	Audit start
August 10, 2026	Audit end
August 17, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	1
High Risk	6
Medium Risk	18
Low Risk	19
Informational	18
Total Issues	**62**

2.5 Security Note

Considering the number of issues identified, it is statistically likely that there are more complex bugs still present that could not be identified given the time-boxed nature of this engagement. It is recommended that a follow-up audit and development of a more complex stateful test suite be undertaken prior to continuing to deploy significant monetary capital to production.

3

Findings Summary

ID	Description	Status
C-1	Any caller can force paid E3 computations into Decryption Timeout via unverified ciphertextCommitment in publishCiphertextOutput()	Resolved
H-1	Any caller can front-run committee finalization with markE3Failed due to overlapping eligibility windows	Resolved
H-2	Committee hashing truncates each member to 64 bits, making the shipped DKG proof flow incompatible with on-chain verification	Resolved
H-3	Compute Deadline Expires During DKG, Penalizing Requester for Protocol Scheduling Error	Resolved
H-4	TicketToken Registry Migration DOS due to Unclaimed Ticket	Resolved
H-5	Requester Can Break Unbiased Committee Selection by Grinding Request Seeds	Resolved
H-6	Requester Loses Funds on an Accepted but Structurally Impossible Schedule	Resolved
M-1	Requester-defined E3 programs can externalize undecryptable ciphertext failures onto committees	Resolved
M-2	Recipient selection rewards slashed or later-expelled operators	Resolved
M-3	Provisional membership can authorize invalid slashes and block failure settlement	Resolved
M-4	Committee options are not bound to supported circuits and gas limits	Resolved
M-5	Pricing omits pre-input availability	Resolved
M-6	Solidity and Rust use inconsistent eligibility and ticket-capacity snapshots for sortition	Resolved
M-7	Banned operator remains active and can join new committees after governance ban	Resolved

ID	Description	Status
M-8	Unbound public-key bytes can be front-run to poison one-shot committee publication	Resolved
M-9	Committee-affecting slash policies can leave nonterminal E3s below threshold	Resolved
M-10	Shared fold-attestation verifiers enable conditional cross-registry DKG replay	Resolved
M-11	slashTicketBalance credits the full slash amount without checking what the exit queue actually yielded	Resolved
M-12	Committee member can withdraw all collateral before E3 completion by deregistering mid-lifecycle	Resolved
M-13	Asset rotation can leave raw-unit economic parameters stale	Resolved
M-14	Synchronous slashing-manager callbacks can block exits and registration	Resolved
M-15	Retained SlashingManagers can drain slashed ticket funds to arbitrary addresses	Resolved
M-16	The E3 projection overwrites owner reward credits with tokenless operator allocations	Resolved
M-17	A post-exit owner reset can redirect rewards earned under the previous owner	Resolved
M-18	Configurable slash failure reasons can charge requesters for committee loss	Resolved
L-1	Incoherent dependency snapshots can strand or misdirect slash proceeds	Resolved
L-2	On-chain slashing ignores the configured accusation-vote validity policy	Resolved
L-3	Late DKG publication can reclassify a supplier-side timeout as requester-paid	Resolved

ID	Description	Status
L-4	Revoking a retained slashing manager can release collateral and strand slash routes	Resolved
L-5	The whitelisted LiquidityLauncher bypasses pre-TGE transfer restrictions	Resolved
L-6	In-flight deadlines and the DKG fold verifier are not snapshotted	Resolved
L-7	Parameter-set identifiers are not consistently bound across E3 components	Resolved
L-8	Fee-on-transfer assets create underfunded fee and slash liabilities	Resolved
L-9	Solidity does not enforce Rust's active-job ticket-capacity adjustment	Resolved
L-10	Non-canonical BN254 inputs let a custom prover decouple proven coefficients from published plaintext	Resolved
L-11	Verifier callers do not defensively reject false results from non-conforming implementations	Resolved
L-12	Operators can claim ticket exits before submitting snapshot-weighted sortition tickets	Resolved
L-13	Donations can grief non-atomic license-token rotation	Resolved
L-14	Storage validation accepts unsafe dynamic-array stride changes and enum reordering	Resolved
L-15	SlashExecuted overstates partial collateral slashes	Resolved
L-16	Static registry writers cannot serve concurrent E3 registry generations	Resolved
L-17	isCiphernodeEligible queries the global bonding registry instead of the per-E3 snapshot	Resolved

ID	Description	Status
L-18	Documented requester cancellation has no requester-accessible implementation	Resolved
L-19	Unmigrated registry rotation desynchronizes operator membership and committee capacity	Resolved
I-1	The inclusive failure-reason bound admits a non-classifiable sentinel	Resolved
I-2	The operator registry has a finite lifetime insertion budget	Resolved
I-3	Minting lifecycle documentation conflicts with the implementation	Resolved
I-4	Honest node refund share permanently stranded when per-node amount rounds to zero	Resolved
I-5	Per-route rounding directs cumulative slash dust to the protocol and final committee address	Resolved
I-6	Floor rounding can leave operators active with zero license collateral	Resolved
I-7	Requests rely on ERC-20 allowances rather than an in-call fee bound	Resolved
I-8	Fail-open active-node lookup can misallocate refunds after an unexpected registry failure	Resolved
I-9	Duplicate cap definitions can desynchronize exposed and enforced limits	Resolved
I-10	Duplicate owner validation in _bondLicense obscures the authorization boundary	Resolved
I-11	onlyBondingRegistry modifier is defined but never used	Resolved
I-12	setAccusationVoteValidity NatSpec contradicts its require guard	Resolved

ID	Description	Status
I-13	Near-zero accusationVoteValidity bypasses the timelock intent	Resolved
I-14	The license-token setter accepts tokens that disable owner transfers for license-bonded positions	Resolved
I-15	E3Program reentrancy can emit regressive lifecycle events	Resolved
I-16	Ticket-token rotation does not verify reciprocal registry authorization	Resolved
I-17	Fresh Interfold controllers collide with retained local E3-ID state	Resolved
I-18	Expelled members retain restricted grace-period failure authority	Resolved

4

Findings

4.1 Critical Risk

A total of 1 critical risk findings were identified.

[C-1] Any caller can force paid E3 computations into Decryption Timeout via unverified `ciphertextCommitment` in `publishCiphertextOutput()`

`SEVERITY`: Critical	`IMPACT`: High
`STATUS`: Resolved	`LIKELIHOOD`: High

Target

- `Interfold.publishCiphertextOutput()`
- `Interfold.sol#L406-L459`
- `InterfoldPricing.sol#L76-L106`
- `InterfoldPricing.sol#L136-L157`
- `IE3Program.sol#L30-L40`
- `E3RefundManager.sol#L254-L304`

Description

`publishCiphertextOutput()` is permissionless and accepts `ciphertextCommitment` as independent caller-controlled calldata.

The publication validator checks only the lifecycle stage, deadlines, and whether a ciphertext hash was already stored; it never receives the commitment (`InterfoldPricing.sol#L136-L157`).

Interfold then:

1. hashes the ciphertext;
2. stores the caller-supplied commitment directly;
3. advances the E3 to `CiphertextReady`; and
4. calls `IE3Program.verify()` with only `e3Id`, `ciphertextOutputHash`, and `proof`.

The verifier interface contains no commitment parameter (`IE3Program.sol#L30-L40`). It

therefore cannot enforce that the commitment stored by Interfold is the commitment authenticated by the computation proof.

An attacker can observe a valid pending publication, copy its ciphertext and proof, replace only `ciphertextCommitment`, and front-run it. Because the proof remains valid, the transaction succeeds. The first publication permanently moves the E3 to `CiphertextReady`, so a later honest publication is rejected.

Plaintext publication later forwards the stored commitment to the decryption verifier (`Interfold.sol#L406-L459`, `InterfoldPricing.sol#L76-L106`). A compliant decryption verifier compares that value with the SAFE commitment recomputed by the decryption proof and rejects on mismatch.

The E3 consequently remains `CiphertextReady` until its decryption deadline expires, after which it can only be failed with `DecryptionTimeout`.

Under the current refund implementation, `DecryptionTimeout` is classified as ciphernode fault and returns the entire original payment to the requester while allocating zero base compensation to honest nodes (`E3RefundManager.sol#L254-L304`). The requester therefore suffers delayed fund availability and denial of service rather than permanent principal loss.

Step-by-step walkthrough

1. Publication is permissionless and the commitment is unverified (Interfold.sol#L358-L391):

```
function publishCiphertextOutput(
    uint256 e3Id,
    bytes calldata ciphertextOutput,
    bytes32 ciphertextCommitment, // caller-controlled, never verified
    bytes calldata proof
) external returns (bool success) {
```

`validatePublishCiphertext` checks stage, timing, and idempotency. It is never given the commitment.

2. Output hash and commitment are stored independently (Interfold.sol#L382-L388):

```
e3s[e3Id].ciphertextOutput = keccak256(ciphertextOutput);     // validated
    by program proof
e3s[e3Id].ciphertextCommitment = ciphertextCommitment;         // stored raw
    from calldata
```

3. The program verifier cannot enforce the binding (Interfold.sol#L390):

```
(success) = e3.e3Program.verify(e3Id, ciphertextOutputHash, proof);
// ciphertextCommitment is absent — the verifier has no way to compare it
```

4. The poisoned commitment becomes permanent. Stage advances to `CiphertextReady`. Any correction attempt reverts with `InvalidStage` because the stage is no longer `KeyPublished`.

5. Decryption verification later trusts the stored commitment (InterfoldPricing.sol#L99-L106): the decryption verifier is explicitly given `ciphertextCommitment` and, per `IDecryptionVerifier.sol`, must reject when the proof-derived commitment doesn't match. An honest decryption proof computes the real commitment from the actual ciphertext; the stored value is the attacker's — mismatch, revert.

6. The only recovery is timeout. After `decryptionDeadline` passes, anyone calls `markE3Failed` → `DecryptionTimeout`. The requester eventually recovers their fee via `processE3Failure` + `claimRequesterRefund`, but the computation was never delivered and honest operators who completed committee formation and DKG receive nothing.

Risk Analysis

- **Impact:** High. Protocol-wide griefing. No E3 computation can ever complete. Honest committee members who performed DKG work receive zero compensation. Requester fees are locked for up to 30 days per E3 and the requested computation is never delivered.
- **Likelihood:** High. Any unprivileged account can execute the attack for the cost of gas (cheap). The attacker needs only to observe a pending `publishCiphertextOutput` transaction in the mempool, copy its valid `ciphertextOutput` and `proof`, replace only the `ciphertextCommitment`, and submit first. No committee membership, compute provider role, or cryptographic knowledge required.
- **Severity:** Critical. At protocol scale, a single attacker monitoring the mempool can front-run every `publishCiphertextOutput()` calls across all active E3s. For the cheap price of gas, the protocol's entire value proposition (confidential computation as a service) is dead **(similar to a permanent DOS)**. Honest operators work for free on every E3. Requesters must wait up to 30 days to recover fees and never get their computation.

Note: Documentation discrepancy

In `requestor-guide.mdx`, under **"3. When things go wrong"**, the table states:

> `DecryptionTimeout` — "The committee never published the decryption." — "~0% (work was already done)"

The current code instead returns **100% of the original payment to the requester** and **0% to honest nodes** for `DecryptionTimeout`.

For this finding, the Solidity behavior should be used for severity: temporary requester fee unavailability, no permanent requester principal loss, and denial of honest-node compensation. This could've been a loss of funds for requesters.

Proof of Concept

POC File: Interfold.spec.ts

Six passing Hardhat tests added to `Interfold.spec.ts` under the describe block `"H-01: publishCiphertextOutput() accepts unbound ciphertextCommitment"`. The tests prove:

- Same `ciphertextOutput` and `proof` accepted with a completely different commitment (core binding gap).
- Honest publisher permanently blocked from correcting the commitment (reverts with `InvalidStage`).
- Any unprivileged address can execute the attack (no caller restriction).
- Commitment is fully orthogonal to proof verification (3 different commitments including `bytes32(0)` all accepted with identical proof across 3 E3s).
- Full lifecycle: poison → fee locked → `markE3Failed` blocked before deadline → warp past deadline → `DecryptionTimeout` → `processE3Failure` → requester claims 100% refund, honest nodes receive 0.
- On-chain event emits the poisoned commitment value.

```
Running Mocha tests


  Interfold
    H-01: publishCiphertextOutput() accepts unbound ciphertextCommitment
      ✔ accepts a poisoned commitment with valid ciphertextOutput and proof (550ms)
      ✔ permanently blocks the honest publisher from correcting the commitment (125ms)
      ✔ any unprivileged address can publish (no caller restriction) (121ms)
      ✔ commitment is orthogonal: same proof validates with any commitment value (493ms)
      ✔ full attack: poison → fee locked → timeout → DecryptionTimeout failure (133ms)
      ✔ emits CiphertextOutputPublished with the poisoned commitment value (179ms)


  6 passing (2s)
```

Mock verifier limitation: The test harness uses permissive mock verifiers, so the PoC cannot demonstrate that a real decryption verifier rejects honest proofs against the poisoned commitment. The code path is verified — the stored commitment IS passed to the verifier — but the rejection behavior depends on the production `IDecryptionVerifier` implementation conforming to its documented interface.

Recommendations:

The root cause is that `ciphertextCommitment` is stored as trusted caller input without being included in the verified statement. The fix must cryptographically bind the commitment to the accepted computation proof.

Preferred fix — extend `IE3Program.verify()` to include the commitment:

```
// IE3Program.sol
function verify(
    uint256 e3Id,
    bytes32 ciphertextOutputHash,
    bytes32 ciphertextCommitment,  // add this parameter
    bytes calldata proof
) external returns (bool success);
```

Then in `publishCiphertextOutput`:

```
(success) = e3.e3Program.verify(
    e3Id,
    ciphertextOutputHash,
    ciphertextCommitment,  // now part of the verified statement
    proof
);
```

Alternative — if the SAFE commitment can be deterministically computed on-chain from `ciphertextOutput`, remove it from caller-controlled calldata and derive it inside Interfold or the verifier.

Interfold: Resolved with `@aa9a6c7fe9 ...`

Zenith: Verified.

4.2 High Risk

A total of 6 high risk findings were identified.

[H-1] Any caller can front-run committee finalization with `markE3Failed` due to overlapping eligibility windows

SEVERITY: High	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- CiphernodeRegistryOwnable.finalizeCommittee()
- Interfold.markE3Failed()

Description

finalizeCommittee() requires `block.timestamp > committeeDeadline`. markE3Failed() for the `Requested` stage also requires `block.timestamp > committeeDeadline` (read from the registry). There is no timestamp window where finalization is valid but failure marking is not.

An attacker can front-run any `finalizeCommittee` call with `markE3Failed`, transitioning the E3 to `Failed` before finalization executes. Finalization's callback to `interfold.onCommitteeFinalized()` then reverts because the stage is no longer `Requested`.

This contradicts the registry's own design intent: if enough operators submitted valid tickets, the committee should succeed.

Proof of Concept

Copy-paste in `Interfold.spec.ts` and run with `npx hardhat test test/Interfold.spec.ts --grep "bug"`

```
Running Mocha tests


  Interfold
    bug: Committee finalization and timeout race
      ✔ markE3Failed front-runs finalizeCommittee despite valid threshold (470ms)


  1 passing (508ms)
```

```
describe("bug: Committee finalization and timeout race", function () {
  it("markE3Failed front-runs finalizeCommittee despite valid threshold",
    async function () {
    const {
      interfold,
      usdcToken,
      ciphernodeRegistryContract: registry,
      request,
      notTheOwner,
      operator1,
      operator2,
      operator3,
    } = await loadFixture(setup);

    // Create E3
    await makeRequest(interfold, usdcToken, {
      ...request,
      inputWindow: [(await time.latest()) + 200, (await time.latest())
    + 500],
    });
    expect(await interfold.getE3Stage(0)).to.equal(1); // Requested

    // All 3 operators submit tickets (threshold met)
    await registry.connect(operator1).submitTicket(0, 1);
    await registry.connect(operator2).submitTicket(0, 1);
    await registry.connect(operator3).submitTicket(0, 1);

    // Advance past committee deadline
    const deadline = await registry.getCommitteeDeadline(0);
    await time.increaseTo(Number(deadline) + 1);

    // Attacker front-runs: markE3Failed before finalizeCommittee
    // After grace period (or if grace == 0), anyone can call this
    const gracePeriod = await interfold.markFailedGracePeriod();
    if (gracePeriod > 0n) {
      await time.increaseTo(Number(deadline) + Number(gracePeriod) + 1);
```

```
    }

    await interfold.connect(notTheOwner).markE3Failed(0);
    expect(await interfold.getE3Stage(0)).to.equal(6); // Failed
    expect(await interfold.getFailureReason(0)).to.equal(1); //
    CommitteeFormationTimeout

    // Honest finalizeCommittee now reverts
    await expect(registry.finalizeCommittee(0)).to.be.revert(ethers);
  });
});
```

Recommendations

Make committee formation outcome authoritative in the registry. Once the submission deadline has passed, a `Requested` E3 should not be independently marked failed by Interfold when the registry has already collected enough valid committee members.

The simplest safe fix is for `markE3Failed()` to consult the registry before applying `CommitteeFormationTimeout`:

```
if (current == E3Stage.Requested) {
    ICiphernodeRegistry registry = _registryFor(e3Id);

    if (registry.committeeThresholdMet(e3Id)) {
        revert CommitteeReadyForFinalization(e3Id);
    }
}
```

The registry can expose a lightweight view:

```
function committeeThresholdMet(
    uint256 e3Id
) external view returns (bool) {
    Committee storage c = committees[e3Id];

    return
        c.stage == CommitteeStage.Requested &&
        c.topNodes.length >= c.threshold[1];
}
```

This ensures:

```
threshold met→
 only finalization is permitted

threshold not met→
 timeout/failure processing is permitted
```

Alternatively, remove the `Requested` stage from the generic `markE3Failed()` path and require callers to use `finalizeCommittee()` to resolve committee formation. Since `finalizeCommittee()` already handles both successful formation and insufficient membership, it can serve as the sole normal-state arbiter.

Do not attempt to solve the race by changing `>` to `≥` or by creating a one-second priority window. Timestamp inequalities do not guarantee transaction ordering and remain vulnerable in the first block after the deadline.

Avoid making `markE3Failed()` directly call the full `finalizeCommittee()` function unless the lifecycle is redesigned to safely handle the resulting nested registry-to-Interfold callback and potentially large finalization gas cost.

Interfold: Resolved with `@e7c11bd886 ...`

Zenith: Verified.

[H-2] Committee hashing truncates each member to 64 bits, making the shipped DKG proof flow incompatible with on-chain verification

SEVERITY: High	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: High

Target

- packages/interfold-contracts/contracts/lib/CommitteeHashLib.sol#L17-L36
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L317-L367
- packages/interfold-contracts/contracts/verifiers/bfv/BfvPkVerifier.sol#L128-L151
- crates/zk-prover/src/circuits/aggregation/node_dkg_fold.rs#L493-L514
- circuits/lib/src/math/committee_hash.nr#L23-L52

Description:

The canonical committee hash should be computed over the concatenation of every member's complete 20-byte address. `CommitteeHashLib.hash` converts each address to `bytes20`, which is already left-aligned when accessed from inline assembly, but then incorrectly shifts it left by another 96 bits:

```
bytes20 a = bytes20(nodes[i]);

assembly {
    mstore(add(add(packed, 0x20), offset), shl(96, a))
}
```

The additional shift discards the first 12 address bytes. Each intended chunk:

```
address[0:20]
```

is instead encoded as:

```
address[12:20] || bytes12(0)
```

The resulting committee hash therefore depends on only the least-significant 64 bits of each address.

The behavior was reproduced against the compiled contract. For a representative committee, the hashes were:

```
canonical: 0x09e24a4819dcbdd505086f5e0ac6ca674aac55178f28f673287383d9950d3119
on-chain:  0x52d06de9c4e5e77ebb2734842efb608e458479f5af5b11678a5e5bf51d61d095
```

The repository-shipped Rust prover preserves all 20 address bytes when deriving `committee_hash_hi` and `committee_hash_lo`, and supplies the actual committee addresses as `committee_members`. The Noir circuit independently extracts and hashes all 20 bytes of every supplied member. Consequently, the proof generated by the shipped production prover contains the canonical hash.

During `publishCommittee`, the registry instead computes the truncated Solidity hash and passes it to `BfvPkVerifier`. The verifier requires the proof's public hash limbs to equal that corrupted value:

```
if (
    publicInputs[committeeHashHiIdx] ≠
    CommitteeHashLib.hi(committeeHash)
) {
    revert DomainBindingMismatch();
}
```

The stock production proof therefore reverts with `DomainBindingMismatch`. The registry's preceding state writes roll back, the public key remains unpublished, and the E3 cannot advance through the shipped production flow. Ordinary tests using mock proof verifiers can conceal this incompatibility because those mocks do not enforce the real public-input relationship.

Strictly, this does not make every conceivable proof impossible. The circuit's private `committee_members` input is not otherwise tied to the wrapper's `sortedNodes`, which are currently unused. A custom prover could supply pseudo-addresses of the form:

```
originalAddress[12:20] || bytes12(0)
```

and thereby construct the hash expected by the buggy Solidity implementation. This is not a valid mitigation: it is unsupported by the shipped prover, reduces the effective committee binding to 64 bits per member, and remains inconsistent with other repository components that derive canonical committee and decryption domains from complete addresses.

No practical address-suffix collision attack is required for the primary impact—the shipped prover mismatch occurs deterministically without attacker interaction. The truncation also weakens collision resistance, although constructing exploitable committee-address collisions was not demonstrated.

The issue does not directly steal funds, but it prevents the proof-backed protocol from com-

pleting E3s through its intended production path and can force affected requests into timeout and failure handling.

Recommendations:

It is recommended to encode exactly the 20 bytes of each address without an additional shift. A straightforward memory-safe implementation is:

```
function hash(address[] memory nodes) internal pure returns (bytes32) {
    bytes memory packed = new bytes(nodes.length * 20);

    for (uint256 i = 0; i < nodes.length; ++i) {
        bytes20 node = bytes20(nodes[i]);
        uint256 offset = i * 20;

        for (uint256 j = 0; j < 20; ++j) {
            packed[offset + j] = node[j];
        }
    }

    return keccak256(packed);
}
```

Using `mstore(ptr, a)` with the existing `bytes20` value corrects the alignment but still writes a complete 32-byte word. The final store can extend up to 12 bytes beyond the logical buffer. If assembly is retained for gas efficiency, reserve sufficient physical memory, write the correct left-aligned value, and maintain the free-memory pointer correctly.

Do not replace the implementation with `abi.encodePacked(nodes)`, because packed encoding of an `address[]` pads its array elements. Update comments and interfaces to define the intended encoding unambiguously as:

```
keccak256(concat(bytes20(nodes[0]), ..., bytes20(nodes[n-1])))
```

Add permanent known-vector tests comparing:

- `CommitteeHashLib.hash`;
- the production Rust committee-hash helper;
- the Noir committee-hash circuit; and
- a reference concatenation of raw 20-byte addresses.

Include addresses with leading zero bytes and nonzero high-order bytes so alignment errors cannot pass accidentally.

Because this internal library function is compiled into `CiphernodeRegistryOwnable`, de-

ployed systems must upgrade or redeploy the registry implementation; updating the library source alone does not change deployed bytecode.

Production publication normally reverts before storing the corrupted value. Any E3 previously accepted through a mock verifier, custom verifier, or custom-proof workaround should be restarted after the upgrade rather than mutating its stored hash in place, because the hash participates in subsequent decryption-domain construction.

Interfold: Resolved with `@19aa047d3e ...`

Zenith: Verified.

[H-3] Compute Deadline Expires During DKG, Penalizing Requester for Protocol Scheduling Error

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- InterfoldPricing.sol#L266-L283
- Interfold.sol#L297-L300

Description:

Foreword: The other finding "Requester Loses Funds on an Accepted but Structurally Impossible Schedule" covers the case where `computeDeadline < committeeDeadline` — sortition alone consumes the compute window. Its minimum recommended fix validates `computeDeadline > committeeDeadline` but explicitly frames the DKG-inclusive check as a design choice. This finding covers the case that fix leaves open: DKG completes within its allowed window but after `computeDeadline`.`computeDeadline` is set at request time:

```
_e3Deadlines[e3Id].computeDeadline = requestParams.inputWindow[1]
    + _timeoutConfig.computeWindow;
```

`publishCiphertextOutput` requires `computeDeadline ≥ block.timestamp`. Computation cannot begin until `KeyPublished`, which requires both sortition and DKG to complete. `validateRequest` checks neither phase against the compute window.

When `computeDeadline > committeeDeadline` but `computeDeadline < committeeDeadline + dkgWindow`, the committee can finalize on time, but if DKG takes any significant portion of its allowed window, key publication occurs after the compute deadline. `publishCiphertextOutput` permanently reverts with `CommitteeDutiesCompleted`. The E3 terminates as `ComputeTimeout` / `FailurePayer.Requester`, penalizing the requester with 45% of the fee (40% `workCompletedBps` to operators + 5% `protocolBps`) for a scheduling failure the protocol accepted.

Step-by-Step Example

Configuration (all within governance bounds):

- `sortitionSubmissionWindow` = 3600s (1 hour)

- `dkgWindow` = 86400s (1 day)
- `computeWindow` = 7200s (2 hours)
- `decryptionWindow` = 3600s

Requester submits `inputWindow = [now+100, now+200]`.

`computeDeadline = now + 200 + 7200 = now + 7400` `committeeDeadline = now + 3600`

The other finding's minimum fix: `7400 > 3600` → **passes**.

Time	Event	`computeDeadline` state
T+0	`request()`	set to T+**7400**
T+3601	`finalizeCommittee()`	`dkgDeadline` = T+**90001**
T+7401		**compute deadline expired**
T+50000	`publishCommittee()` (within `dkgDeadline`)	deadline passed **42600s ago**
T+50000	`publishCiphertextOutput()`	reverts: `CommitteeDutiesCompleted`
T+50000	`markE3Failed()`	`ComputeTimeout` → `FailurePayer.Requester`

DKG completed within its allowed window. The requester loses 40% of the fee for a schedule the protocol accepted and could never fulfil.

Risk Analysis

Impact: High. Deterministic requester fund loss (default 45% of E3 fee) for a computation impossible to complete at the moment of request. Operators receive compensation from the penalty despite the protocol's own scheduling causing the failure.

Likelihood: Medium. Requires `computeWindow < sortitionSubmissionWindow + dkgWindow` while `computeWindow > sortitionSubmissionWindow` (otherwise the other finding catches it first). Not reachable under recorded defaults but achievable through valid governance configuration — a short compute window for lightweight tasks with a long DKG window. No malice required from the requester.

Recommendations:

Include the DKG window in the schedule validation, as the other finding's stronger recommendation already proposes:

```
require(
    computeDeadline > committeeDeadline + _timeoutConfig.dkgWindow,
    "Compute deadline would expire before key publication"
);
```

Alternatively, defer `computeDeadline` assignment to `onCommitteePublished`, setting it relative to the actual moment computation becomes possible:

```
function onCommitteePublished(uint256 e3Id, ...) external {
    _e3Deadlines[e3Id].computeDeadline =
        max(block.timestamp, e3s[e3Id].inputWindow[1]) +
        _timeoutConfig.computeWindow;
}
```

Interfold: Resolved with PR-1793.

Zenith: Verified.

[H-4] TicketToken Registry Migration DOS due to Unclaimed Ticket

SEVERITY: High	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: High

Target

- BondingRegistry.sol#L501-L522
- InterfoldTicketToken.sol#L366-L371

Description:

`burnTickets()` decrements `totalSupply` and increments `payableBalance` by the same amount. All registry-change paths on the TicketToken (`setRegistry`, `requestRegistryChange`, `activateRegistryChange`) require `payableBalance == 0`. Only the current registry can call `payout()`, and `BondingRegistry` only calls it inside `claimExits()`, which only the operator can trigger. No third party or owner can force a payout.

The `payableBalance` gate is intentionally protective — migrating with outstanding payable balance would strand operator USDC. The issue is the absence of any privileged path to settle unlocked exits during migration, giving any single operator an indefinite veto over TicketToken registry changes.

Note that `BondingRegistry.setTicketToken()` independently requires `totalSupply() + payableBalance() == 0`, so this vector is only reachable after all operators have deregistered and burned their tickets. The `payableBalance` component is the residual blocker when an operator has burned but not claimed.

Risk Analysis

Impact: Medium. Governance cannot complete a TicketToken registry migration while any operator has an unclaimed exit, even if the exit is fully unlocked and the operator is no longer registered. No permanent fund loss — the operator's USDC is safe in the exit queue — but the migration capability is held hostage by a single non-cooperative or key-lost operator. The practical blast radius is bounded: the `totalSupply` gate means the entire operator set must already be wound down before `payableBalance` becomes the blocking factor.

Likelihood: High. The blocking condition requires at least one operator to have burned tickets without claiming the exit. This does not require an attacker. The finding only matters during a full operator wind-down — the exact scenario where every operator who ever

participated has gone through `burnTickets`. Across any real operator set running over a meaningful time period, some will have lost keys, abandoned the protocol, or simply never claimed. Unclaimed exits have no expiry. The larger the operator set and the longer the protocol has run, the closer to a certainty this condition exists at migration time. The intentional front-run variant (burn during the 1-day activation delay, no capital at risk) adds a second independent path to the same outcome.

Proof of Concept

Copy-paste in `E3Integration.spec.ts` and run with `npx hardhat test --grep "bug"`

```
describe("bug: unclaimed ticket exit vetoes TicketToken registry
    migration", function () {
  it("single operator blocks setTicketToken by never claiming exits",
    async function () {
    const {
      bondingRegistry,
      owner,
      operator1,
      operator2,
      operator3,
      setupOperator,
    } = await loadFixture(setup);

    await setupOperator(operator1);
    await setupOperator(operator2);
    await setupOperator(operator3);

    // 1. operator1 withdraws 1 ticket unit into the exit queue
    const withdrawAmount = await bondingRegistry.ticketPrice();
    await
    bondingRegistry.connect(operator1).removeTicketBalance(withdrawAmount);

    // 2. Verify payableBalance is now > 0 on the TicketToken
    const ticketTokenAddress = await bondingRegistry.ticketToken();
    const ticketToken = await ethers.getContractAt(
      "InterfoldTicketToken",
      ticketTokenAddress,
    );
    const payableBalance = await ticketToken.payableBalance();
    expect(payableBalance).to.be.gt(0);

    // 3. Wait past exit delay so the exit is claimable
    const exitDelay = await bondingRegistry.exitDelay();
    await time.increase(Number(exitDelay) + 1);
```

```
    // 4. operator1 deliberately does NOT call claimExits()

    // 5. Owner tries to migrate to a new TicketToken — REVERTS
    //    setTicketToken checks: current.totalSupply() +
    current.payableBalance() ≠ 0
    const mockNewToken = await operator2.getAddress(); // any address with
    code won't work,
    // but the revert happens before the code check when liabilities ≠ 0

    // Deploy a minimal mock to pass the code-length check
    // For simplicity, just verify the revert condition directly:
    const currentToken = await bondingRegistry.ticketToken();
    const currentTicketToken = await ethers.getContractAt(
      "InterfoldTicketToken",
      currentToken,
    );
    const totalSupply = await currentTicketToken.totalSupply();
    const payable = await currentTicketToken.payableBalance();
    const liabilities = totalSupply + payable;

    // liabilities > 0 means ANY setTicketToken call with a different
    address will revert
    expect(liabilities).to.be.gt(0);

    // Even if totalSupply were 0 (all operators deregistered and burned),
    // payableBalance alone blocks migration
    // This demonstrates the griefing vector: operator1's unclaimed exit
    // holds payableBalance > 0 indefinitely

    // 6. Verify no third party can force the payout
    //    claimExits is msg.sender-gated
    await expect(
      bondingRegistry.connect(owner).claimExits(withdrawAmount, 1),
    ).to.be.revert(ethers); // owner has no exit queued

    // 7. Verify owner cannot sweep payableBalance
    //    There is no owner function to force-clear payableBalance
    //    The only path is: operator1.claimExits() → bondingRegistry calls
    ticketToken.payout()
  });

  it("front-run variant: operator blocks activation of a pending registry
    change", async function () {
    const {
      bondingRegistry,
      owner,
      operator1,
```

```
      operator2,
      operator3,
      setupOperator,
    } = await loadFixture(setup);

    await setupOperator(operator1);
    await setupOperator(operator2);
    await setupOperator(operator3);

    const ticketTokenAddress = await bondingRegistry.ticketToken();
    const ticketToken = await ethers.getContractAt(
      "InterfoldTicketToken",
      ticketTokenAddress,
    );

    // 1. Verify payableBalance starts at 0
    expect(await ticketToken.payableBalance()).to.equal(0);

    // 2. Suppose owner initiates a registry change on the TicketToken
    //    (requestRegistryChange would succeed here because payableBalance
    == 0)

    // 3. During the delay window, operator1 burns tickets into the exit
    queue
    const withdrawAmount = await bondingRegistry.ticketPrice();
    await
    bondingRegistry.connect(operator1).removeTicketBalance(withdrawAmount);

    // 4. payableBalance is now > 0
    expect(await ticketToken.payableBalance()).to.be.gt(0);

    // 5. When activation is attempted, the second payableBalance check
    reverts
    //    activateRegistryChange() calls
    _revertIfPayableBalanceOutstanding()

    // 6. operator1 never calls claimExits — migration permanently blocked
    //    No governance or third-party bypass exists in the contract
  });
});
```

Recommendations:

Implement a migration mode that freezes new exits and auto-settles unlocked exit-queue entries to their owners, preserving the exit delay and slash-check window while clearing

`payableBalance`. This maintains the protective invariant (no stranded USDC) without giving individual operators a veto over governance migration.

Interfold: Resolved with PR-1793.

Zenith: Verified.

[H-5] Requester Can Break Unbiased Committee Selection by Grinding Request Seeds

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- Interfold.request()

Description

Interfold.request() derives the committee-selection seed from the current block randomness and the next E3 identifier:

```
uint256 seed = uint256(
    keccak256(abi.encode(block.prevrandao, e3Id))
);
```

Committee candidates are ranked using:

```
keccak256(
    abi.encodePacked(node, ticketNumber, e3Id, seed)
);
```

Lower scores enter the committee's top-N set.

Because `request()` returns the newly created `E3`, a requester contract can inspect the generated seed before allowing the transaction to complete:

```
function grind(RequestParams calldata params) external {
    (uint256 e3Id, E3 memory e3) = interfold.request(params);

    if (!isFavorable(e3Id, e3.seed)) {
        revert UnfavorableSeed();
    }
}
```

The wrapper can calculate the scores of every operator and ticket number it controls and

revert unless those scores satisfy a chosen threshold.

Reverting the wrapper transaction also reverts:

- the requester's fee transfer;
- the increment of `nexte3Id`;
- Interfold's E3 state;
- the registry's committee request.

The attacker therefore pays only gas for an unsuccessful attempt and can retry in a later block, where a different `block.prevrandao` supplies another candidate seed.

Because the E3 identifier is restored on revert, the attacker repeatedly samples:

```
keccak256(nextBlockPrevrandao, sameE3Id)
```

until obtaining a seed under which its controlled operators receive unusually low scores.

This does not allow the requester to deterministically choose the final committee. Honest operators may submit unknown ticket numbers, and the attacker must already control active operators with sufficient historical ticket balances. Nevertheless, selectively accepting only favorable seeds changes the distribution from which committees are drawn. Instead of receiving a single unbiased committee, the requester repeatedly samples candidate committees until one exhibits unusually favorable representation among its controlled operators. This violates the protocol's assumption that committee selection is an unbiased random process, even though it does not deterministically guarantee committee majority.

The security consequence depends on the E3 application.

For applications whose security depends on unbiased committee selection—such as threshold decryption, private voting, sealed-bid auctions, and multiparty computation—this bias increases the probability that requester-controlled operators satisfy the application's trust assumptions. Even when the threshold is not reached, increased committee representation improves the requester's ability to withhold DKG participation, delay decryption, censor successful completion, or exploit additional honest failures to render the committee nonviable.

Below that threshold, favorable representation can still improve the requester's ability to:

- withhold DKG or decryption participation;
- cause timeout failures;
- censor successful completion;
- reduce the number of honest faults required to make the committee nonviable.

The issue is therefore a bias in committee randomness, not a direct proof that an arbitrary requester can obtain a committee majority without collateral.

Risk Analysis

- **Impact: High** — A requester can repeatedly sample candidate committee seeds until obtaining one that disproportionately favors operators it already controls, violating the protocol's unbiased committee-selection guarantee. Committee randomness is a fundamental security assumption of Interfold's threshold cryptographic design. By replacing a single unbiased committee draw with a requester-selected sample from many candidate committees, the attacker increases the probability of obtaining committees favorable to withholding, censorship, or threshold-collusion attacks. While grinding does not create additional operator weight or guarantee threshold control, it fundamentally biases the committee-selection process on which the protocol's security model relies.
- **Likelihood: Medium** — Each rejected seed costs only transaction gas because reverting restores the fee transfer and all E3 state. However, meaningful exploitation requires the requester to control multiple active, sufficiently funded operator identities and to retry across multiple blocks.
- **Severity: High**: **Interfold no longer provides unbiased committee selection.**

The protocol effectively changes from:

```
committee = RandomCommittee()
```

to

```
committee = BestOfN(RandomCommittee())
```

where **N is chosen by the requester**.

That is a **cryptographic security-property violation**

Recommendations

Generate committee randomness only after the request has become irreversible from the requester's perspective.

For example:

1. accept and persist the paid E3 request;
2. obtain randomness from a later block or asynchronous verifiable-randomness callback;
3. initialize committee selection only after that randomness is available.

The requester must not be able to observe a candidate seed and atomically revert the state transition that consumes it.

A commit-reveal construction is safe only if the requester cannot abort or selectively reveal after learning whether the resulting seed is favorable. Any requester contribution should be combined with an unpredictable value revealed after the request is irrevocably committed, with a defined fallback when the requester does not reveal.

Using only `block.prevrandao` in the same transaction remains grindable through conditional reversion. Restricting request callers or charging non-refundable retry fees could increase attack cost but would not restore unbiased randomness.

Interfold: Resolved with PR-1792 with added support for Arbitrum in PR-1803.

Zenith: Verified.

[H-6] Requester Loses Funds on an Accepted but Structurally Impossible Schedule

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target

- Interfold.request()
- InterfoldPricing.validateRequest()
- Cross-contract timing assumptions between Interfold and `CiphernodeRegistryOwnable`

Description

Interfold.request() accepts and charges for E3 schedules where the compute deadline expires before committee finalization is legally permitted.

(Interfold.sol#L297-L300) sets the compute deadline to:

```
inputWindow[1] + computeWindow
```

Separately, the registry sets the committee-submission deadline relative to the request time. Committee finalization is unavailable until that submission window has closed.

Consequently, the protocol can accept the contradictory relation:

```
computeDeadline < committeeDeadline
```

The lifecycle then requires two mutually incompatible conditions:

```
finalizeCommittee():
    block.timestamp > committeeDeadline

publishCiphertextOutput():
    block.timestamp <= computeDeadline
```

When `computeDeadline < committeeDeadline`, there is no timestamp at which the committee can first be finalized and the ciphertext can subsequently be published within the compute deadline.

However, InterfoldPricing.validateRequest() validates only the requester-facing input duration together with the compute and decryption windows against `maxDuration`. It does not account for the registry's protocol-controlled sortition window.

The PoC configures:

```
sortition submission window: 24 hours
input window end:            approximately request time + 20 seconds
compute window:              1 hour
```

The resulting compute deadline is approximately 23 hours earlier than the earliest legal committee finalization.

Despite this contradiction, the request succeeds and the full E3 fee is transferred from the requester. Operators can submit valid tickets, finalize the committee after the submission deadline, and publish the committee key. At this point the E3 reaches `KeyPublished`, but any call to `publishCiphertextOutput()` necessarily reverts with `CommitteeDutiesCompleted` because the compute deadline expired before finalization was possible.

The E3 must then terminate through `ComputeTimeout`.

Under the default failure allocation demonstrated by the PoC:

```
requester:     55%
honest nodes:  40%
protocol:       5%
```

The operators are compensated for committee-formation and DKG work they genuinely performed. Nevertheless, this work could never lead to a successful computation because the schedule was already impossible when Interfold accepted and charged for the request.

The requester therefore permanently loses 45% of the fee without ever having had a possible path to receive the requested result.

Risk Analysis

- **Impact: High** — A requester can permanently lose part of the E3 fee even though the requested computation had no possible successful execution path when it was accepted. Under the default failure allocation demonstrated by the PoC, the requester receives 55% while 40% is allocated to honest nodes and 5% to the protocol.
- **Likelihood: Medium** — The issue requires a short input/compute period relative to the registry's sortition window. Such combinations are plausible for applications requesting low-latency computations while the protocol uses a longer operator-submission window, but they are unlikely under carefully curated client presets.

Proof of Concept

POC File: Interfold.spec.ts (run with `npx hardhat test test/Interfold.spec.ts --grep "bug"`)

```
Running Mocha tests


  Interfold
    bug: Accepted impossible scheduling causes guaranteed fee loss
      ✔ accepts an E3 whose compute deadline expires before committee finalization is permitted (545ms)


  1 passing (546ms)
```

The test demonstrates that:

1. Interfold accepts and charges for the request.
2. `computeDeadline < committeeDeadline`.
3. Valid operators can submit sortition tickets.
4. The committee can be finalized only after the compute deadline.
5. The E3 can subsequently reach `KeyPublished`.
6. An otherwise valid ciphertext publication reverts with `CommitteeDutiesCompleted`.
7. The E3 is marked failed with `ComputeTimeout`.
8. Failure settlement returns less than the original payment to the requester.
9. The difference is allocated to honest operators and the protocol.

The decisive invariant is:

```
expect(computeDeadline).to.be.lt(committeeDeadline);
```

This proves structural impossibility independently of operator performance: even instantaneous DKG after committee finalization cannot make ciphertext publication occur before the already-expired compute deadline.

Recommendations

Validate the complete cross-contract schedule before accepting payment.

At minimum, require the compute deadline to occur after the earliest legal committee finalization:

```
uint256 committeeDeadline =
    block.timestamp +
    ciphernodeRegistry.sortitionSubmissionWindow();

uint256 computeDeadline =
    inputWindow[1] +
    _timeoutConfig.computeWindow;

require(
    computeDeadline > committeeDeadline,
    ComputeDeadlinePrecedesCommitteeFinalization(
        computeDeadline,
        committeeDeadline
    )
);
```

Interfold should preferably obtain the relevant timing value through a registry interface rather than duplicating the configuration.

A stronger policy may reserve additional time for DKG:

```
require(
    computeDeadline >
        committeeDeadline +
        _timeoutConfig.dkgWindow,
    InsufficientTimeForDKG()
);
```

This stronger condition is a protocol-design choice rather than the minimum security invariant. The DKG window is a maximum allowed duration, so requiring the entire window to fit may reject schedules that could succeed through faster DKG.

Interfold: Resolved with PR-1753.

Zenith: Verified.

4.3 Medium Risk

A total of 18 medium risk findings were identified.

[M-1] Requester-defined E3 programs can externalize undecryptable ciphertext failures onto committees

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/Interfold.sol#L318-L402
- packages/interfold-contracts/contracts/Interfold.sol#L637-L644
- packages/interfold-contracts/contracts/interfaces/IE3Program.sol#L8-L40
- packages/interfold-contracts/contracts/E3RefundManager.sol#L254-L305

Description:

Permissionless custom E3 programs are an explicit protocol design choice. The Q&A documents `registerE3Program()` as permissionless and enable-only, while `IE3Program` allows each program to define its application-specific computation and validation rules.

The issue is not that arbitrary programs can be registered. It is that application-defined output validity is also the only gate before the protocol assigns the committee a BFV decryption obligation and later attributes failure to the committee.

Any account can register a requester-controlled program:

```
function registerE3Program(IE3Program e3Program) external {
    require(
        !e3Programs[e3Program],
        ModuleAlreadyEnabled(address(e3Program))
    );
    e3Programs[e3Program] = true;
}
```

Its `validate()` function can return a legitimate, owner-configured encryption-scheme identifier. This selects genuine PK and decryption verifiers, but neither verifier checks the ci-

phertext when it is published.

After the committee completes DKG, the requester-controlled program can make its application-level `verify()` function accept an output and commitment pair for which no valid C6/C7 decryption witness exists. Interfold then advances the E3 into the committee's decryption phase:

```
e3s[e3Id].ciphertextOutput = ciphertextOutputHash;
e3s[e3Id].ciphertextCommitment = ciphertextCommitment;
_e3Stages[e3Id] = E3Stage.CiphertextReady;

success = e3.e3Program.verify(
    e3Id,
    ciphertextOutputHash,
    proof
);
require(success, InvalidOutput(ciphertextOutput));
```

A failed application proof reverts the entire transaction, so the ordering above is not itself problematic. The problem arises when a requester-defined verifier deliberately returns `true`: no protocol-owned, encryption-scheme-specific check establishes that the resulting decryption duty is satisfiable.

The PK verifier authenticates committee DKG, while the decryption verifier is invoked only during final plaintext publication. If no valid decryption witness exists, the one-shot ciphertext publication cannot be corrected and the E3 eventually reaches `DecryptionTimeout`.

`E3RefundManager` classifies `DecryptionTimeout` as ciphernode-payer:

```
if (
    reason == IInterfold.FailureReason.DecryptionTimeout ||
    reason == IInterfold.FailureReason.DecryptionInvalidShares ||
    reason == IInterfold.FailureReason.VerificationFailed
) {
    return FailurePayer.Ciphernodes;
}
```

The requester can therefore recover the complete fee after failure processing and claiming, while honest operators receive no fee-funded compensation for committee formation, DKG, retained secret state, and attempted decryption. The attacker pays transaction gas and temporarily locks the fee but can repeat the sequence to consume committee capacity.

This behavior is not covered by a documented trust assumption. The Q&A makes programs permissionless but does not state that every program is trusted to create satisfiable BFV duties, that nodes review or opt into programs, or that malformed requester output should be refunded as a node-attributable failure. The first-party Rust request model also omits the E3-program address, so nodes do not currently enforce a program-specific participation

allowlist.

This remains distinct from the unbound-commitment finding. Even if an E3 program authenticates both the ciphertext output and its commitment, a requester-controlled program can intentionally define proof semantics that admit an output outside the BFV decryption language.

The sequence does not automatically slash operators. Financial penalties still require separate valid evidence, attestations, and an enabled slash policy.

Recommendations:

It is recommended to separate application-specific output verification from protocol-level BFV well-formedness verification.

Before advancing to `CiphertextReady`, invoke an owner-configured verifier for the selected encryption scheme that establishes that the published output and commitment form a satisfiable decryption obligation. The verification should bind at least:

- the E3 and encryption-scheme identifiers;
- the committee public key or committee hash;
- the ciphertext output hash; and
- the proof-derived ciphertext commitment.

Requester-selected programs may additionally enforce application-specific computation semantics, but must not be the sole authority deciding whether honest nodes receive a decryptable task.

If protocol-owned well-formedness verification is unavailable, do not classify an unproven decryption obligation as a ciphernode fault. Preserve compensation for completed committee formation and DKG work through a nonrefundable base fee or classify the failure as requester-attributable unless valid node-fault evidence exists.

As a weaker alternative, nodes could explicitly opt into reviewed E3 programs. That would require including the program identity in the first-party Rust event model and enforcing the allowlist before sortition. Restricting on-chain registration should be considered only if the documented permissionless-program design is intentionally abandoned.

Add a regression test using a self-registered program whose `verify()` always returns `true`. Publishing an output without a valid scheme-level decryption witness must revert without consuming the one-shot ciphertext slot or creating a ciphernode-attributable timeout.

Interfold: Resolved with `@2c18861769 ...`

Zenith: Verified.

[M-2] Recipient selection rewards slashed or later-expelled operators

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L763-L849
- packages/interfold-contracts/contracts/Interfold.sol#L730-L748
- packages/interfold-contracts/contracts/E3RefundManager.sol#L172-L242
- packages/interfold-contracts/contracts/E3RefundManager.sol#L485-L607

Description:

Recipient selection relies on current committee membership without preserving slash provenance or accounting for unresolved same-E3 slashes. This produces two related allocation errors.

First, a valid policy may impose a ticket penalty with `affectsCommittee=false` and `failureReason=0`. The penalty is executed, but expulsion is skipped:

```
if (p.affectsCommittee) {
    dependencies.registry.expelCommitteeMember(
        p.e3Id,
        p.operator,
        p.reason
    );
}
```

The target therefore remains in the active committee. When its ticket proceeds are settled, the refund manager reads that active committee and distributes the funds without excluding the operator from whom they originated:

```
(address[] memory nodes, ) = ICiphernodeRegistry(
    _e3PolicySnapshots[e3Id].registry
).getActiveCommitteeNodes(e3Id);

_creditNodeSlashedClaims(e3Id, token, nodes, toHonestNodes);
```

After failure, the target can recover approximately `1 / n` of its own penalty. After success, it can receive its share of the configured node allocation. Continuing to receive ordinary E3 rewards may be intentional for a non-expelling policy, but receiving proceeds from its own penalty weakens the configured slash.

Second, an expelling proposal may remain unresolved while the E3 independently fails. For requester-attributable failures such as `ComputeTimeout`, processing the failure before slash execution stores the target as an honest node and creates an immutable fee-funded claim. Executing the slash afterward expels the target but does not revoke that claim. Executing the same slash first excludes the target and reallocates its share among the remaining nodes.

The same ordering dependency affects other slashed funds that are already pending settlement: settling them before expulsion credits the target, while settling after expulsion excludes it. This does not apply to the target's own expelling slash because `_executeSlash` performs expulsion before routing that slash's proceeds. Ciphernode-attributable failures also create no base fee-funded node reward, although the separate slashed-fund ordering issue remains.

A focused reproduction confirmed that a three-member committee returned one-third of a completed non-expelling ticket slash to its target. It also confirmed that reversing failure processing and expelling-slash execution changed the target's fee-funded claim from nonzero to zero despite the same final E3 and slash state.

Recommendations:

It is recommended to preserve the proposal ID, target, token, and amount when routing slashed funds.

For completed non-expelling slashes, exclude the target only from distributions funded by its own penalty. This need not remove the target from ordinary E3 rewards or unrelated slash distributions.

For unresolved expelling proposals, escrow only the accused operator's prospective share. Release it if the operator is cleared, or reallocate it after the slash executes. Do not block settlement for other recipients while an appeal remains unresolved.

Use one E3-scoped recipient-entitlement rule for fee rewards and slashed funds so transaction ordering cannot change the final allocation.

Interfold: Resolved with `@86b2ab1ce3 ...`

Zenith: Verified.

[M-3] Provisional membership can authorize invalid slashes and block failure settlement

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L913-L973
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L1108-L1140
- interfold-contracts/contracts/slashing/SlashingManager.sol#L455-L746
- interfold-contracts/contracts/E3RefundManager.sol#L527-L538

Description:

`_insertTopN` marks provisional candidates `Active` before committee finalization. Because the membership views do not check the committee stage, provisional voters gain Lane A attestation authority and provisional targets become slash-eligible.

```
if (top.length < cap) {
    top.push(node);
    c.scoreOf[node] = score;
    c.memberStatus[node] = ICiphernodeRegistry.MemberStatus.Active;
    return true;
}
```

An atomic `affectsCommittee=true` slash reverts before finalization when expulsion fails. However, a deferred proposal can execute after its target or voters have been displaced, while an `affectsCommittee=false` policy can slash a provisional target before finalization. Lane B can similarly lock and later slash provisional targets.

Lane A requires `M` signed attestations and Lane B requires `SLASHER_ROLE`. The committed local Lane A policies use the protected atomic configuration, making exploitation conditional on another configuration permitted by `setSlashPolicy`.

Before finalization, `activeCount` is zero while provisional entries are marked `Active`. `getActiveCommitteeNodes` therefore allocates empty arrays and writes into them, causing an out-of-bounds panic.

```
nodes = new address[](c.activeCount);
scores = new uint256[](c.activeCount);

for (uint256 i = 0; i < c.topNodes.length; ++i) {
    address node = c.topNodes[i];
    if (c.memberStatus[node] == ICiphernodeRegistry.MemberStatus.Active) {
        nodes[idx] = node;
        scores[idx] = c.scoreOf[node];
        idx++;
    }
}
```

If a non-expelling ticket slash reaches `E3RefundManager` before committee formation fails, `_settleSlashedFunds` encounters this panic. When the slash token matches the E3 fee token—as in the standard deployment—the revert rolls back `processE3Failure` and blocks the requester's refund. With different tokens, the refund can complete, but the slash remains unsettled.

Both the displaced-member slash and matching-token refund failure were reproduced on the current codebase.

Recommendations:

It is recommended to keep provisional candidate state separate and assign `MemberStatus.Active` only during successful finalization. Require `CommitteeStage.Finalized` when creating Lane A or Lane B proposals, validate targets against that finalized committee, and require Lane A voters to be active finalized members.

Do not require those participants to remain active at deferred execution, since a later expulsion should not invalidate a proposal validly authorized after finalization.

`getActiveCommitteeNodes` should return empty arrays when committee formation never finalized and derive the result length through a first pass over active entries. Calculate the requester's base refund independently and defer failed slash settlement to `settleSlashedFunds` or terminal treasury routing.

Interfold: Resolved with @6d476473b1 ...

Zenith: Verified.

[M-4] Committee options are not bound to supported circuits and gas limits

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/Interfold.sol#L95-L109
- interfold-contracts/contracts/Interfold.sol#L997-L1008
- interfold-contracts/contracts/lib/InterfoldPricing.sol#L26-L30
- interfold-contracts/contracts/lib/InterfoldPricing.sol#L235-L255
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L589-L632
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L1084-L1140
- crates/evm/src/interfold/events.rs#L28-L45
- crates/zk-helpers/src/ciphernodes_committee.rs#L37-L121

Description:

Each `CommitteeSize` enum slot can be assigned any `[M,N]` row satisfying only `0 < M ⩽ N ⩽ 256`. The contract does not require the configured row to match the committee parameters supported by the corresponding nodes, circuits, and verifiers.

```
uint32 internal constant MAX_COMMITTEE_SIZE = 256;

function setCommitteeThresholds(
    CommitteeSize size,
    uint32[2] calldata threshold
) external onlyOwner {
    InterfoldPricing.validateCommitteeThresholds(
        threshold,
        _pricingConfig.minCommitteeSize,
        _pricingConfig.minThreshold
    );
    committeeThresholds[size] = threshold;
}
```

The current Rust and circuit stack supports fixed configurations:

- Minimum: `T=1, H=2, N=3`
- Micro: `T=4, H=5, N=9`
- Small: `T=9, H=10, N=19`

When processing `E3Requested`, Rust derives `T` and `N` exclusively from the enum:

```
let committee_size = match self.0.e3.committeeSize {
    0 ⇒ CiphernodesCommitteeSize::Minimum,
    1 ⇒ CiphernodesCommitteeSize::Micro,
    2 ⇒ CiphernodesCommitteeSize::Small,
    _ ⇒ bail!(/* ... */),
};

let committee = committee_size.values();
let threshold_m = committee.threshold; // circuit T
let threshold_n = committee.n;
```

Although the configured `[M,N]` row is emitted separately through `CommitteeRequested`, sortition and DKG metadata are derived from `E3Requested`. Changing configured `N` therefore changes the registry's committee capacity and finalization requirement while nodes continue selecting members and generating proofs for the enum's canonical `N`.

The first configured value is also semantically overloaded:

- Production configuration sets on-chain `M=H=T+1`.
- Rust's misleadingly named `threshold_m` contains circuit threshold `T`.
- Solidity uses `M` for post-expulsion viability and the Lane-A attestation quorum.
- Pricing treats `M` as the number of decryption participants.

An arbitrary `M` can therefore underprice or overprice work, accept an attestation quorum below the intended honest roster, or keep an E3 viable after fewer than `H` decryption participants remain.

Committee configurations also determine verifier layouts. However, `Interfold` stores only one PK verifier and one decryption verifier per encryption-scheme identifier. The repository configures all three committee rows while registering one active committee-specific verifier pair. Once enough operators exist for another configured size, a requester can select an option whose proofs are incompatible with the active verifier.

The accepted `MAX_COMMITTEE_SIZE` also exceeds the demonstrated gas capacity of both committee selection and finalization.

Every ticket submission calls `_insertTopN()`, which scans up to `N` stored candidates to identify the current worst score. The implementation's own comment estimates acceptable operation for current parameters but warns that it "will not scale to `N > ~50`." Despite that warning, threshold validation permits `N=256`.

Finalization adds a separate quadratic nested-loop sort directly against storage:

```
for (uint256 i = 0; i < len; ++i) {
    for (uint256 j = i + 1; j < len; ++j) {
        address left = c.topNodes[i];
        address right = c.topNodes[j];
        if (right < left) {
            c.topNodes[i] = right;
            c.topNodes[j] = left;
        }
    }
}
```

Each swap performs two nonzero-to-nonzero storage writes. A focused harness reproducing the same sorting algorithm and compiler settings measured:

- approximately 24.0M gas for 256 already-sorted addresses;
- approximately 39.2M gas for one random list containing 15,343 swaps; and
- approximately 55.5M gas for a reverse-sorted list.

These figures measure only the sorting function, excluding the remaining work performed by `finalizeCommittee()`. Every measured 256-member ordering nevertheless exceeds Ethereum's $2^{24} = 16{,}777{,}216$ per-transaction gas cap introduced by EIP-7825. This cap applies independently of Ethereum's 60M block gas limit and is not modeled by the local development configuration.

A permitted large committee can therefore accept ticket submissions but never finalize successfully. The requester's fee is not permanently trapped: after the committee deadline, the E3 can instead be marked failed and processed through the refund path. However, the requested computation cannot complete, operators waste ticket-submission and failed-finalization gas, and the requester must wait for failure processing and claim recovery.

Changing arbitrary rows requires the owner, and a 256-member request requires at least 256 active operators. Once an unsupported row is configured, however, requesters can trigger E3s that cannot complete because of inconsistent off-chain semantics, verifier incompatibility, or transaction gas limits. The demonstrated consequences are incorrect pricing, wasted node work, rejected proofs, and timeouts—not invalid-proof acceptance or direct theft.

Recommendations:

It is recommended to replace independently interpreted parameter and committee indexes with an immutable, versioned cryptographic configuration binding:

- the BFV parameter-set hash;
- circuit threshold `T`;
- required honest/decryption roster `H`;
- total committee size `N`;

- circuit and verification-key version; and
- compatible PK and decryption verifier contracts.

```
struct CryptoConfig {
    bytes32 paramSetHash;
    uint32 t;
    uint32 h;
    uint32 n;
    bytes32 circuitVersion;
    IPkVerifier pkVerifier;
    IDecryptionVerifier decryptionVerifier;
    bool enabledForNewRequests;
}
```

Configuration contents and verifier bindings should be append-only. Governance may disable a configuration for new requests, but existing E3s must retain their snapshotted configuration identifier and verifier addresses. Until dynamic circuits exist, only exact canonical configurations with deployed matching verifiers should be enabled.

The protocol specification should separately define whether `T` or `H` governs decryption pricing, committee viability, and slashing quorum instead of overloading a generic `M`.

Replace the quadratic storage sort with an in-memory `O(N log N)` sort followed by one bounded storage write-back, or maintain bounded canonical ordering during `_insertTopN()`. Set `MAX_COMMITTEE_SIZE` to the largest canonical `N` for which the complete ticket-submission and `finalizeCommittee()` paths have been benchmarked under the target chain's per-transaction gas cap. Do not infer the safe bound solely from the block gas limit.

Interfold: Resolved with `@93d20380c3 ...`

Zenith: Verified.

[M-5] Pricing omits pre-input availability

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/Interfold.sol#L243-L271
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L257-L283
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L328-L402
- packages/interfold-contracts/contracts/Interfold.sol#L918-L934

Description:

Request validation includes the delay from the request until the end of the input window when enforcing `maxDuration`.

```
uint256 totalDuration = inputWindow[1] -
    nowTs +
    computeWindow +
    decryptionWindow;

if (totalDuration >= maxDuration)
    revert IInterfold.InvalidDuration(totalDuration);
```

Fee calculation instead charges availability based on the sortition window, the length of the input window, and weighted timeout windows. It does not include the delay between the request and `inputWindowStart`.

```
uint256 duration = sortitionWindow +
    inputWindowEnd -
    inputWindowStart +
    weightedTimeoutsBps /
    uint256(BPS_BASE);

baseFee += pc.availabilityPerNodePerSec * n * duration;
```

Moving both input-window timestamps forward by the same amount therefore leaves the quote unchanged. A requester can place a short input window near the configured `max-`

`Duration` while paying the same availability component as an otherwise identical request whose input window begins shortly after submission.

The delay creates a real committee obligation. Sortition and DKG begin immediately after the request. Once the committee key is published, members retain persistent secret state and must remain available for decryption. A `KeyPublished` E3 cannot time out until `inputWindowEnd + computeWindow`.

```
if (stage == E3Stage.KeyPublished)
    return (d.computeDeadline, FailureReason.ComputeTimeout);
```

The first-party Rust client also counts the E3 as an active job from committee publication until completion or failure, reducing that client's locally available sortition tickets. This capacity rule is not enforced by Solidity, but the persistent state and underpayment remain regardless of client implementation.

A contract-fixture reproduction using mock verification compared two equal one-second input windows starting shortly after the request and approximately 29.9 days later. Both received exactly the same quote. The delayed request succeeded, its committee key was published immediately, and no failure condition became available until near the delayed compute deadline.

Current deployment scripts and records configure `maxDuration` to 30 days. Governance can increase it up to the contract's 365-day hard cap. The omitted interval can therefore approach the configured `maxDuration`; it is not unconditionally one year.

Repeated delayed requests can underpay committee availability, retain secret job state, and consume honest first-party operators' local sortition capacity. This is an economic and availability issue rather than direct theft.

Recommendations:

It is recommended to define when billable availability begins. The protocol can reserve capacity from request time or charge idle retention beginning at expected committee/key publication. In either case, the fee must increase when an equal-length input window is moved further into the future.

If capacity is reserved from request time, preserve the existing sortition, DKG, and input-window charge while enforcing `inputWindowEnd - requestTime` as a lower bound. Perform the calculation in BPS-scaled units and divide once to preserve the current rounding behavior.

```
uint256 inputWindowLength = inputWindowEnd - inputWindowStart;

uint256 preComputeBps =
    (sortitionWindow + inputWindowLength) * BPS_BASE +
    tc.dkgWindow * uint256(pc.dkgUtilizationBps);
```

```
uint256 reservedThroughInputEndBps =
    (inputWindowEnd - nowTs) * BPS_BASE;

uint256 durationBps =
    Math.max(preComputeBps, reservedThroughInputEndBps) +
    tc.computeWindow * uint256(pc.computeUtilizationBps) +
    tc.decryptionWindow * uint256(pc.decryptUtilizationBps);

uint256 duration = durationBps / BPS_BASE;
```

If availability should begin at expected key publication instead, add only the positive idle interval after expected sortition and DKG completion.

Pass `block.timestamp` into the quote calculation. `request` should call `validateRequest` before calculating the quote, and public `getE3Quote` should apply equivalent timestamp validation. This prevents malformed or expired windows from causing arithmetic panics after timestamp subtraction.

If long-scheduled E3s are unnecessary, an alternative is to bound `inputWindowStart - requestTime`.

Add tests confirming that:

- equal-length windows become more expensive as their start is delayed;
- near-term requests preserve the existing sortition and weighted-timeout charge;
- maximum-delay requests pay for the reserved interval;
- invalid or expired windows revert with the intended custom errors;
- BPS-weighted duration is rounded only once.

Interfold: Resolved with `@46a01ad11f ...`

Zenith: Verified.

[M-6] Solidity and Rust use inconsistent eligibility and ticket-capacity snapshots for sortition

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- interfold-contracts/contracts/Interfold.sol#L302-L310
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L543-L582
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L1028-L1056
- interfold-contracts/contracts/registry/BondingRegistry.sol#L531-L556
- interfold-contracts/contracts/registry/BondingRegistry.sol#L747-L755
- interfold-contracts/contracts/registry/BondingRegistry.sol#L1010-L1055
- crates/evm/src/interfold/events.rs#L88-L96
- crates/sortition/src/sortition/selection_backend.rs#L84-L125

Description:

Committee sortition is not derived from one immutable, consistently interpreted per-E3 state specification. Solidity combines historical and live values:

```
require(
    isEnabled(msg.sender) && _bondingFor(e3Id).isActive(msg.sender),
    NodeNotEligible()
);

uint256 ticketBalance = e3Bonding.getTicketBalanceAtBlock(
    node,
    c.requestBlock - 1
);
uint256 ticketPrice = e3Bonding.ticketPrice();
uint256 availableTickets = ticketBalance / ticketPrice;
```

Ticket balance is read at `requestBlock - 1`, while enabled status, active status, and ticket price are read when the ticket is submitted.

Rust uses another state boundary. Although the emitted E3 data contains `requestBlock`, the Rust EVM conversion omits it. Rust therefore constructs ticket sets from its current event-

derived node state:

```
let count = node_state.available_tickets(&addr_str);
let tickets = (1..=count).map(|i| Ticket { ticket_id: i }).collect();
```

Rust's separate active-job subtraction is addressed by another finding. The relevant issue here is that Rust's current ticket balance and activity can differ from the historical balance and eligibility assumptions enforced by Solidity.

Selective post-request activation

A registered operator can unbond enough license collateral to become inactive while remaining registered and enabled in the ciphernode registry. This contradicts the `IBondingRegistry` documentation, which states that `unbondLicense()` reverts while the operator remains registered.

The operator can prepare multiple dormant identities over time by:

1. bonding the license collateral required to register an identity;
2. registering the identity and retaining its ticket balance;
3. unbonding and waiting through the exit delay;
4. reusing the released license collateral to prepare another identity.

One license position cannot activate several identities simultaneously, and every identity still needs its own ticket capital. Nevertheless, after this delayed preparation, the operator can observe an E3 seed, determine which dormant identity has the most favorable historical ticket, and bond the license to that identity. Solidity accepts the submission because it combines the identity's historical ticket balance with its newly restored live activity.

Honest Rust sortition excludes the identity while it is inactive, so the operator can submit a ticket that was absent from the candidate set used by honest clients.

The stored registry root does not prevent this behavior. It represents registered membership rather than active eligibility, and `submitTicket` checks the current `isEnabled` mapping instead of requiring an eligibility proof against the stored root. The request-time active-operator check records only a count, not the identities contributing to that count.

Mutable ticket price

The owner can change `ticketPrice` during the submission window. A decrease creates additional valid ticket numbers for historical balances, while an increase invalidates tickets not yet submitted.

Previously accepted tickets are not revalidated. One E3 can consequently contain submissions admitted under different ticket prices. Eligibility-version invalidation does not prevent this because operators can refresh their current status and submit under the new price.

This branch requires the trusted owner and is therefore primarily a configuration and integration hazard rather than a permissionless attack.

Same-timestamp Rust/Solidity divergence

A balance change followed by an E3 request in the same Ethereum block shares timestamp `T`.

Ordered event processing applies the balance update before Rust handles `E3Requested`, so Rust uses the updated balance. Solidity records `requestBlock = T` but subsequently reads voting power at `T - 1`, excluding the update.

If a deposit increased the balance, Rust may select a best ticket whose number exceeds the historical on-chain capacity. That submission reverts. The increased balance does not make every selected ticket invalid: rejection occurs only when the selected ticket number exceeds the historical capacity.

A partial withdrawal creates the inverse discrepancy. Rust may omit an operator or ticket that Solidity would still accept from the historical checkpoint. A same-timestamp activation can also contribute to `numActiveOperators` even though the associated ticket weight is excluded on chain.

Only operators selected by Rust generate and submit tickets. Operators omitted by Rust remain silent even when Solidity would accept them. Rust selects `N` operators plus a safety buffer, so an isolated discrepancy will not ordinarily fail formation. Committee failure requires enough invalid or omitted submissions to overcome that buffer and leave fewer than `N` valid submissions.

Post-request reactivation and mutable ticket-price behavior were reproduced. The same-timestamp branch follows from the timestamp checkpoint and ordered event-processing semantics but was not covered by the cited Solidity reproductions.

The consequences are biased committee selection, reduced dual-collateral Sybil resistance, disagreement between honest Rust clients and Solidity, and possible committee-formation failure. Solidity remains authoritative and rejects tickets outside its calculated capacity, so the issue does not directly permit an invalid finalized committee or theft.

Recommendations:

It is recommended to define an explicit, immutable per-E3 sortition specification shared by Solidity and Rust.

At minimum:

1. Preserve `requestBlock` in the Rust `E3Requested` representation.
2. Make Rust derive ticket balances from exactly the historical timepoint used by Solidity.
3. Snapshot `ticketPrice` for each E3 when the request is executed and use that value throughout submission.
4. Make the request-time eligible count and individual eligibility checks use the same eligibility definition.
5. Prevent an operator that was ineligible at the defined request boundary from becoming eligible for that E3 after observing its seed.

The balance and policy boundaries do not necessarily need to be identical if the distinction is intentional. For example, the protocol can use balance and eligibility checkpoints at `requestBlock - 1` while transaction-order-exactly snapshotting the current ticket price during the request. However, that complete specification must be immutable and consumed identically by the request-time count, Rust selection, and Solidity validation.

One possible implementation is:

```
uint256 timepoint = committee.requestBlock - 1;
uint256 price = sortitionTicketPrice[e3Id];

require(
    bonding.wasEligibleAt(node, timepoint),
    NodeNotEligibleAtRequest()
);

uint256 ticketBalance =
    bonding.getTicketBalanceAtBlock(node, timepoint);
uint256 availableTickets = ticketBalance / price;
```

The corresponding request check should use `numEligibleAt(timepoint)` or an equivalent committed eligible set. A policy-version snapshot alone is insufficient because it does not identify which operators satisfied that policy.

If current activity is needed for liveness, enforce it in addition to historical eligibility:

```
require(
    bonding.wasEligibleAt(node, timepoint) &&
        bonding.isActive(node),
    NodeNotEligible()
);
```

Alternatively, use an authoritative eligibility commitment. Its leaves must be available to Rust, and Solidity must verify eligibility and balance proofs during submission. The existing registered-node root, an informational root, or a caller-supplied root is insufficient.

Do not change Solidity to query voting power at the request timestamp. A later query at timestamp `T` can include checkpoints written after the request but within the same timestamp, reopening same-block manipulation.

Also reconcile `unbondLicense()` with its documented behavior. Requiring deregistration before license unbonding directly closes the selective-license-activation path. If inactive registered identities are intentional, document that behavior and enforce historical per-E3 eligibility instead.

Add regression tests covering:

- post-seed reactivation;
- sequential license reuse across dormant identities;
- ticket-price increases and decreases during submission;
- mixed-price submissions;
- deposits, reactivations, and partial withdrawals preceding a same-timestamp request;
- agreement between Rust and Solidity at the selected historical boundary; and
- consistency between the request-time eligible count and individual eligibility.

Interfold: Resolved with `@1641fa722e ...`

Zenith: Verified.

[M-7] Banned operator remains active and can join new committees after governance ban

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- BondingRegistry.registerOperator()
- CiphernodeRegistryOwnable.sol.submitTicket()

Description

`SlashingManager.confirmBan()` sets `banned[node] = true`. This flag is checked only in BondingRegistry.registerOperator(), which iterates `_authorizedSlashingManagers` and calls `isBanned()` for each. An operator that is ALREADY registered when banned bypasses this check entirely.

submitTicket() checks only `isEnabled(msg.sender) && bondingFor(e3Id).isActive(msg.sender)`. Neither `isEnabled` nor `isActive` consults the ban status. The banned operator's `ciphernodeEnabled` flag remains true, their `registered` flag remains true, and their `active` status is unaffected.

The result: a banned operator can submit tickets to committees for E3s created AFTER the ban, win committee seats, participate in DKG and decryption, and receive rewards — exactly the behavior the ban was intended to prevent.

Risk Analysis

- **Impact:** Medium — A governance-banned operator continues to participate in new E3 committees, receive rewards, and vote on accusations. The ban achieves no operational effect until the operator voluntarily deregisters and re-registers.
- **Likelihood:** Medium — Occurs whenever governance bans an already-registered operator. The ban is a documented governance action (`confirmBan`), but its expected consequence (operator exclusion) does not materialize.

Proof of Concept

Copy-paste in `Interfold.spec.ts` and run with `npx hardhat test test/Interfold.spec.ts --grep "bug"`

```
Running Mocha tests


  Interfold
    bug: Banned operator remains eligible for new committees
      ✔ banned operator successfully submits ticket to post-ban E3 (453ms)


  1 passing (454ms)
```

```
describe("bug: Banned operator remains eligible for new committees",
    function () {
  it("banned operator successfully submits ticket to post-ban E3",
    async function () {
    const {
      interfold,
      usdcToken,
      ciphernodeRegistryContract: registry,
      bondingRegistry,
      request,
      owner,
      operator1,
      operator2,
      operator3,
    } = await loadFixture(setup);

    // Verify operator1 is active
    expect(await bondingRegistry.isActive(operator1.address)).to.be.true;
    expect(await registry.isEnabled(operator1.address)).to.be.true;

    // Ban operator1 via SlashingManager
    // (This requires SlashingManager fixture access — adjust to your
    fixture pattern)
    // await slashingManager.connect(owner).confirmBan(operator1.address);
    // expect(await slashingManager.isBanned(operator1.address)).to.be.true;

    // After ban: still active, still enabled
    expect(await bondingRegistry.isActive(operator1.address)).to.be.true;
    expect(await registry.isEnabled(operator1.address)).to.be.true;
    expect(await
    registry.isCiphernodeEligible(operator1.address)).to.be.true;

    // Create a NEW E3 after the ban
    await makeRequest(interfold, usdcToken, {
      ...request,
```

```
      inputWindow: [(await time.latest()) + 200, (await time.latest())
    + 500],
    });

    // Banned operator can still submit a ticket
    await registry.connect(operator1).submitTicket(0, 1);

    // Banned operator is on the committee
    // (Would need to finalize and check membership)
  });
});
```

Note: The PoC requires `SlashingManager` fixtures to call `confirmBan`. The core assertion is that after `isBanned(operator) = true`, `isActive(operator)` and `isEnabled(operator)` remain true and `submitTicket` succeeds.

Recommendations

A ban should synchronously deactivate the operator. Either:

1. `confirmBan` calls `bondingRegistry.deregisterOperator(node)` and `registry.removeCiphernode(node)` directly.
2. `isActive()` and `isEnabled()` check the canonical ban state before returning true.
3. `submitTicket()` adds an explicit `!isBanned(msg.sender)` check.

Interfold: Resolved with PR-1752.

Zenith: Verified.

[M-8] Unbound public-key bytes can be front-run to poison one-shot committee publication

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L301-L369
- packages/interfold-contracts/contracts/verifiers/bfv/BfvPkVerifier.sol#L103-L156
- packages/interfold-contracts/contracts/lib/DkgFoldAttestationLib.sol#L74-L103

Description:

`publishCommittee` is permissionless and accepts serialized `publicKey` bytes alongside the C5-proven semantic `pkCommitment`. The supplied bytes are omitted from both proof verification and fold-attestation signatures.

```
// publishCommittee
_verifyAndStoreDkgAnchors(
    e3Id,
    e3,
    roots[e3Id],
    c.topNodes,
    pkCommitment,
    committeeHash,
    proof,
    dkgAttestationBundle
);

emit CommitteePublished(
    e3Id,
    c.topNodes,
    publicKey,
    pkCommitment,
    proof
);

// _verifyAndStoreDkgAnchors
e3.pkVerifier.verify(
```

```
    e3Id,
    committeeRoot,
    sortedNodes,
    pkCommitment,
    committeeHash,
    proof
);
```

The fold attestations likewise omit the transported key:

```
keccak256(
    abi.encode(
        TYPEHASH,
        e3Id,
        partyId,
        skAggCommit,
        esmAggCommit
    )
);
```

An observer of a legitimate public transaction can copy its valid proof, `pkCommitment`, and attestation bundle, replace only `publicKey`, and submit the modified transaction with higher priority. The replacement may be empty or malformed bytes for denial of service, or a valid attacker-controlled BFV key against clients that fail to validate the commitment.

If the modified transaction executes first:

1. The correct proven `pkCommitment` and DKG anchors are stored.
2. Interfold advances the E3 to `KeyPublished`.
3. `CommitteePublished` emits the substituted bytes.
4. The legitimate transaction reverts because `c.publicKey` is already nonzero.

This sequence was reproduced against the fully wired contract fixture: an unrelated outsider successfully published empty `publicKey` bytes with an unchanged valid mock proof and bundle, the registry stored the expected commitment, Interfold entered `KeyPublished`, and the subsequent honest publication was rejected. The production verifier does not change this result because its interface also omits `publicKey`; an attacker reuses the already-valid production proof without modifying it.

Existing codebase protections do not prevent the attack:

- The Rust writer submits only when the local node is the active aggregator, but this is off-chain policy and cannot restrict direct contract calls.
- The manual Hardhat task rejects empty keys, but callers can invoke the contract directly.

- The production writer uses an ordinary provider `send()` path; no private-order-flow requirement is enforced.
- The C5 verifier checks `pkCommitment`, recursive verification-key hashes, and committee binding, but not the serialized bytes.
- Fold attestations authenticate DKG anchors, not the serialized aggregate key.
- Client-side validation happens only after the one-shot publication has succeeded.

The Rust indexer and default React client correctly reject mismatched bytes. However, there is no supported on-chain correction or republication path. Event-only workflows therefore cannot recover through the normal protocol flow and may miss the input window unless they obtain the correct key from another source.

The correct key is not cryptographically destroyed. It remains available to ciphernodes through the gossiped `PublicKeyAggregated` artifact and may be recovered from legitimate transaction calldata or recomputed from public shares. These are manual or off-chain recovery paths that ordinary clients do not implement.

If no useful ciphertext output is subsequently published, anyone can call `markE3Failed` after `computeDeadline`, producing a requester-attributed `ComputeTimeout`. Under the current default refund allocation, 40% of the fee pays completed work, 5% goes to the protocol, and 55% is refunded.

The TypeScript SDK exposes commitment validation separately rather than enforcing it in its low-level encryption methods. A non-validating integration may therefore encrypt and publish private inputs under an attacker-controlled valid key, allowing the attacker to decrypt them. This conditional confidentiality impact does not apply to integrations such as CRISP or the default React client that enforce semantic key-commitment binding.

Private transaction submission can reduce transaction visibility, but it does not repair the missing protocol binding and is not specified as a required security assumption.

Recommendations:

It is recommended to prevent one-shot publication from accepting unauthenticated public-key transport bytes.

The most practical solution is to separate proof-backed committee publication from key transport:

1. Finalize the committee using the proven `pkCommitment`, proof, and DKG anchors.
2. Allow repeatable, permissionless submission of bounded public-key candidates.
3. Require clients to ignore invalid candidates and accept a candidate only when its recomputed semantic commitment equals the on-chain `pkCommitment`.
4. Do not let an invalid candidate consume a final transport slot.

This preserves permissionless liveness without requiring Solidity to deserialize and validate the approximately 9 KB BFV key. Candidate length should be bounded, and duplicate candidate hashes may be suppressed to limit event spam.

Alternatively, add a post-C5 H-party EIP-712 attestation over at least:

```
(
    chainId,
    registry,
    e3Id,
    pkCommitment,
    keccak256(publicKey)
)
```

Each signer must first semantically validate that the serialized key corresponds to `pkCommitment`. The registry should verify the required canonical honest-party signers and the supplied byte hash before completing the one-shot publication.

If proof-level binding is chosen, the circuit must prove that the serialized bytes—or a formally defined canonical representation—encode the same aggregate key proven by C5. Merely exposing a caller-supplied `keccak256(publicKey)` as a public input would not establish this relationship.

Requiring a nonempty key or restricting the transaction sender alone is insufficient: a caller can supply a different valid key, and publisher restriction would weaken the intended permissionless takeover behavior.

As defense in depth, add a high-level SDK encryption API that obtains the E3 commitment and validates the key automatically before encryption. Preserve semantic validation because the current BFV decode/re-encode process is not byte-stable.

Add a regression test that reuses a valid proof and attestation bundle while changing only `publicKey`. The call must either revert without consuming publication state or leave a supported path for a later valid key candidate.

Interfold: Resolved with `@6e37b1675f ...`

Zenith: Verified.

[M-9] Committee-affecting slash policies can leave nonterminal E3s below threshold

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L330-L361
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L793-L815
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L878-L910
- packages/interfold-contracts/contracts/Interfold.sol#L405-L500

Description:

The protocol's stated committee invariant requires a nonterminal E3 to fail rather than continue when active committee membership falls below `thresholdM`.

When a committee-affecting slash executes, the registry expels the operator, decrements `activeCount`, and reports the resulting viability:

```
c.memberStatus[node] = ICiphernodeRegistry.MemberStatus.Expelled;
c.activeCount--;

bool viable = activeCount >= thresholdM;
emit CommitteeViabilityUpdated(e3Id, activeCount, thresholdM, viable);
```

`SlashingManager` receives the new count but notifies Interfold only when the proposal contains a nonzero failure reason:

```
if (activeCount < thresholdM && p.failureReason > 0) {
    try dependencies.interfoldContract.onE3Failed(
        p.e3Id,
        p.failureReason
    ) {
        // ...
    } catch {
        emit RoutingFailed(p.e3Id, 0);
    }
}
```

Two paths can therefore leave a nonterminal E3 below threshold without transitioning it to `Failed`.

First, policy validation permits:

```
affectsCommittee = true
failureReason = 0
```

Such a policy deliberately skips failure notification after expulsion. Repository tests use this combination, while the local-development policy configuration helper assigns nonzero supplier-side reasons. No production slash-policy configuration was verified.

Second, a policy may contain a nonzero reason, but the catch-all handler suppresses any unexpected failure from `onE3Failed`. The expulsion and reduced `activeCount` then remain committed while Interfold stays in its previous stage.

Before key publication, insufficient active attestations may prevent progress and eventually cause a timeout. After key publication, ciphertext and plaintext publication do not independently enforce current committee viability. Decryption proofs remain tied to previously recorded DKG anchors rather than current member activity. Expelled operators retain their cryptographic knowledge and may continue cooperating, so a valid proof can complete an E3 despite the active committee being below the intended threshold.

This does not automatically disclose encrypted inputs. The confidentiality risk arises because the protocol may continue accepting ciphertext under a committee key after its assumed honest-participant threshold no longer holds.

If every member is expelled under a zero-reason policy, a valid plaintext can still complete the E3. Reward distribution then observes no active recipients and returns the complete fee to the requester, providing both the result and a full refund. This is a downstream economic consequence of the same viability gap.

No unprivileged caller can configure slash policy, and valid slashing or privileged evidence is still required to expel members. The issue therefore depends on governance configuration or an unexpected callback failure. Expulsion after an E3 reaches `Complete` or `Failed` should remain permitted without retroactively changing its terminal state.

Recommendations:

It is recommended to enforce below-threshold failure atomically for every nonterminal E3.

Introduce a dedicated supplier-side failure reason such as `CommitteeViabilityLost` and map it to the correct refund payer. Committee-affecting policies should either carry this reason or use it automatically when expulsion drops `activeCount` below `thresholdM`.

Handle terminal states explicitly and do not suppress failure-transition errors for a nonterminal E3:

```
if (activeCount < thresholdM) {
    IInterfold.E3Stage stage = dependencies.interfoldContract.getE3Stage(
        p.e3Id
    );

    if (
        stage ≠ IInterfold.E3Stage.Complete &&
        stage ≠ IInterfold.E3Stage.Failed
    ) {
        dependencies.interfoldContract.onE3Failed(
            p.e3Id,
            uint8(IInterfold.FailureReason.CommitteeViabilityLost)
        );
    }
}
```

Without catch-all suppression, an unexpected notification failure reverts the slash transaction and rolls back the expulsion, preserving consistency between registry viability and the Interfold lifecycle.

As defense in depth, require current viability before nonterminal lifecycle transitions:

- the registry should reject committee-key publication below `thresholdM`;
- Interfold should reject ciphertext and plaintext publication while the snapshotted registry reports a nonviable committee.

If failure notification must remain asynchronous, record a durable pending failure and block these transitions until permissionless retry succeeds. A retry mechanism alone is insufficient if the E3 can continue in the meantime.

If governance intentionally permits nonterminal operation below `thresholdM`, revise the documented invariant and define the allowed stages, policy types, settlement behavior, and security assumptions.

Add tests covering zero and requester-attributable failure reasons, atomic rollback when notification fails, successful below-threshold transition, publication while nonviable, permissionless retry with lifecycle blocking, zero-member fee settlement, and expulsion after terminal completion.

Interfold: Resolved with `@a43bcb6df1 ...`

Zenith: Verified.

[M-10] Shared fold-attestation verifiers enable conditional cross-registry DKG replay

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L301-L399
- packages/interfold-contracts/contracts/verifiers/bfv/BfvPkVerifier.sol#L102-L158
- packages/interfold-contracts/contracts/verifiers/DkgFoldAttestationVerifier.sol#L27-L85
- packages/interfold-contracts/contracts/verifiers/DkgFoldAttestationVerifier.sol#L196-L261
- packages/interfold-contracts/contracts/lib/DkgFoldAttestationLib.sol#L57-L121

Description:

`BfvPkVerifier.verify` receives the E3 ID, committee root, and sorted committee addresses, but explicitly discards them:

```
e3Id;
committeeRoot;
sortedNodes;
```

The PK wrapper directly binds the proof only to its immutable verification-key hashes, the committee hash, and the PK commitment. It does not bind the proof to the chain, registry, Interfold deployment, E3 ID, or registry membership root.

Fold attestations provide a separate context check. Their EIP-712 digest binds:

```
chainId
DkgFoldAttestationVerifier address
e3Id
partyId
skAggCommit
esmAggCommit
```

The verifier checks that each signer occupies the corresponding canonical party slot in the registry supplied at verification time. However, that registry address is not included in the signed digest. The EIP-712 `verifyingContract` is the stateless fold-attestation verifier.

Consequently, previously published proof and attestation calldata can satisfy a second registry when all of the following conditions hold:

- both registries are deployed on the same chain;
- both use the same `DkgFoldAttestationVerifier` address;
- both use the same numeric `e3Id`;
- both select the same canonical committee addresses in the same party slots; and
- their PK verifiers use compatible circuits and verification keys.

An external caller can copy the first registry's `publishCommittee` calldata and submit the same public key, commitment, proof, and attestation bundle after the second registry finalizes its committee. The PK wrapper accepts the unchanged committee hash and commitment. The signatures remain valid because their chain ID, fold-verifier address, E3 ID, party IDs, and fold commitments are identical. Runtime membership checks also pass against the second registry because its canonical committee matches.

The second registry's Interfold address, membership root, and wider registered-node set may differ because none are covered by the accepted proof or signatures. Publication is permissionless, allowing replay to race honest fresh-DKG publication.

A successful replay installs an old PK commitment and old DKG anchors instead of fresh outputs. This can desynchronize the new committee and cause E3 failure. Confidentiality may also be weakened if old secret material, sufficient old shares, or the reconstructed key remain available to an unauthorized party.

Same-registry replay remains blocked by monotonically unique E3 IDs and the single-publication guard. Cross-chain replay remains blocked by `chainId`. No evidence establishes that tracked deployments currently share a fold-attestation verifier, so exploitation depends on a future or parallel same-chain deployment reusing that stateless verifier.

Recommendations:

It is recommended to bind the registry address directly into the signed fold-attestation context:

```
DkgFoldAttestation(
    address registry,
    uint256 e3Id,
    uint256 partyId,
    bytes32 skAggCommit,
    bytes32 esmAggCommit
)
```

Ciphernodes must sign the registry-specific digest, and `DkgFoldAttestationVerifier` must recover signatures using the registry passed to `verify`. This is the minimal change that closes the identified shared-verifier replay.

For defense in depth, bind the PK proof itself to a circuit public-input domain containing:

```
chainId
registry address
PK-verifier address
e3Id
committee root
committee hash
PK commitment
```

The wrapper can derive the domain as:

```
bytes32 domain = keccak256(
    abi.encode(
        block.chainid,
        msg.sender,
        address(this),
        e3Id,
        committeeRoot,
        committeeHash,
        pkCommitment
    )
);
```

Expose and constrain a field-compatible representation of this domain in the DKG aggregation circuit and compare it in `BfvPkVerifier`. `sortedNodes` need not be included separately because `committeeHash` already commits to the canonical address array.

As an interim mitigation, deploy a unique fold-attestation verifier for every registry and prohibit sharing verifier addresses across protocol instances. Treat this only as an operational safeguard because the invariant is not currently enforced on-chain.

Correct the PK-verifier documentation, which claims full call-context binding despite the explicit implementation relaxation.

Add a regression test with two same-chain registries sharing one fold verifier, the same E3 ID, and the same canonical committee. Replay the first publication calldata in the second registry, then verify that registry-bound attestations reject it.

Interfold: Resolved with `@278f9c5c33 ...`

Zenith: Verified.

[M-11] `slashTicketBalance` credits the full slash amount without checking what the exit queue actually yielded

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- BondingRegistry.slashTicketBalance
- BondingRegistry.slashLicenseBond

Description:

Asymmetric handling of the same library call. License path:

```
(, uint256 pendingSlashed) = _exits.slashPendingAssets(operator, 0,
    remainingSlashAmount, true);
require(pendingSlashed == remainingSlashAmount, InsufficientBalance());
```

Ticket path:

```
if (remainingToSlash > 0) {
    _exits.slashPendingAssets(operator, remainingToSlash, 0, true);   //
    return value discarded
}
slashedTicketBalance += actualSlashAmount;                            // full
    amount credited
```

A short return would inflate `slashedTicketBalance` while `pendingTotals[operator].ticketAmount` retains the difference. Both then draw on the same `InterfoldTicketToken.payableBalance`; first caller wins, second gets `"Exceeds payable balance"`.

Not reachable today. `pendingTotals.ticketAmount` is always fully reachable from `queueHeadIndexTicket`, and `remainingToSlash ≤ pendingTicketBalance` by construction. Reported because the guard exists on the sibling path, the head logic is the most intricate code in scope, and `MAX_ACTIVE_TRANCHES` / `_pruneEmptyTail` / `setExitDelay` non-monotonicity is exactly the interaction a future change breaks silently.

Recommendations:

```
if (remainingToSlash > 0) {
    (uint256 pendingSlashed, ) = _exits.slashPendingAssets(operator,
    remainingToSlash, 0, true);
    require(pendingSlashed == remainingToSlash, InsufficientBalance());
}
```

Interfold: Resolved with @debc252887 ...

Zenith: Verified.

[M-12] Committee member can withdraw all collateral before E3 completion by deregistering mid-lifecycle

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- CiphernodeRegistryOwnable.submitTicket()
- CiphernodeRegistryOwnable.getActiveCommitteeNodes()
- BondingRegistry.deregisterOperator()
- BondingRegistry.claimExits()

Description

Committee membership is determined by submitTicket() and frozen at finalization. getActiveCommitteeNodes() reads the historical `memberStatus`, not current registration or bond status.

After selection, an operator can call deregisterOperator(), moving all collateral to the exit queue. After the exit delay (as low as 1 day), claimExits() withdraws everything — provided no slash proposal is open at that moment. The operator is then still recognized as an active committee member for the in-flight E3 but has zero slashable balance.

`noOpenSlashProposal` only blocks claims if a proposal is CURRENTLY open. The operator must deregister before any proposal is filed. Since committee selection happens before any potential misbehavior, no proposal exists yet.

Risk Analysis

- **Impact:** Medium — An operator on an active committee can exit with zero slashable collateral remaining. If they later misbehave (withhold shares, submit invalid proofs), slashing collects nothing. The operator bears no financial consequence. If their absence drops the committee below threshold, the E3 fails.
- **Likelihood:** Medium — Requires only that the operator deregisters after committee selection, waits through the exit delay (minimum 1 day), and claims before the E3's decryption phase. Plausible for any E3 with a lifecycle longer than the exit delay.

Proof of Concept

Copy-paste in `Interfold.spec.ts` and run with `npx hardhat test test/Interfold.spec.ts --grep "bug"`

```
Running Mocha tests


  Interfold
    bug: Committee member exits before service ends
      ✔ operator deregisters after committee selection, withdraws all collateral, remains active member (485ms)


  1 passing (487ms)
```

```
describe("bug: Committee member exits before service ends", function () {
  it("operator deregisters after committee selection, withdraws all
    collateral, remains active member", async function () {
    const {
      interfold,
      usdcToken,
      ciphernodeRegistryContract: registry,
      bondingRegistry,
      request,
      operator1,
      operator2,
      operator3,
    } = await loadFixture(setup);

    // Create E3 and form committee
    await makeRequest(interfold, usdcToken, {
      ...request,
      inputWindow: [(await time.latest()) + 200, (await time.latest())
    + 86400],
    });
    await registry.connect(operator1).submitTicket(0, 1);
    await registry.connect(operator2).submitTicket(0, 1);
    await registry.connect(operator3).submitTicket(0, 1);

    const deadline = await registry.getCommitteeDeadline(0);
    await time.increaseTo(Number(deadline) + 1);
    await registry.finalizeCommittee(0);

    // operator1 is on the committee
    expect(await registry.isCommitteeMemberActive(0,
    operator1.address)).to.be.true;
```

```
    // operator1 immediately deregisters
    await bondingRegistry.connect(operator1).deregisterOperator();
    expect(await
    bondingRegistry.isRegistered(operator1.address)).to.be.false;

    // Wait through exit delay and claim everything
    const exitDelay = await bondingRegistry.exitDelay();
    await time.increase(Number(exitDelay) + 1);
    await bondingRegistry.connect(operator1).claimExits(
      ethers.MaxUint256,
      ethers.MaxUint256,
    );

    // operator1 has zero collateral
    expect(await
    bondingRegistry.totalBonded(operator1.address)).to.equal(0);

    // But is STILL an active committee member for E3 #0
    expect(await registry.isCommitteeMemberActive(0,
    operator1.address)).to.be.true;

    // And would still receive rewards on success
    const [activeNodes] = await registry.getActiveCommitteeNodes(0);
    expect(activeNodes).to.include(operator1.address);
  });
});
```

Recommendations

Track per-operator unresolved committee obligations. Prevent `deregisterOperator` or `claimExits` while the operator is on any active (non-terminal) committee, or reserve a per-E3 minimum slashable balance until the E3 reaches a terminal state.

Interfold: Resolved with `@3e0b44db21 ...`

Zenith: Verified.

[M-13] Asset rotation can leave raw-unit economic parameters stale

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/Interfold.sol#L602-L619
- interfold-contracts/contracts/interfaces/IInterfold.sol#L83-L100
- interfold-contracts/contracts/registry/BondingRegistry.sol#L747-L875
- interfold-contracts/contracts/token/InterfoldTicketToken.sol#L161-L178
- interfold-contracts/contracts/interfaces/ISlashingManager.sol#L40-L64
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L1028-L1056

Description:

Protocol prices, collateral requirements, and slash penalties are stored as raw token-unit integers:

```
uint256 public ticketPrice;
uint256 public licenseRequiredBond;

struct SlashPolicy {
    uint256 ticketPenalty;
    uint256 licensePenalty;
    // ...
}
```

The asset setters do not update these values. `setFeeToken` retains the existing `PricingConfig`; `setTicketToken` retains `ticketPrice`; and `setLicenseToken` retains `licenseRequiredBond` and every slashing policy.

The tFOLD underlying is immutable and cannot rotate in place. However, `BondingRegistry` can replace `ticketToken` with a newly deployed wrapper over a different underlying.

For equally valued tokens, retaining six-decimal fee coefficients after activating an 18-decimal fee token makes service quotes `10^12` times cheaper in whole-token terms. Retaining `ticketPrice = 10e6` when replacing a six-decimal wrapper with an 18-decimal wrapper changes one ticket from 10 whole tokens to `10^-11` whole tokens. The inverse direction can

make requests or collateral requirements prohibitively expensive.

Decimals alone do not represent economic value. Equal-decimal tokens can have different market values, while decimal rescaling preserves only whole-token quantities.

Historical state is only partially isolated. Interfold snapshots each E3's fee token, and BondingRegistry requires old collateral liabilities to be drained before rotation. Slashing policies and proposals, however, store raw penalty amounts without asset addresses. An unresolved proposal can apply old-asset units to replacement collateral if new collateral is deposited before execution.

Ticket rotation also affects in-flight committee selection. Eligibility reads historical voting balances and `ticketPrice` through the current `BondingRegistry`. Replacing the wrapper or changing its price after committee request can alter or invalidate request-time ticket semantics.

Governance must first create the inconsistent configuration, so this is not a permissionless rotation attack. Once activated, however, requesters or operators can exploit underpriced requests, tickets, bonds, or penalties.

Recommendations:

It is recommended to bind economic configuration to the exact asset for which it was approved:

```
struct FeeAssetConfig {
    IERC20 token;
    uint8 expectedDecimals;
    PricingConfig pricing;
}
```

When activating an asset, require deployed code, safely query and compare its declared decimals, validate the complete associated configuration, and emit the token address, expected decimals, and raw-unit values. Treat decimals as a sanity check rather than an economic price.

For ticket and license rotation, use an atomic Safe batch or coordinated function that installs:

- the replacement token or wrapper;
- `ticketPrice`;
- `licenseRequiredBond`; and
- all ticket and license slash penalties.

Maintain an enumerable or versioned registry of slash policy identifiers so the batch cannot silently omit a configured policy. Updating policies is insufficient for existing proposals because proposal amounts are already snapshotted.

Prevent ticket or license rotation while any relevant slash proposal, pending route, exit, liability, or committee request remains unresolved. Existing proposals should terminate or be explicitly migrated before rotation. Supporting their execution across assets would require multi-asset collateral accounting, not merely storing a token address.

Similarly, do not rotate the ticket wrapper while committees remain in the ticket-submission phase unless each committee snapshots and queries its exact wrapper address and request-time `ticketPrice`.

Verify before execution that the Safe controls both BondingRegistry ownership and SlashingManager governance. Do not trust migrated liability counters until the legacy bonding-storage migration has initialized and reconciled them.

Add tests covering six-to-18 and 18-to-six decimal rotations, same-decimal assets with different values, omitted policy updates, existing slash proposals, and in-flight committee requests.

Interfold: Resolved with `@060973a977 ...`

Zenith: Verified.

[M-14] Synchronous slashing-manager callbacks can block exits and registration

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L204-L217
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L399-L416
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L897-L935
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L233-L243

Description:

`BondingRegistry` synchronously queries every authorized slashing manager before allowing an operator to remove collateral:

```
for (uint256 i = 0; i < len; i++) {
    if (
        ISlashingManager(_authorizedSlashingManagers[i])
            .hasOpenSlashProposal(operator)
    ) {
        revert OperatorUnderSlash();
    }
}
```

This check affects `deregisterOperator`, `removeTicketBalance`, `unbondLicense`, and `claimExits`. `registerOperator` performs similar fan-out through `isBanned`.

`setSlashingManager` accepts any nonzero address without checking its bytecode, API version, required selectors, return-data shape, or configured `BondingRegistry`. A missing selector, malformed return value, reverting callback, or gas-consuming implementation therefore reverts the complete user operation. A globally failing manager blocks every operator, while a selectively reverting manager can block chosen operators.

The current official `SlashingManager` is immutable and both callbacks are constant-cost mapping reads. Consequently, this is not a permissionless attack against a fresh, correctly configured deployment. Reachability requires owner misconfiguration, authorization of a custom or upgradeable manager, failure of a retained dependency, or an ABI-incompatible

protocol migration.

There is a concrete compatibility hazard at the remediation boundary. The legacy manager implements `hasOpenLaneBProposal`, whereas the current `BondingRegistry` calls `hasOpenSlashProposal`. A direct upgrade alone does not trigger this callback failure because the newly introduced authorization array remains empty, as separately described by the bonding migration finding. However, adding the legacy manager to that array during migration makes all four collateral-removal functions revert. Its compatible `isBanned` selector means registration is not blocked by this particular legacy mismatch.

The owner can recover by rotating and revoking a failed manager, but the current manager must first be replaced. Revocation can also be unsafe while the old manager retains unresolved proposals, bans, or current-generation pending routes: it can release collateral locks, allow banned operators to register, prevent slash execution, or orphan reserved funds. Pending-route risk applies to current-generation managers, not the tracked legacy implementation.

Recommendations:

It is recommended to remove withdrawal-time dependency on external manager callbacks. Store manager-scoped ban status and open-proposal locks in `BondingRegistry`, together with aggregate per-operator values. Authorized managers should update this state atomically when proposals or bans are created and resolved.

```
mapping(address operator ⇒ uint256 count) internal _openSlashLocks;
mapping(address operator ⇒ uint256 count) internal _activeBans;

modifier noOpenSlashProposal(address operator) {
    if (_openSlashLocks[operator] ≠ 0) revert OperatorUnderSlash();
    _;
}
```

Introduce explicit manager lifecycle states and permit removal only after live E3 assignments, proposal locks, and pending routes reach zero and bans have been migrated or deliberately cleared. A failed manager must not simply be omitted from checks, because that would fail open and release slashable collateral.

Before authorization, require nonempty code, an approved immutable implementation or governed upgrade path, an explicit API version, the expected `BondingRegistry`, and successful probes of every required selector with exact return-data validation. Gas-capped `staticcall` can contain gas consumption and improve diagnostics, but does not itself restore liveness while failures remain fail-closed.

For legacy migrations, either drain and migrate all obligations before authorizing only the current manager, or add an explicit versioned legacy callback path. A query adapter is safe only if query routing is separated from slash authority so the original manager remains authorized to execute existing proposals.

Interfold: Resolved with @9693eb32b1 ...

Zenith: Verified.

[M-15] Retained SlashingManagers can drain slashed ticket funds to arbitrary addresses

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/registry/BondingRegistry.sol#L328—L355
- interfold-contracts/contracts/registry/BondingRegistry.sol#L693—L702
- interfold-contracts/contracts/registry/BondingRegistry.sol#L705—L716

Description:

`setSlashingManager` adds each new manager to `_authorizedSlashingManagers` but does not revoke the old one. Revocation requires a separate, manual `revokeSlashingManager` call. Any retained historical manager therefore remains authorized indefinitely.

`reserveSlashedTicketFunds` and `redirectReservedSlashedTicketFunds` are both gated by `onlyAuthorizedSlashingManager`. Neither function scopes its authorization to a specific E3, proposal, or recipient. `redirectReservedSlashedTicketFunds` accepts an arbitrary `to` address:

```
File: BondingRegistry.sol
705:     function redirectReservedSlashedTicketFunds(
706:         address to,
707:         uint256 amount
708:     ) external onlyAuthorizedSlashingManager {
709:         require(to ≠ address(0), ZeroAddress());
710:         require(amount > 0, ZeroAmount());
711:         require(amount <= reservedSlashedTicketBalance,
    InsufficientBalance());
712:
713:         reservedSlashedTicketBalance -= amount;
714:         slashedTicketBalance -= amount;
715:         ticketToken.payout(to, amount);
716:     }
```

A compromised or buggy retained manager can reserve all unreserved slashed balance and redirect it to any address in a single transaction.

The precondition for non-zero `reservedSlashedTicketBalance` is a failed slash routing attempt. This occurs naturally when the `routePendingSlashFunds` internal self-call reverts — for instance, when `getActiveCommitteeNodes` panics due to the provisional `Active` status issue described in "Provisional Active status during sortition leaks into downstream consumers" (since `activeCount == 0` while members are marked Active, causing an out-of-bounds array write), or when gas exhaustion exceeds `INITIAL_ROUTE_GAS` on large committees with prior settled tranches.

Step by Step Example

1. E3 #0 committee is finalized. Operator A is slashed for 50 USDC. Routing fails (e.g., registry `getActiveCommitteeNodes` reverts due to provisional status panic). `slashedTicketBalance = 50`, `reservedSlashedTicketBalance = 50`. Route stays pending.
2. Governance rotates the slashing manager: `setSlashingManager(newManager)`. Old manager is not revoked. Both old and new are authorized.
3. Old manager is compromised. Attacker calls `bondingRegistry.redirectReservedSlashedTicketFunds(attacker, 50)` through the old manager.
4. 50 USDC paid out to the attacker. `slashedTicketBalance = 0`, `reservedSlashedTicketBalance = 0`.
5. Legitimate `retrySlashRoute` reverts — the reserved funds are gone.

Proof of Concept

Setup: Add the `describe` block below inside the existing `describe("E3 Integration - Refund/Timeout Mechanism", ...)` in `E3Integration.spec.ts`. No additional imports are needed beyond the existing `MockBlacklistUSDC` type import already present in the file.**Run:** `npx hardhat test --grep "bug"`**Description:** The first test creates a pending slash route by blacklisting the refund manager before slashing (causing `routePendingSlashFunds` to revert). It then rotates the slashing manager without revoking the old one, impersonates the old manager, and calls `redirectReservedSlashedTicketFunds` to drain the reserved funds to an attacker address. The second test confirms that revoking the old manager blocks the drain.In production, the blacklisting precondition is not required — routing failures arise from internal protocol conditions such as the provisional `Active` status panic or gas exhaustion. Blacklisting is used in the PoC because it produces the routing failure deterministically without introducing additional fixtures.

```
describe("bug — Retained SlashingManager Drain", function () {
  it("retained (non-revoked) old SlashingManager can drain slashed ticket
    funds to arbitrary address", async function () {
    const {
      interfold,
      bondingRegistry,
      registry,
```

```
      slashingManager,
      e3RefundManager,
      usdcToken,
      makeRequest,
      owner,
      requester,
      operator1,
      operator2,
      operator3,
      computeProvider: attacker,
      setupOperator,
    } = await loadFixture(setup);

    await setupOperator(operator1);
    await setupOperator(operator2);
    await setupOperator(operator3);
    await makeRequest();

    await registry.connect(operator1).submitTicket(0, 1);
    await registry.connect(operator2).submitTicket(0, 1);
    await registry.connect(operator3).submitTicket(0, 1);
    await time.increase(SORTITION_SUBMISSION_WINDOW + 1);
    await registry.finalizeCommittee(0);

    const publicKey = "0x1234567890abcdef1234567890abcdef";
    const pkCommitment = ethers.keccak256(publicKey);
    await registry.publishCommittee(
      0,
      publicKey,
      pkCommitment,
      encodeMockDkgProof(pkCommitment),
      "0x01",
    );

    // Fail the E3 and process it so calculateRefund runs
    const e3 = await interfold.getE3(0);
    const computeDeadline =
      Number(e3.inputWindow[1]) + defaultTimeoutConfig.computeWindow;
    await time.increaseTo(computeDeadline + 1);
    await interfold.markE3Failed(0);
    await interfold.processE3Failure(0);

    // Blacklist refund manager so slash routing reverts
    const blacklistToken = usdcToken as unknown as MockBlacklistUSDC;
    const refundManagerAddress = await e3RefundManager.getAddress();
    await blacklistToken.blacklist(refundManagerAddress);
```

```
// Slash operator1 — routing fails, funds stay reserved
const proof = await signAndEncodeAttestation(
  [operator2, operator3],
  0,
  await operator1.getAddress(),
  await slashingManager.getAddress(),
);
await expect(
  slashingManager.proposeSlash(0, await operator1.getAddress(), proof),
).to.emit(slashingManager, "SlashRoutePending");

const pending = await slashingManager.getPendingSlashRoute(0);
expect(pending.pending).to.equal(true);
const reservedAmount =
  await bondingRegistry.reservedSlashedTicketBalance();
expect(reservedAmount).to.equal(pending.amount);
expect(reservedAmount).to.be.gt(0);

// Unblacklist so USDC transfer to attacker succeeds
await blacklistToken.unblacklist(refundManagerAddress);

// Owner rotates manager but does NOT revoke the old one
const oldSlashingManagerAddress = await slashingManager.getAddress();
const newManagerAddress = await attacker.getAddress();
await bondingRegistry
  .connect(owner)
  .setSlashingManager(newManagerAddress);

expect(
  await bondingRegistry.isAuthorizedSlashingManager(
    oldSlashingManagerAddress,
  ),
).to.equal(true);

// Impersonate old manager and drain reserved funds
await ethers.provider.send("hardhat_impersonateAccount", [
  oldSlashingManagerAddress,
]);
await ethers.provider.send("hardhat_setBalance", [
  oldSlashingManagerAddress,
  "0x1000000000000000000",
]);
const oldManagerSigner = await ethers.getSigner(
  oldSlashingManagerAddress,
);

const attackerAddress = await attacker.getAddress();
```

```
    const attackerBalanceBefore = await
    usdcToken.balanceOf(attackerAddress);

    await bondingRegistry
      .connect(oldManagerSigner)
      .redirectReservedSlashedTicketFunds(attackerAddress, reservedAmount);

    await ethers.provider.send("hardhat_stopImpersonatingAccount", [
      oldSlashingManagerAddress,
    ]);

    // Attacker received the funds
    const attackerBalanceAfter = await usdcToken.balanceOf(attackerAddress);
    expect(attackerBalanceAfter - attackerBalanceBefore).to.equal(
      reservedAmount,
    );
    expect(await bondingRegistry.slashedTicketBalance()).to.equal(0);
    expect(await
    bondingRegistry.reservedSlashedTicketBalance()).to.equal(0);

    // Legitimate retry now fails — funds are gone
    await expect(
      slashingManager.connect(requester).retrySlashRoute(0),
    ).to.be.revert(ethers);
  });

  it("revoking the old manager blocks the drain", async function () {
    const {
      interfold,
      bondingRegistry,
      registry,
      slashingManager,
      e3RefundManager,
      usdcToken,
      makeRequest,
      owner,
      operator1,
      operator2,
      operator3,
      computeProvider: attacker,
      setupOperator,
    } = await loadFixture(setup);

    await setupOperator(operator1);
    await setupOperator(operator2);
    await setupOperator(operator3);
    await makeRequest();
```

```
await registry.connect(operator1).submitTicket(0, 1);
await registry.connect(operator2).submitTicket(0, 1);
await registry.connect(operator3).submitTicket(0, 1);
await time.increase(SORTITION_SUBMISSION_WINDOW + 1);
await registry.finalizeCommittee(0);

const publicKey = "0x1234567890abcdef1234567890abcdef";
await registry.publishCommittee(
  0,
  publicKey,
  ethers.keccak256(publicKey),
  encodeMockDkgProof(ethers.keccak256(publicKey)),
  "0x01",
);

const e3 = await interfold.getE3(0);
const computeDeadline =
  Number(e3.inputWindow[1]) + defaultTimeoutConfig.computeWindow;
await time.increaseTo(computeDeadline + 1);
await interfold.markE3Failed(0);
await interfold.processE3Failure(0);

const blacklistToken = usdcToken as unknown as MockBlacklistUSDC;
const refundManagerAddress = await e3RefundManager.getAddress();
await blacklistToken.blacklist(refundManagerAddress);

const proof = await signAndEncodeAttestation(
  [operator2, operator3],
  0,
  await operator1.getAddress(),
  await slashingManager.getAddress(),
);
await slashingManager.proposeSlash(
  0,
  await operator1.getAddress(),
  proof,
);
await blacklistToken.unblacklist(refundManagerAddress);

const oldManagerAddress = await slashingManager.getAddress();

// Rotate AND revoke
await bondingRegistry
  .connect(owner)
  .setSlashingManager(await attacker.getAddress());
await bondingRegistry
```

```
      .connect(owner)
      .revokeSlashingManager(oldManagerAddress);

    expect(
      await bondingRegistry.isAuthorizedSlashingManager(oldManagerAddress),
    ).to.equal(false);

    await ethers.provider.send("hardhat_impersonateAccount", [
      oldManagerAddress,
    ]);
    await ethers.provider.send("hardhat_setBalance", [
      oldManagerAddress,
      "0x1000000000000000000",
    ]);
    const oldManagerSigner = await ethers.getSigner(oldManagerAddress);

    const stealable =
      (await bondingRegistry.slashedTicketBalance()) -
      (await bondingRegistry.reservedSlashedTicketBalance());

    if (stealable > 0n) {
      await expect(
        bondingRegistry
          .connect(oldManagerSigner)
          .reserveSlashedTicketFunds(stealable),
      ).to.be.revertedWithCustomError(bondingRegistry, "Unauthorized");
    }

    await ethers.provider.send("hardhat_stopImpersonatingAccount", [
      oldManagerAddress,
    ]);
  });
});
```

Recommendations

Scope `redirectReservedSlashedTicketFunds` so `to` must be the refund manager registered for the E3 whose proposal created the reservation, or require the caller to supply a valid `proposalId` that is verified against the pending route. Alternatively, auto-revoke old managers on rotation, with a migration path for pending routes.

Interfold: Resolved with `@08c72af325 ...`

Zenith: Verified.

[M-16] The E3 projection overwrites owner reward credits with tokenless operator allocations

SEVERITY: Medium	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: High

Target

- packages/interfold-contracts/contracts/interfaces/IInterfold.sol#L163-L184
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L327-L350
- packages/interfold-contracts/contracts/Interfold.sol#L526-L536
- crates/dashboard/src/projection.rs#L345-L370
- crates/dashboard/src/projection.rs#L448-L476

Description:

For each nonzero committee reward, `InterfoldPricing` credits the operator's bond owner. It emits `RewardCredited` with that recipient. After the library returns, `Interfold` emits `RewardsDistributed` with the committee operators.

```
address operator = nodes[i];
address recipient = bonding.bondOwnerOf(operator);
if (recipient == address(0)) recipient = operator;

pendingRewards[e3Id][recipient] += amount;
emit RewardCredited(e3Id, recipient, token, amount);

// Emitted after computeAndCreditRewards returns.
emit RewardsDistributed(e3Id, activeNodes, amounts);
```

For a successful E3 with nonzero ciphernode rewards, the production order places `RewardCredited` before `RewardsDistributed`.

The E3 projection first appends the canonical owner credits. When `RewardsDistributed` arrives, it replaces the complete list with tokenless operator-allocation rows.

The `RewardClaimed` handler requires an exact account, token, and amount match. It cannot match these tokenless rows. The E3 trace therefore continues to show a claimed reward as unclaimed. A split-owner configuration also displays the operator instead of the recipient.

One owner can receive several credits for different committee operators. The contract com-

bines these credits into one `(e3Id, owner)` balance. A claim drains the aggregate balance. Exact matching against one individual credit cannot reconcile that claim, even if the replacement defect is fixed.

The chain-level projection also ignores owner credits when it filters events by the local operator address. This behavior is incorrect only if the operator dashboard intends to show rewards attributable to the operator. An explicit operator-recipient association is necessary for that view.

The existing replay test does not reproduce the production flow. It places `RewardsDistributed` before `RewardCredited` and uses one address for both the operator and recipient.

The on-chain reward ledger remains correct. Recipients can claim their funds. The impact is limited to incorrect recipient, token, and claim-status information in the dashboard.

Recommendations:

It is recommended to preserve `RewardCredited` as the canonical payment record. Store `RewardsDistributed` operator allocations separately and do not replace recipient credits.

Group credits by `(e3Id, recipient, token)`. When `RewardClaimed` arrives, verify the aggregate amount and mark all credits in the matching group as claimed.

Correct the event documentation. `RewardsDistributed.nodes` contains operators, while `RewardCredited.account` contains the bond owner or operator fallback.

Add a replay test with the production event order, distinct operators and owners, multiple operators sharing one owner, and the subsequent aggregate claim.

If the operator dashboard must associate each allocation with its recipient, add that relationship to the aggregate event. Update the Solidity interface, library return values, event decoder, and projection together.

```
event RewardsDistributed(
    uint256 indexed e3Id,
    address[] operators,
    address[] recipients,
    uint256[] amounts
);
```

Interfold: Resolved with @c248091250 ...

Zenith: Verified.

[M-17] A post-exit owner reset can redirect rewards earned under the previous owner

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L426-L448
- packages/interfold-contracts/contracts/E3RefundManager.sol#L418-L488
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L327-L350
- packages/interfold-contracts/contracts/E3RefundManager.sol#L588-L619

Description:

`setBondOwner` lets an operator replace an established owner whenever its current position is empty. The check does not record whether the position was previously funded or has unresolved E3 entitlements.

```
address currentOwner = _bondOwnerOf[msg.sender];
if (currentOwner ≠ address(0)) {
    (uint256 pendingTicket, uint256 pendingLicense) = _exits
        .getPendingAmounts(msg.sender);
    Operator storage op = operators[msg.sender];
    if (
        op.registered ||
        op.licenseBond ≠ 0 ||
        pendingLicense ≠ 0 ||
        ticketToken.balanceOf(msg.sender) ≠ 0 ||
        pendingTicket ≠ 0
    ) {
        revert BondOwnerAlreadySet(msg.sender, currentOwner);
    }
}

delete _pendingBondOwnerOf[msg.sender];
_bondOwnerOf[msg.sender] = bondOwner;
emit BondOwnerSet(msg.sender, bondOwner);
```

Owner `A` can fund operator `O`, deregister the position, and claim all collateral after the exit

delay. The position then passes the empty-state check. The operator can assign an attacker-controlled owner without consent from A or the new owner.

For failed E3s, `claimHonestNodeReward` resolves the recipient from the live owner mapping:

```
require(
    !_honestNodeClaimed[e3Id][operator],
    AlreadyClaimed(e3Id, operator)
);

// Check that the supplied operator is an honest node.
address[] memory nodes = _honestNodes[e3Id];
bool isHonest = false;
for (uint256 i = 0; i < nodes.length && !isHonest; i++) {
    isHonest = (nodes[i] == operator);
}
require(isHonest, NotHonestNode(e3Id, operator));

address recipient = IBondingRegistry(
    _e3PolicySnapshots[e3Id].bondingRegistry
).bondOwnerOf(operator);
if (recipient == address(0)) recipient = operator;
require(msg.sender == recipient, Unauthorized());
```

The new owner can consume the operator-keyed claim. `_honestNodeClaimed[e3Id][operator]` then prevents the previous owner from claiming the same entitlement.

This base reward exists only for requester-attributable failures with a nonzero completed-work allocation. Ciphernode-attributable failures give honest nodes only the ticket assets that were actually slashed.

Success rewards and slashed-fund allocations have conditional variants of the same issue. Both routes resolve the live owner when they credit their pull-payment ledgers. An owner reset before settlement changes the recipient. Credits created before the reset remain safe.

Valid configurations let an E3 outlive the collateral exit. The protocol permits a one-day exit delay and a maximum E3 duration of up to 365 days. The deployment script uses a seven-day exit delay and a maximum duration of 30 days.

The current tests verify owner correction before initial funding. They do not test an owner reset after a previously funded position becomes empty.

This finding assumes that rewards earned while owner A funded the position remain payable to A. The documentation states that protocol rewards go to the bond owner, but it does not define the entitlement time.

If rewards intentionally follow the current owner until claim or settlement, the direct theft

classification does not apply. The three reward routes still use inconsistent entitlement times and require an explicit policy.

Recommendations:

It is recommended to define one reward-entitlement time and apply it to every reward route.

Snapshot the recipient for each `(e3Id, operator)` when the committee becomes final. Use that snapshot for success rewards, failed-E3 rewards, and slashed-fund allocations.

Credit failed-E3 rewards to an owner-keyed pull ledger during `calculateRefund`. Do not resolve the recipient from `bondOwnerOf(operator)` during the claim.

Keep post-exit `setBondOwner` available for future positions. Historical rewards must remain bound to their snapshotted recipients.

If reward rights must move during a consensual owner transfer, `acceptBondOwner` must migrate those rights explicitly. The protocol must not infer the transfer from a later live-owner lookup.

Add tests for:

1. A full exit followed by an owner reset and failed-E3 claim.
2. An owner reset before successful E3 settlement.
3. An owner reset before slashed-fund allocation.
4. A consensual owner transfer during an in-flight E3.
5. Multiple unresolved E3s across one owner reset.

Interfold: Resolved with `@df695327f1 ...`

Zenith: Verified.

[M-18] Configurable slash failure reasons can charge requesters for committee loss

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L330-L361
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L793-L815
- packages/interfold-contracts/contracts/Interfold.sol#L818-L830
- packages/interfold-contracts/contracts/E3RefundManager.sol#L254-L307

Description:

Governance can assign any nonzero, in-range `FailureReason` to a slash policy that affects an E3 committee. Policy validation requires committee expulsion but does not restrict the selected reason to one classified as ciphernode-payer by `E3RefundManager`.

```
if (policy.failureReason > 0) {
    require(policy.affectsCommittee, InvalidPolicy());
    require(
        policy.failureReason <
            uint8(IInterfold.FailureReason._MAX_FAILURE_REASON),
        InvalidPolicy()
    );
}
```

The configured reason is snapshotted into each slash proposal. When execution expels the targeted node and reduces the active committee below `M`, `SlashingManager` forwards that reason to `Interfold`:

```
if (activeCount < thresholdM && p.failureReason > 0) {
    dependencies.interfoldContract.onE3Failed(
        p.e3Id,
        p.failureReason
    );
}
```

`Interfold.onE3Failed()` verifies that the caller is the snapshotted registry or slashing manager and that the E3 is nonterminal, but it does not ensure that a reason supplied by the slashing manager represents a ciphernode-payer failure. `E3RefundManager` subsequently uses the reason to allocate the requester's escrow:

```
if (getFailurePayer(reason) == FailurePayer.Ciphernodes) {
    return (0, originalPayment, 0);
}

honestNodeAmount = (originalPayment * workCompletedBps) / BPS_BASE;
requesterAmount = (originalPayment * workRemainingBps) / BPS_BASE;
protocolAmount = originalPayment - honestNodeAmount - requesterAmount;
```

For example, governance can assign `ComputeTimeout`, which is classified as requester-payer, to a node-slashing policy. If a valid slash then expels that node and reduces the committee below threshold, the E3 fails because of supplier-side committee loss but settles as a requester-payer failure.

Under the default allocation, `ComputeTimeout` attributes 40% of the payment to completed committee/DKG work and 5% to the protocol. When at least one honest committee member remains, the requester receives only 55% instead of the full refund provided for ciphernode-payer failures. If no honest member remains, the node share is folded into the requester refund, but the protocol still retains its share.

Failure reasons also imply a settlement stage independently of the E3's actual state. A requester-payer reason supplied after a slash can therefore compensate work that was not completed in the affected E3.

Exploitation requires governance to configure an incompatible reason, followed by a valid slash that expels a node and reduces active membership below `M`. Current committee-expulsion tests use `DKGInvalidShares` and `DecryptionInvalidShares`, which are both correctly classified as ciphernode-payer; no active unsafe production policy is established.

Recommendations:

It is recommended to remove configurable payer semantics from committee-threshold failures. When slashing causes active membership to fall below `M`, use a dedicated stage-independent reason such as `CommitteeThresholdLostAfterSlash`, classified unconditionally as ciphernode-payer:

```
if (activeCount < thresholdM) {
    dependencies.interfoldContract.onE3Failed(
        p.e3Id,
        uint8(IInterfold.FailureReason.CommitteeThresholdLostAfterSlash)
    );
}
```

The policy's existing `p.reason` can retain the detailed misconduct category for events and slashing records without controlling requester settlement.

If configurable failure reasons must remain, policy validation should use a shared payer-classification library and accept only ciphernode-payer reasons for committee-affecting policies. The same classification must be used by `E3RefundManager` to prevent validation and settlement from diverging.

Do not validate a delayed slash solely against the E3 stage observed at execution, because an appeal window may allow the E3 to advance after the original misconduct. If stage-specific attribution is required, snapshot the fault or evidence stage when the proposal is created and validate against that snapshot.

Tests should verify that requester-payer reasons are rejected for committee-threshold policies and that a threshold-loss callback always preserves the requester's full base-fee refund.

Interfold: Resolved with `@5e8b7a133a ...`

Zenith: Verified.

4.4 Low Risk

A total of 19 low risk findings were identified.

[L-1] Incoherent dependency snapshots can strand or misdirect slash proceeds

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L273-L282
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L254-L272
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L309-L323
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L818-L884

Description:

`Interfold`, `CiphernodeRegistryOwnable`, and `SlashingManager` independently snapshot their current dependency pointers for each E3. They do not share a canonical configuration identifier or verify that their snapshots describe the same contract graph.

`Interfold` snapshots its own registry, refund manager, and slashing manager:

```
E3Dependencies storage dependencies = _e3Dependencies[e3Id];
dependencies.registry = ciphernodeRegistry;
dependencies.refundManager = e3RefundManager;
dependencies.slashManager = slashingManager;
```

The registry independently snapshots its current pointers and asks its configured slashing manager to snapshot that manager's global configuration:

```
dependencies.interfoldContract = IInterfold(msg.sender);
dependencies.bonding = bondingRegistry;
dependencies.slashManager = slashingManager;
dependencies.slashManager.snapshotE3Dependencies(e3Id);
```

```
dependencies.bonding = bondingRegistry;
dependencies.registry = ciphernodeRegistry;
dependencies.interfoldContract = interfold;
dependencies.refundManager = e3RefundManager;
dependencies.initialized = true;
```

The existing caller checks make some incorrect configurations fail during request creation. They do not, however, establish the following required equalities:

- Interfold and the registry use the same slashing manager;
- the slashing manager points back to the requesting Interfold and registry;
- the registry and slashing manager use the same bonding registry;
- Interfold and the slashing manager use the same refund manager;
- the refund manager authorizes the requesting Interfold; and
- the bonding registry points to the same registry and authorizes the intended slashing manager.

`Interfold.initialize` does not configure `slashingManager`, and `request` does not require the snapshotted manager to be nonzero. A request can therefore succeed with a zero or stale Interfold slashing-manager snapshot when the registry-side path is otherwise sufficiently configured.

If the manager executing a slash differs from the manager snapshotted by Interfold, the financial penalty, ban, and committee expulsion can succeed before routing fails. The executing manager reserves the slashed ticket proceeds and attempts to route them through its snapshotted Interfold. Interfold rejects the callback because the caller is not its E3-specific manager.

The routing attempt is isolated by `try/catch`, so its token transfer rolls back while the earlier reservation and pending route remain:

```
dependencies.bonding.reserveSlashedTicketFunds(actualTicketSlashed);
route.pending = true;

try this.routePendingSlashFunds(proposalId) returns (bool routed) {
    require(routed, InvalidProposal());
} catch {
    emit RoutingFailed(p.e3Id, actualTicketSlashed);
}
```

Every retry encounters the same immutable authorization mismatch. The proceeds remain reserved and cannot be withdrawn by the treasury or distributed to recipients. A related `onE3Failed` callback failure is also swallowed, potentially suppressing slash-triggered E3 failure.

A refund-manager mismatch creates another failure mode. `SlashingManager` transfers the

tokens to its snapshotted refund manager, while its Interfold callback records the escrow in Interfold's independently snapshotted refund manager:

```
dependencies.bonding.redirectReservedSlashedTicketFunds(
    address(dependencies.refundManager),
    route.amount
);
dependencies.interfoldContract.escrowSlashedFunds(
    route.e3Id,
    IERC20(route.token),
    route.amount
);
```

The accounting refund manager checks aggregate token solvency but does not verify that its balance increased by this particular transfer. Therefore:

- if it lacks sufficient balance, escrow reverts and the route remains permanently reserved; or
- if it holds unrelated same-token funds, the solvency check can pass using those funds as apparent backing.

In the second case, slash claims can consume balances owed to other E3 participants while the actual slash proceeds remain unaccounted in the other refund manager, producing cross-E3 insolvency.

A mismatched bonding-registry snapshot can similarly make the slashing manager penalize the same operator address in a different deployment, provided that deployment authorizes the manager and holds collateral for that address.

The production deployment script currently establishes a coherent graph. Reachability therefore depends on privileged deployment or governance misconfiguration, or a permissionless request submitted during a multi-transaction dependency rotation.

Recommendations:

It is recommended to establish one canonical, versioned dependency configuration and reject requests unless the complete graph is nonzero, deployed, ready, and reciprocally coherent.

```
require(configurationReady, ConfigurationNotReady());
require(address(slashingManager) ≠ address(0), InvalidConfiguration());
require(
    address(registry.slashingManager()) = address(slashingManager) &&
    address(registry.interfold()) = address(this) &&
    address(registry.bondingRegistry()) = address(bondingRegistry) &&
    address(slashingManager.interfold()) = address(this) &&
```

```
        address(slashingManager.ciphernodeRegistry()) == address(registry) &&
        address(slashingManager.bondingRegistry()) == address(bondingRegistry)
        &&
        address(slashingManager.e3RefundManager()) == address(e3RefundManager),
        IncoherentConfiguration()
    );
```

Populate all per-E3 snapshots from this canonical configuration rather than independently reading mutable globals. Pause requests during dependency rotations and activate a new configuration atomically only after validating every reciprocal edge.

Derive the slash-routing recipient from Interfold's E3 snapshot and require it to equal the refund manager used by `SlashingManager`. The routing transaction should also verify the recipient's exact balance increase before recording escrow.

Provide a governed, tightly constrained migration or cancellation mechanism for existing pending routes. Any migration must only permit routing to the validated canonical refund manager for the affected E3.

Interfold: Resolved with `@145014a499 ...`

Zenith: Verified.

[L-2] On-chain slashing ignores the configured accusation-vote validity policy

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L138-L160
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L690-L745
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L642-L727

Description:

`CiphernodeRegistryOwnable` stores an `accusationVoteValidity` period intended to limit the lifetime of accusation-vote signatures. Production ciphernodes fetch this value at startup, calculate deadlines from it, and reject peer accusations whose deadlines exceed their locally configured window.

Zeroing this value is additionally documented as disabling accusation voting. Zero updates require the two-day propose/commit flow, while nonzero updates can be applied immediately.

`SlashingManager` does not read or enforce this policy. It only checks the absolute deadline supplied and signed by the voters:

```
(
    uint256 proofType,
    address[] memory voters,
    bytes32[] memory dataHashes,
    bytes memory evidence,
    uint256 deadline,
    bytes[] memory signatures
) = abi.decode(proof, (uint256, address[], bytes32[], bytes, uint256,
    bytes[]));

require(block.timestamp <= deadline, SignatureExpired());
```

Consequently:

- A threshold of modified clients can sign an arbitrarily distant deadline.

- Committing `accusationVoteValidity = 0` does not disable on-chain Lane-A slashing.
- Ciphernodes cache the value at startup, so processes that have not restarted can continue producing and accepting votes under the previous configuration even after zero is committed.
- Signatures collected under a stale or longer policy remain admissible until their signer-selected deadline.

A single modified node cannot normally obtain a long-deadline quorum because correctly configured peers reject excessive deadlines. The attack therefore generally requires a modified or stale-configured quorum. This prerequisite may arise naturally after a governance update because every running node can retain the previous cached value.

The active-voter check does not provide a reliable temporal bound: committee members remain marked `Active` for an E3 unless individually expelled, including after the E3 has completed. Evidence replay protection prevents a second proposal for the same E3, operator, and proof type, but does not prevent the first proposal from being submitted long after the intended validity window.

A late proposal can therefore slash collateral, ban the operator, or expel it based on evidence that the configured validity policy should no longer accept. The signatures remain strongly scoped to a chain, contract, E3, operator, proof type, and evidence hash, which limits the issue to delayed submission of a specific accusation rather than arbitrary signature reuse.

Recommendations:

It is recommended to enforce both vote freshness and an objective slash-submission window on chain.

Add a signed `issuedAt` field and enforce a snapshotted per-E3 validity period. This limits the lifetime of signatures generated by conforming signers and protects leaked honest signatures:

```
require(issuedAt <= block.timestamp + MAX_CLOCK_SKEW, InvalidIssuedAt());
require(block.timestamp <= deadline, SignatureExpired());
require(deadline >= issuedAt, InvalidDeadline());
require(
    deadline - issuedAt <= snapshottedVoteValidity[e3Id],
    ExcessiveVoteValidity()
);
```

Because `issuedAt` is signer-asserted, a malicious quorum can postdate it or create fresh signatures later. Therefore, also enforce an objective per-E3 `slashSubmissionDeadline` derived from snapshotted lifecycle deadlines plus an explicitly configured reporting period:

```
require(
    block.timestamp <= slashSubmissionDeadline[e3Id],
```

```
    SlashSubmissionWindowClosed()
);
```

Implement a separate live on-chain pause, or directly reject Lane-A proposals while the registry value is zero:

```
require(!slashingPaused, SlashingPaused());
```

Clearly define whether nonzero validity changes apply only to newly created E3s or retroactively. Ciphernodes should also refresh the policy when it changes, but off-chain enforcement must remain defense-in-depth rather than the only control.

Interfold: Resolved with `@145014a499 ...` and `@1914ef40a1 ...`

Zenith: Verified.

[L-3] Late DKG publication can reclassify a supplier-side timeout as requester-paid

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L301-L346
- packages/interfold-contracts/contracts/Interfold.sol#L768-L815
- packages/interfold-contracts/contracts/Interfold.sol#L903-L934
- packages/interfold-contracts/contracts/E3RefundManager.sol#L254-L307

Description:

Interfold records a DKG deadline when a committee is finalized.

```
_e3Stages[e3Id] = E3Stage.CommitteeFinalized;
_e3Deadlines[e3Id].dkgDeadline =
    block.timestamp +
    _timeoutConfig.dkgWindow;
```

Neither `CiphernodeRegistryOwnable.publishCommittee` nor `Interfold.onCommitteePublished` enforces that deadline. The DKG proof and signed fold attestations also contain no timestamp or expiry, so a valid publication payload remains usable after `dkgDeadline` while the E3 remains in `CommitteeFinalized`.

```
// No dkgDeadline check
_verifyAndStoreDkgAnchors(/* ... */);
_interfoldFor(e3Id).onCommitteePublished(e3Id, pkCommitment);

// Interfold advances based only on the current stage
_e3Stages[e3Id] = E3Stage.KeyPublished;
e3.committeePublicKey = committeePublicKey;
```

Failure classification depends on the current stage:

```
if (stage == E3Stage.CommitteeFinalized)
    return (d.dkgDeadline, FailureReason.DKGTimeout);
```

```
if (stage == E3Stage.KeyPublished)
    return (d.computeDeadline, FailureReason.ComputeTimeout);
```

A party holding a valid DKG publication payload can therefore:

1. Withhold publication until after `dkgDeadline`.
2. Win transaction ordering against `markE3Failed(DKGTimeout)`.
3. Publish and move the E3 from `CommitteeFinalized` to `KeyPublished`.
4. Allow the already-established compute deadline to expire without ciphertext publication.
5. Cause the E3 to fail as `ComputeTimeout` rather than `DKGTimeout`.

The refund manager attributes these reasons differently. `DKGTimeout` is a ciphernode-paid failure and refunds the full payment to the requester. `ComputeTimeout` is requester-paid and allocates completed-work value to the active committee.

```
if (getFailurePayer(reason) == FailurePayer.Ciphernodes) {
    return (0, originalPayment, 0);
}

// A KeyPublished failure treats committee formation and DKG as completed
workCompletedBps =
    alloc.committeeFormationBps +
    alloc.dkgBps;
```

Under the default request-time allocation, and provided at least one active committee member remains, reclassification changes the distribution from a 100% requester refund to:

- 40% for active committee members;
- 5% for the protocol;
- 55% for the requester.

Publication is permissionless, but an arbitrary caller cannot construct the required proof and attestation bundle. The payload would normally be held by a committee participant or aggregator. A non-member aggregator can trigger the reclassification but profits from it only if it colludes with or controls an active reward recipient.

The economic effect requires late publication to execute before DKG failure, a subsequent compute timeout, and at least one active committee member when failure is processed. Late publication alone does not cause this loss if the E3 completes successfully.

A positive `markFailedGracePeriod` does not eliminate the issue. It restricts callers of `markE3Failed`, not `publishCommittee`, and therefore leaves publication and failure eligibility overlapping and order-dependent.

Recommendations:

It is recommended to define one consistent deadline model. The preferred model is a hard DKG publication deadline, matching the hard compute and decryption deadlines.

Enforce the deadline authoritatively in `Interfold.onCommitteePublished`:

```
uint256 deadline = _e3Deadlines[e3Id].dkgDeadline;
if (block.timestamp > deadline) {
    revert DKGDeadlinePassed(e3Id, deadline);
}
```

This preserves the existing boundary semantics: publication remains valid at the deadline, while timeout failure becomes available only afterward. The registry may perform the same check before proof verification to avoid unnecessary gas expenditure, but Interfold should remain authoritative. A callback revert atomically rolls back the registry's preceding writes.

If operational tolerance for delayed inclusion is required, define one effective deadline:

```
effectiveDkgDeadline = dkgDeadline + publicationGrace;
```

Publication should be allowed only through that effective deadline, and DKG failure should become available only afterward. A caller-restricted grace period that still permits both transitions does not remove the ordering race.

If deadlines are intentionally optimistic and late DKG publication must remain valid, record the first missed deadline and preserve its economic attribution. A late stage transition must not erase an overdue supplier obligation or allow a later failure to change the payer.

Add regression tests covering publication exactly at the effective deadline, publication immediately afterward, both transaction orderings, and the resulting refund distributions.

Interfold: Resolved with `@741ed6dbbb ...`

Zenith: Verified.

[L-4] Revoking a retained slashing manager can release collateral and strand slash routes

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L180-L217
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L897-L935
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L818-L884

Description:

`BondingRegistry` retains historical slashing managers after rotation so they can process obligations from previously assigned E3s. Exit protection queries only managers that remain authorized:

```
for (uint256 i = 0; i < _authorizedSlashingManagers.length; i++) {
    if (
        ISlashingManager(_authorizedSlashingManagers[i])
            .hasOpenSlashProposal(operator)
    ) {
        revert OperatorUnderSlash();
    }
}
```

`revokeSlashingManager` removes an old manager without checking its assigned E3s, unresolved proposals, pending routes, or historical bans.

Once removed, an open proposal no longer blocks its target's exit. Already-unlocked queued collateral can be claimed immediately; otherwise, the operator can initiate the normal exit delay. The proposal cannot execute while the old manager is unauthorized because its calls to the registry's slash functions revert.

Revocation can also strand funds after a financial slash succeeds but routing to the refund manager fails. The old manager retains the pending route while the corresponding amount remains reserved in `BondingRegistry`. Route retries revert because the revoked manager cannot call `redirectReservedSlashedTicketFunds`. The normal recovery path requires rotating the old manager back into the current-manager position.

Finally, operator registration stops querying bans recorded by the revoked manager, causing those historical bans to be forgotten.

The documented instruction to revoke managers only after their obligations terminate is not enforced on-chain.

Recommendations:

It is recommended to replace immediate removal with explicit `Active`, `Retiring`, and `Removed` states.

A retiring manager should receive no new E3 assignments, but must remain able to accept evidence for previously assigned E3s until their defined accusation windows close, execute existing proposals, complete historical routes, and participate in exit and ban checks.

Final removal should require:

- every assigned E3's accusation window to be closed;
- global open-proposal and pending-route counts to be zero; and
- historical bans to be migrated into canonical storage.

Record route reservations by manager and proposal so a successor can complete or migrate them without restoring all authority to the old manager. Maintain retirement obligations in the registry rather than relying exclusively on external manager queries, and provide a governance-controlled emergency handoff for compromised managers.

Interfold: Resolved with `@9693eb32b1 ...` and `@1914ef40a1 ...`

Zenith: Verified.

[L-5] The whitelisted LiquidityLauncher bypasses pre-TGE transfer restrictions

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/registry/BondingRegistry.sol#L64-L68
- interfold-contracts/contracts/token/InterfoldToken.sol#L690-L712
- interfold-contracts/contracts/token/sale/InterfoldTokenSaleDeployer.sol#L148-L166
- interfold-contracts/contracts/token/sale/InterfoldTokenSaleDeployer.sol#L276-L287

Description:

Before TGE, FOLD permits a transfer whenever either endpoint is whitelisted:

```
bool isWhitelisted = transferWhitelist[from] || transferWhitelist[to];
return !isBonding && !isCcaDistribution && !isWhitelisted;
```

The sale deployer whitelists the LiquidityLauncher for the initial LBP distribution but does not revoke the privilege after distribution completes.

The configured canonical LiquidityLauncher v3 at `0x00004c4ccc709Ef590F7C81102C0689F0263D4e9` exposes permissionless token deposit and distribution functions. Its caller selects an arbitrary strategy, which receives an allowance over tokens held by the launcher.

An account holding transferable, unlocked FOLD can therefore:

1. Transfer or deposit FOLD into the whitelisted launcher.
2. Call `distributeToken` with a malicious strategy.
3. Have the strategy consume its allowance through `transferFrom(launcher, attackerRecipient, amount)`.

Both token movements pass the one-sided whitelist check because the launcher is the recipient of the first transfer and the sender of the second. The launcher's allowance-consumption check does not prevent this because the malicious strategy consumes the full allowance.

The initial sale inventory is distributed atomically and is not exposed by this sequence. The bypass becomes available afterward while the launcher remains whitelisted. Locked FOLD remains protected, and practical exploitation requires an ordinary account to hold unlocked FOLD before TGE.

Recommendations:

It is recommended to revoke the launcher's transfer privilege immediately after the initial distribution and before transferring ownership:

```
ILiquidityLauncher(config.liquidityLauncher).distributeToken(
    fold,
    Distribution({
        strategy: config.lbpStrategy,
        amount: uint128(distributionAmount),
        configData: _lbpConfigData(fold, config)
    }),
    config.distributionSalt
);

IFoldToken(fold).setTransferWhitelisted(
    config.liquidityLauncher,
    false
);

IFoldToken(fold).transferOwnership(protocolAdmin);
```

Also review the LBP strategy and position manager for public forwarding functionality, and revoke each operational whitelist privilege as soon as it is no longer required.

Interfold: Resolved with @c88f4faf91 ...

Zenith: Verified.

[L-6] In-flight deadlines and the DKG fold verifier are not snapshotted

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L264-L300
- packages/interfold-contracts/contracts/Interfold.sol#L373-L388
- packages/interfold-contracts/contracts/Interfold.sol#L768-L781
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L373-L455

Description:

The compute deadline is fixed when an E3 is requested, but the DKG and decryption deadlines are derived from the live global timeout configuration when their respective stages begin.

```
_e3Deadlines[e3Id].computeDeadline =
    requestParams.inputWindow[1] +
    _timeoutConfig.computeWindow;

// During committee finalization:
_e3Deadlines[e3Id].dkgDeadline =
    block.timestamp +
    _timeoutConfig.dkgWindow;

// During ciphertext publication:
_e3Deadlines[e3Id].decryptionDeadline =
    block.timestamp +
    _timeoutConfig.decryptionWindow;
```

The DKG and decryption windows are both used in the request-time fee quote. However, `maxDuration` validation includes only the input horizon, compute window, and decryption window; it already omits sortition and DKG.

An owner can change the global configuration after payment but before the later deadlines are created. DKG changes therefore alter paid-for economics and failure eligibility, while a decryption-window increase can exceed the duration validated at request.

The DKG deadline is an optimistic failure threshold rather than a hard publication deadline. Shortening it enables earlier `markE3Failed` execution—permissionless immediately by default or after the configured grace period—but late `publishCommittee` execution remains possible until failure is recorded.

The registry also verifies fold attestations through its current global `dkgFoldAttestationVerifier`.

```
dkgFoldAttestationVerifier.verify(
    address(this),
    block.chainid,
    e3Id,
    proof,
    dkgAttestationBundle
);

// A later governance action changes the verifier used above.
dkgFoldAttestationVerifier = verifier;
```

The verifier address is the attestation's EIP-712 `verifyingContract`. Rotating V1 to V2 therefore invalidates signatures collected for V1. The two-day timelock reduces likelihood but does not protect an E3 overlapping an already-mature update. Nodes also cache the verifier at startup and continue signing for the old domain until restarted.

The live deadline changes and rejection of a V1-signed bundle after rotation to V2 were reproduced on the current codebase.

Recommendations:

It is recommended to snapshot the complete timeout configuration and `dkgFoldAttestationVerifier` for each E3 at request time. Later lifecycle transitions must use those snapshots.

Define whether `maxDuration` covers the full lifecycle. If it does, include sortition and DKG in the request-time duration calculation and use the same snapshotted values for validation, pricing, and deadlines.

Expose the per-E3 verifier through an event or getter and make nodes resolve it per E3 rather than caching one process-wide address. Verifier rotations should affect only E3s requested after activation. Emergency invalidation of existing work should use an explicit cancellation and refund path.

Interfold: Resolved with `@278f9c5c33 ...` and `@145014a499 ...`

Zenith: Verified.

[L-7] Parameter-set identifiers are not consistently bound across E3 components

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L302-L344
- packages/interfold-contracts/contracts/Interfold.sol#L689-L704
- crates/evm/src/interfold/events.rs#L46-L96
- crates/indexer/src/indexer.rs#L341-L398
- packages/interfold-contracts/scripts/deployInterfold.ts#L454-L461
- scripts/generate-verifiers.ts#L61-L81

Description:

An E3 stores only a `uint8` parameter-set index. At request time, `Interfold` passes the current bytes under that index to the E3 program, but selects the PK and decryption verifiers only by `encryptionSchemeId`.

```
e3.paramSet = requestParams.paramSet;

bytes memory e3ProgramParams =
    paramSetRegistry[requestParams.paramSet];

bytes32 encryptionSchemeId = requestParams.e3Program.validate(
    e3Id,
    seed,
    e3ProgramParams,
    requestParams.computeProviderParams,
    requestParams.customParams
);

IDecryptionVerifier decryptionVerifier =
    decryptionVerifiers[encryptionSchemeId];
IPkVerifier pkVerifier = pkVerifiers[encryptionSchemeId];
```

The components resolving this index do not share one immutable source of truth:

- The default and CRISP E3 programs hash the registry bytes received at request time.
- Ciphernodes ignore the registry bytes and map indexes `0` and `1` to fixed, compiled `BfvPreset` values.
- When processing `CommitteePublished`, the indexer queries the live `paramSetRegistry[e3.paramSet]` value.
- The on-chain generated verifiers are preset-dependent, but only one verifier pair can be registered for the BFV encryption-scheme identifier at a time.

The standard deployment registers both `Insecure512` and `Secure8192`, even though a deployed generated verifier corresponds to only one preset. A requester can therefore select the other registered preset, have ciphernodes generate proofs for that preset, and reach a verifier compiled for different parameters. This path does not require an owner overwrite.

The owner can introduce additional divergence because existing parameter entries are explicitly mutable:

```
bytes memory previous = paramSetRegistry[paramSet];
paramSetRegistry[paramSet] = encodedParams;
```

If an entry is replaced after a request, the E3 program remains bound to the request-time hash, ciphernodes continue using the preset compiled for the numeric index, and the indexer can retrieve the replacement bytes. Because the indexer performs an unpinned live query while processing `CommitteePublished`, even an update mined after publication but before event processing can affect the parameters it validates and stores.

Arbitrary nonempty indexes may also be registered and accepted by E3 programs that do not reject them, including the default program, while current ciphernodes skip indexes other than `0` and `1`.

These inconsistencies are expected to fail closed through rejected public-key validation, incompatible ciphertexts or proofs, skipped requests, and DKG or computation timeouts. No invalid-proof acceptance or direct theft of funds is demonstrated.

Recommendations:

It is recommended to replace the independently interpreted numeric index with an immutable cryptographic configuration identifier binding the encryption scheme, canonical parameter hash, circuit version, and compatible verifier contracts. Requests should only accept configurations explicitly activated for new E3s and should snapshot that identifier and its verifier addresses.

Parameter bytes and their hashes must be append-only. Updates should register a new configuration; old configurations may be disabled for new requests but must remain available unchanged for existing E3s.

```
bytes32 configId = keccak256(
    abi.encode(
        encryptionSchemeId,
        paramSetHash,
        circuitVersion
    )
);

CryptoConfig storage config = cryptoConfigs[configId];
require(config.enabledForNewRequests, UnsupportedCryptoConfig(configId));

e3.cryptoConfigId = configId;
e3.paramSetHash = config.paramSetHash;
e3.pkVerifier = config.pkVerifier;
e3.decryptionVerifier = config.decryptionVerifier;
```

Ciphernodes should require their locally compiled parameter and circuit artifacts to match the emitted configuration hash. The indexer should consume the request-time configuration snapshot or verify exact emitted bytes against that hash instead of querying mutable latest state.

Interfold: Resolved with `@1914ef40a1 ...`

Zenith: Verified.

[L-8] Fee-on-transfer assets create underfunded fee and slash liabilities

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/token/InterfoldTicketToken.sol#L251-L295
- packages/interfold-contracts/contracts/token/InterfoldTicketToken.sol#L322-L343
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L693-L716
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L864-L897
- packages/interfold-contracts/contracts/E3RefundManager.sol#L484-L500
- packages/interfold-contracts/contracts/E3RefundManager.sol#L824-L847
- packages/interfold-contracts/contracts/Interfold.sol#L288-L347

Description:

This issue requires governance to configure a fee-on-transfer or rebasing asset, or an approved token to enable such behavior after configuration. Current deployments use an exact-transfer token and are not presently affected. However, the ticket wrapper explicitly implements and tests fee-on-transfer deposit support, while the production stablecoin has not yet been fixed.

`InterfoldTicketToken` mints the actual amount received during deposits. Its payout path instead decreases `payableBalance` and transfers a nominal amount without checking the recipient's balance increase.

```
function payout(address to, uint256 amount)
    external onlyRegistry nonReentrant {
    require(amount <= payableBalance, "Exceeds payable balance");
    payableBalance -= amount;
    SafeERC20.safeTransfer(IERC20(address(underlying())), to, amount);
    emit Payout(to, amount);
}
```

This creates an inconsistency during slashed-fund routing:

1. `BondingRegistry` reserves the nominal amount slashed.

2. `redirectReservedSlashedTicketFunds` decreases the reservation and calls `payout(refundManager, amount)`.
3. A transfer fee causes the refund manager to receive less than `amount`.
4. `SlashingManager` nevertheless calls `escrowSlashedFunds` with the nominal amount.
5. `E3RefundManager` creates a slashed-fund liability for that nominal amount.

```
dependencies.bonding.redirectReservedSlashedTicketFunds(
    address(dependencies.refundManager),
    route.amount
);
dependencies.interfoldContract.escrowSlashedFunds(
    route.e3Id,
    IERC20(route.token),
    route.amount
);
```

`E3RefundManager` checks its balance after increasing `_tokenLiability`. However, `_tokenLiability` covers only pending and settled slashed-fund obligations. Requester refunds, honest-node rewards, and base treasury credits are recorded separately but omitted from this aggregate solvency floor.

```
function _increaseTokenLiability(IERC20 token, uint256 amount) internal {
    _tokenLiability[token] += amount;
    _assertTokenSolvent(token);
}

function _assertTokenSolvent(IERC20 token) internal view {
    uint256 liability = _tokenLiability[token];
    uint256 balance = token.balanceOf(address(this));
    if (balance < liability) {
        revert InsolventToken(token, liability, balance);
    }
}
```

If the refund manager has no unrelated balance of that token, the solvency assertion reverts. The payout and escrow execute in one nested transaction, so the transfer and accounting changes roll back while the slash route remains pending. Retries continue to revert while the fee remains enabled and no surplus covers the shortfall.

If the refund manager holds enough of the same token for base distributions, that balance can mask the deficient slash receipt. Base claims may proceed only while the remaining balance still covers the nominal slashed liability. The residual base funds then become reserved for the overcredited slash claims, potentially leaving later requester, honest-node,

or base treasury claims underfunded. Outbound transfer fees can additionally cause slash claimants to receive less than their recorded entitlement.

A contract integration fixture using the real ticket, bonding, slashing, Interfold, and refund accounting contracts with mock cryptographic verification reproduced the isolated failure. With a 1% transfer fee and a nominal `50,000,000` ticket slash, the route remained pending, the full `50,000,000` remained reserved in `BondingRegistry`, and the refund manager received and credited zero because the underfunded route reverted atomically.

The same nominal-accounting problem exists during E3 fee collection. `Interfold` records the quoted fee in `e3Payments` and then calls `safeTransferFrom` without measuring its balance increase. A fee-on-transfer token therefore creates rewards or refunds exceeding the assets received. Since Interfold pools tokens for multiple E3s without an aggregate liability floor, other E3 balances may temporarily mask the deficit and be consumed by earlier claims.

The owner-controlled token allowlist does not prevent this behavior because it approves addresses rather than enforcing transfer semantics. The active fee token is also automatically allowlisted when configured.

Recommendations:

It is recommended to support only exact-transfer, non-rebasing assets unless the complete custody and settlement system is converted to net-receipt accounting.

For an exact-transfer policy:

- measure every custody contract's balance before and after an inbound transfer and require `received = expected`;
- measure the recipient's balance around outbound transfers and require the complete nominal amount to be received;
- revert ticket deposits, ticket payouts, E3 fee collection, and refund-manager funding on any mismatch;
- enforce these checks at runtime because an allowlisted token may later enable fees;
- reject rebasing assets explicitly, as transfer-local delta checks cannot protect liabilities from later negative rebases.

If fee-on-transfer assets must be supported:

- measure the refund manager's balance around slash routing;
- preserve the nominal slash for penalty records but escrow and distribute only the measured amount received;
- emit both nominal and received amounts;
- record actual E3 fee receipts, or gross up payments only where the token's fee is predictable;
- measure Interfold-to-refund-manager receipts before calculating base distributions;
- define whether outbound fees reduce the recipient's entitlement or must be grossed up.

Each custody contract should maintain a complete per-token liability aggregate without double-counting obligations as they move between states:

- `Interfold`: active E3 payments, pending node rewards, and treasury credits;
- `E3RefundManager`: requester, node, treasury, pending-slash, and settled-slash obligations;
- `InterfoldTicketToken`: `totalSupply` plus `payableBalance`.

Add tests covering isolated route failure, a route masked by base escrow, base-versus-slash claim ordering, cross-E3 fee subsidization, outbound short payment, and tokens that enable fees after being allowlisted.

Interfold: Resolved with `@49358ebf18 ...`

Zenith: Verified.

[L-9] Solidity does not enforce Rust's active-job ticket-capacity adjustment

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L248-L288
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L1028-L1056
- crates/sortition/src/sortition/node_registry.rs#L57-L76
- crates/sortition/src/sortition/node_registry.rs#L186-L209
- crates/sortition/src/sortition/selection_backend.rs#L84-L130

Description:

Rust reduces an operator's ticket range according to its active E3 jobs:

```
available tickets = floor(ticket balance / ticket price) − active jobs
```

The reduced range is used during honest sortition. More active jobs therefore give the operator fewer candidate ticket scores, and an operator with no remaining capacity is excluded.

```
let total_tickets = (node.ticket_balance / self.ticket_price)
    .try_into()
    .unwrap_or(0u64);

total_tickets.saturating_sub(node.active_jobs)
```

Solidity does not apply this adjustment. It validates ticket IDs using the operator's full historical balance:

```
uint256 ticketBalance = e3Bonding.getTicketBalanceAtBlock(
    node,
    c.requestBlock - 1
);
uint256 availableTickets = ticketBalance / ticketPrice;
```

```
require(ticketNumber <= availableTickets, InvalidTicketNumber());
```

There is no on-chain active-job counter, capacity adjustment, or cross-E3 ticket-use restriction. A modified operator can select its best score from the full ticket range while honest operators with active jobs select from a smaller range. This gives the modified operator a better expected score for the same ticket balance.

A focused test confirmed that the same operator and ticket ID could be submitted successfully to two concurrent requested E3s. The test did not finalize and publish both committees, but the finalization path contains no capacity revalidation, so no later contract step enforces Rust's adjustment.

The mismatch also creates an honest liveness risk. `requestCommittee()` checks `numActiveOperators()` without considering Rust-side remaining capacity. A request can therefore be accepted even though too few honest operators will submit because their locally calculated available-ticket count is zero.

Rust increments `active_jobs` only when `CommitteePublished` is observed. Committee selection, finalization, and DKG work before key publication consume no capacity, even for honest nodes. The mechanism therefore does not fully account for concurrent work under its current timing.

All concurrent E3 penalties draw from the same finite ticket balance. `slashTicketBalance()` caps each penalty at the remaining active and pending balance, so successive penalties can become partial or zero after depletion. This constitutes undercollateralization only if each active job is intended to consume a ticket-backed unit of slashable capacity.

The intended semantics are ambiguous. The contracts README describes tickets as "USDC-backed sortition capacity deposits," while the flow documentation describes `active_jobs` as load balancing. If it is only a voluntary load-balancing heuristic, modified nodes ignoring it are expected and the collateral impact does not apply. The client divergence and potential honest liveness failure nevertheless remain.

Recommendations:

It is recommended to define whether `active_jobs` represents enforceable protocol capacity or voluntary local load balancing.

If it represents security-relevant capacity:

- implement a canonical on-chain active-job or reserved-capacity counter;
- reserve capacity no later than committee finalization so DKG work is included;
- atomically revalidate capacity when concurrent committees finalize;
- retain sufficient score-ordered backup candidates if an unavailable candidate must be skipped;
- release capacity only when the E3 reaches `Complete` or `Failed`;
- prevent reserved collateral from being withdrawn;

- include remaining capacity when determining whether a committee request can be fulfilled; and
- use the same calculation and lifecycle boundaries in Solidity and Rust.

Also define how one capacity unit relates to configurable slashing penalties. Subtracting one ticket per job does not guarantee complete collateralization when one penalty can exceed one ticket's value.

If `active_jobs` is only load balancing:

- document that it is not a security, eligibility, or collateral invariant;
- do not use other operators' active-job counts to construct a globally expected committee;
- apply workload limits as each node's local participation policy; and
- document that the on-chain submitted set is authoritative and modified nodes may ignore the heuristic.

Add cross-language tests covering repeated ticket use, concurrent committee finalization, capacity exhaustion, reservation release, and requests where the active-operator count exceeds honest remaining capacity.

Interfold: Resolved with `@8e555eac21..` and `@d3c0a4cd3a...`

Zenith: Verified.

[L-10] Non-canonical BN254 inputs let a custom prover decouple proven coefficients from published plaintext

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- contracts/Interfold.sol#L432-L440
- contracts/verifiers/bfv/BfvDecryptionVerifier.sol#L203-L231
- contracts/verifiers/bfv/honk/DecryptionAggregatorVerifier.sol#L284-L295
- contracts/verifiers/bfv/honk/DecryptionAggregatorVerifier.sol#L714-L750
- contracts/verifiers/bfv/honk/DecryptionAggregatorVerifier.sol#L2546-L2569

Description:

`Interfold.publishPlaintextOutput()` relies on `BfvDecryptionVerifier` to prove that the submitted plaintext is the result authenticated by the decryption circuit. However, the generated Honk verifier and the wrapper interpret message public inputs differently.

The generated verifier reduces each public input modulo the BN254 scalar-field modulus before using it in the proof relation. The wrapper instead truncates the unreduced input to 64 bits when reconstructing the plaintext.

```
function fromBytes32(bytes32 value) internal pure returns (Fr) {
    return Fr.wrap(uint256(value) % MODULUS);
}

Fr pubInput = FrLib.fromBytes32(publicInputs[i]);

uint64 coeff = uint64(uint256(publicInputs[offset + i]));
```

For a circuit-proven coefficient `x`, a non-canonical public input can be constructed as:

```
y = x + k * MODULUS
```

For the small decoded coefficients produced by the C7 circuit, up to five such aliases fit in 256 bits. The proof relation interprets `y` as `x`, while the wrapper hashes `uint64(y)`. Since the BN254 modulus is not divisible by `2^64`, these plaintext bytes differ. For example, `MODULUS`

represents zero in the circuit but becomes the 64-bit wrapper value `0x43e1f593f0000001`.

The raw public inputs are included in the Fiat—Shamir transcript. An attacker therefore cannot modify an existing honest proof. This was confirmed by replacing `0x15` with `0x15 + MODULUS` in a genuine accepted proof, which caused `SumcheckFailed()`.

Exploitation requires a party possessing valid decryption-aggregation witness data to use a modified prover. The prover must place `y` in the transcript while using `x = y mod MODULUS` in the field relation and public-input delta. The on-chain verifier does not enforce that both representations are identical.

This does not provide arbitrary plaintext control or directly steal protocol funds. It allows the prover to select among several non-canonical byte encodings for each proven coefficient. If the resulting proof is accepted, `Interfold` stores those bytes, moves the E3 to `Complete`, and distributes committee rewards. The normal protocol lifecycle cannot subsequently replace the incorrect output.

Recommendations:

It is recommended to reject non-canonical public inputs in `BfvDecryptionVerifier` immediately after validating the array length and before hashing, comparing, or forwarding them to the generated verifier.

```
uint256 internal constant BN254_SCALAR_MODULUS =
21888242871839275222246405745257275088548364400416034343698204186575808495617;

error NonCanonicalPublicInput(uint256 index);

for (uint256 i = 0; i < publicInputs.length; ++i) {
    if (uint256(publicInputs[i]) >= BN254_SCALAR_MODULUS) {
        revert NonCanonicalPublicInput(i);
    }
}
```

Also require each message coefficient to be below the BFV plaintext modulus bound to the deployed circuit. If that modulus is unavailable in the wrapper, requiring an exact `uint64` representation is a weaker defense-in-depth measure.

Implement the check in the maintained wrapper or a shared field-encoding library so verifier regeneration cannot remove it. Add tests confirming that canonical proofs pass and every `x + k * MODULUS` representation reverts before circuit verification.

Interfold: Resolved with `@fcea01eeba ...`

Zenith: Verified.

[L-11] Verifier callers do not defensively reject false results from non-conforming implementations

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L348-L399
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L76-L107
- packages/interfold-contracts/contracts/interfaces/IPkVerifier.sol#L39-L59
- packages/interfold-contracts/contracts/interfaces/IDecryptionVerifier.sol#L49-L67

Description:

The public-key and decryption-verifier interfaces return a boolean indicating whether verification succeeded. Their callers invoke `verify` as a statement and discard the result:

```
e3.pkVerifier.verify(
    e3Id,
    committeeRoot,
    sortedNodes,
    pkCommitment,
    committeeHash,
    proof
);

IDecryptionVerifier(verifierAddress).verify(
    e3Id,
    decryptionDomain,
    plaintextHash,
    committeeHash,
    ciphertextCommitment,
    proof
);
```

A verifier that returns ABI-encoded `false` without reverting would therefore be treated as successful.

Both interfaces explicitly document a stricter semantic convention: successful verification

returns `true`, while every failure reverts. The current `BfvPkVerifier` and `BfvDecryptionVerifier` implementations conform to this convention by converting a rejected underlying proof into `InvalidProof()` and returning only `true`. No current production-verifier bypass was identified.

Reachability requires the trusted owner to configure a future or alternative verifier that is ABI-compatible but violates the documented convention by returning `false`. Verifier addresses are snapshotted when an E3 is requested, so such a configuration affects subsequent E3s rather than existing requests.

For decryption verification, accepting `false` could directly allow an invalid plaintext proof to pass the verifier integration boundary. The DKG path has an additional fold-attestation requirement after public-key proof verification, so ignoring a false public-key result does not independently permit publication without also satisfying that attestation layer.

A malicious owner could already install an always-accepting verifier. This issue therefore concerns accidental integration mismatch and defense in depth, not protection against compromised governance.

Recommendations:

It is recommended to defensively enforce both boolean results:

```
if (
    !e3.pkVerifier.verify(
        e3Id,
        committeeRoot,
        sortedNodes,
        pkCommitment,
        committeeHash,
        proof
    )
) {
    revert IPkVerifier.InvalidProof();
}

if (
    !IDecryptionVerifier(verifierAddress).verify(
        e3Id,
        decryptionDomain,
        plaintextHash,
        committeeHash,
        ciphertextCommitment,
        proof
    )
) {
    revert IDecryptionVerifier.InvalidProof();
```

```
}
```

Alternatively, remove `returns (bool)` from both wrapper interfaces and define successful return versus revert as the only permitted contract. This matches the existing production implementations more clearly, but source interfaces, mocks, generated bindings, and integrations must be updated accordingly.

Add tests using ABI-compatible verifiers that return `false` and confirm that both public-key publication and plaintext publication revert.

Interfold: Resolved with `@d90a09b3f8 ...`

Zenith: Verified.

[L-12] Operators can claim ticket exits before submitting snapshot-weighted sortition tickets

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/registry/BondingRegistry.sol#L500-L625
- interfold-contracts/contracts/registry/BondingRegistry.sol#L803-L816
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L539-L583
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L678-L688
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L1019-L1057

Description:

Ticket submissions are weighted using the operator's historical balance at `requestBlock - 1`. The live eligibility check only requires the operator to retain the configured minimum ticket balance:

```
uint256 ticketBalance = e3Bonding.getTicketBalanceAtBlock(
    node,
    c.requestBlock - 1
);
uint256 availableTickets = ticketBalance / ticketPrice;

require(ticketNumber <= availableTickets, InvalidTicketNumber());
```

After this snapshot, an operator can burn most of its active tickets and queue the corresponding underlying tokens for exit while retaining enough tickets to remain active:

```
ticketToken.burnTickets(msg.sender, amount);
_exits.queueTicketsForExit(msg.sender, exitDelay, amount);
_updateOperatorStatus(msg.sender);
```

Queued tickets remain slashable until claimed. However, `claimExits` does not account for open sortition requests. When:

```
removalTime + exitDelay <= requestTime + sortitionSubmissionWindow
```

the operator can claim the queued collateral and subsequently submit a ticket derived from its larger historical balance.

For example, the operator can hold `1,000 × ticketPrice` at the snapshot, retain only the active minimum, claim the remaining exit after its delay, and still submit any historical ticket number up to 1,000 before the inclusive submission deadline.

The condition is reachable because `exitDelay` and `sortitionSubmissionWindow` are configured independently. In particular, the contracts permit a one-day exit delay and a seven-day submission window. Equality is also unsafe when removal occurs in the request timestamp because claiming and submission can both succeed at the deadline.

This does not necessarily mean that the operator must retain its entire historical balance throughout the resulting E3. Ticket penalties are fixed governance-configured amounts, and the off-chain capacity model accounts for one ticket per active job. The concrete security impact arises when the remaining collateral is less than the protocol-defined per-duty requirement or an applicable ticket penalty. In that case, an operator can assume committee duties without enough ticket collateral remaining to enforce the intended penalty.

The tracked deployment arguments use a seven-day exit delay and a ten-second submission window, so this exact sequence is not enabled by those recorded initial values. Both values are mutable, and the deployment record does not prove their current on-chain state.

Recommendations:

It is recommended to enforce the strict cross-contract timing invariant:

```
exitDelay > sortitionSubmissionWindow
```

Validate this relationship during initialization and whenever either value is changed. Using $\geqslant$ is insufficient because exits unlock at equality while ticket submission remains permitted through the deadline.

As defense in depth, validate and reserve collateral when an operator submits:

- If the complete snapshot weight is intended to remain economically backed during sortition, require that weight to remain slashable and reserve it until the candidate is displaced or committee selection is finalized.
- For selected operators, retain a protocol-defined per-duty reservation through E3 and dispute finality. The reservation should be no smaller than the maximum ticket penalty that can apply to the duty.
- Release excess snapshot-weight collateral after finalization if the full historical balance is not intended to secure one committee duty.
- Aggregate reservations across concurrent E3s so the same collateral cannot support

multiple obligations.
- Prevent claims from consuming active or pending ticket collateral covered by a reservation.

If governance can change slash penalties during an active duty, either snapshot those penalties per E3 or cap them so they cannot exceed the reserved collateral.

Add regression tests covering a one-day exit delay with a seven-day submission window, equality between both windows, same-timestamp removal, claim-before-submission, collateral below the applicable penalty, concurrent duty reservations, candidate displacement, and release after dispute finality.

Interfold: Resolved with `@9fbd231394 ...` and `@50adaff2f6 ...`

Zenith: Verified.

[L-13] Donations can grief non-atomic license-token rotation

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L850-L887

Description:

When replacing an existing license token, `setLicenseToken` treats the contract's complete old-token balance as outstanding liabilities:

```
uint256 liabilities = current.balanceOf(address(this));
if (liabilities ≠ 0) {
    revert OutstandingAssetLiabilities(address(current), liabilities);
}
```

ERC-20 transfers do not require recipient consent. Any holder with transferable tokens can therefore donate one base unit to `BondingRegistry` and make rotation revert despite `totalLicenseLiability == 0`. `InterfoldToken` specifically permits transfers involving its configured bonding registry, including before TGE.

`sweepLicenseSurplus` removes unsolicited tokens without affecting liabilities, but it is a separate transaction. An observer can donate again between a standalone sweep and rotation, repeatedly delaying a non-atomic rotation at negligible cost.

This does not create claims, steal funds, or affect ordinary bonding. It only delays a rare owner-controlled configuration operation. The configured Safe can already eliminate the interleaving window by executing `sweepLicenseSurplus` and `setLicenseToken` atomically through MultiSend. Initial assignment from the zero-address placeholder is also unaffected.

Recommendations:

It is recommended to require every subsequent license-token rotation to use an atomic Safe batch containing `sweepLicenseSurplus` followed by `setLicenseToken`.

Alternatively, enforce atomicity inside `BondingRegistry`:

```
error OldLicenseTokenBalanceRemaining(address token, uint256 balance);

function rotateLicenseToken(
    IERC20 next
) external onlyOwner nonReentrant {
    address nextAddress = address(next);
    if (nextAddress.code.length == 0) {
        revert InvalidBondingAsset(nextAddress);
    }

    IERC20 current = licenseToken;
    if (current == next) return;

    uint256 liabilities = totalLicenseLiability;
    if (liabilities != 0) {
        revert OutstandingAssetLiabilities(
            address(current),
            liabilities
        );
    }

    uint256 surplus = current.balanceOf(address(this));
    if (surplus != 0) {
        current.safeTransfer(slashedFundsTreasury, surplus);
    }

    uint256 remaining = current.balanceOf(address(this));
    if (remaining != 0) {
        revert OldLicenseTokenBalanceRemaining(
            address(current),
            remaining
        );
    }

    licenseToken = next;
    emit LicenseTokenSet(nextAddress);
}
```

Do not rely on `totalLicenseLiability` for a legacy proxy until the bonding-storage migration has initialized and reconciled it.

If governance must be able to escape a non-transferable or blacklisting old token, permit rotation once verified liabilities are zero and retain the abandoned token in a separate retired-asset rescue ledger instead of requiring its raw balance to be zero. This path must never abandon claimant liabilities.

Keep initial zero-placeholder assignment separate and add tests proving that a donation be-

tween standalone transactions blocks rotation while an atomic Safe batch or atomic rotation function succeeds.

Interfold: Resolved with `@9d1b649c6f ...`

Zenith: Verified.

[L-14] Storage validation accepts unsafe dynamic-array stride changes and enum reordering

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/scripts/storageLayouts.ts#L182-L275
- packages/interfold-contracts/scripts/snapshotStorageLayouts.ts#L26-L60
- packages/interfold-contracts/contracts/lib/ExitQueueLib.sol#L23-L57

Description:

The custom storage-layout validator does not compare a struct's `numberOfBytes` when the old type exposes members. It checks existing members but permits additional members:

```
if (before.members) {
  for (const oldMember of before.members) {
    // Check only existing member location and type.
  }
} else if (before.numberOfBytes !== after.numberOfBytes) {
  errors.push(/* ... */);
}
```

This produces a demonstrated false negative when a struct determines the stride of a dynamic array. `ExitQueueState.operatorQueues` stores `ExitTranche[]`, and the current `ExitTranche` occupies three storage slots. Appending a field increases that stride.

After such an upgrade, element zero's existing fields remain readable, but its new field aliases legacy data belonging to element one. Existing fields in subsequent elements are read from incorrect slots. A synthetic dynamic-array element increase from 32 to 64 bytes returned no errors from `diffLayouts`.

The upgrade initially reinterprets existing storage; permanent corruption occurs when the incompatible implementation writes through the new layout. No current `ExitTranche` stride incompatibility was identified.

Imported persisted enums form a second false-negative class. Solc's stored type metadata contains their label and byte width but not their ordered member definitions. Inserting or reordering same-width enum members can therefore pass validation while changing the

meaning of existing numeric values.

The baseline generator authenticates the primary contract source against the supplied commit but does not authenticate the complete compiler input, including imported sources. This is a provenance limitation; the candidate source hash should not be compared directly with the old baseline because legitimate upgrades change source. Instead, the old build must be reproducible from its archived complete input and the candidate must be compiled freshly.

ERC-7201 namespaced storage used by upgradeable dependencies is also outside the custom snapshot schema.

Appending a field directly to `ExitQueueState` is a different case. Because `_exits` precedes later `BondingRegistry` state, increasing its top-level size moves those variables and should be detected when the validator uses the correct deployed baseline.

Recommendations:

It is recommended to make recursive comparison aware of whether a type determines dynamic-array element stride. Reject any byte-size change at that boundary, including changes reached through nested types:

```
if (
  context.requiresStableArrayStride &&
  before.numberOfBytes !== after.numberOfBytes
) {
  errors.push(
    `${contract}: ${pathLabel} array-element stride changed`,
  );
}
```

Continue validating all existing member positions and types. Do not reject every appended struct member categorically: additions to some mapping-valued or packed structs can be storage-compatible when they neither change array stride nor move existing state.

Integrate a Hardhat-compatible OpenZeppelin upgrade validator or `@openzeppelin/upgrades-core` as defense-in-depth. Retain explicit negative tests proving that both validators reject dynamic-array element growth, enum insertion or reordering, nested persisted-type changes, and unsafe namespace changes.

Archive and fingerprint the complete standard compiler input for each deployed baseline, including all imported source contents and compiler settings. Extract persisted-enum definitions from the AST and compare their ordered members explicitly. Validate ERC-7201 namespaces and pin relevant dependency versions.

Do not resize `ExitTranche` or append fields to `ExitQueueState` for an existing deployment. Add future queue metadata as new top-level `BondingRegistry` variables that correctly consume the reserved gap, or place it in a new ERC-7201 namespace. Use an explicit migration

if existing queue data must move.

Interfold: Resolved with `@cc0b6243ad ...`

Zenith: Verified.

[L-15] `SlashExecuted` overstates partial collateral slashes

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L761-L778
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L851-L860
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L587-L605
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L641-L657

Description:

Slash proposals snapshot the ticket and license penalties configured by policy. During execution, BondingRegistry limits each penalty to the operator's available active and pending collateral.

```
uint256 actualSlashAmount = Math.min(
    requestedSlashAmount,
    totalAvailableBalance
);
```

SlashingManager captures the actual ticket slash returned by `slashTicketBalance`, but does not retain the actual license slash because `slashLicenseBond` returns no value.

```
actualTicketSlashed = dependencies.bonding.slashTicketBalance(
    p.operator,
    p.ticketAmount,
    p.reason
);

dependencies.bonding.slashLicenseBond(
    p.operator,
    p.licenseAmount,
    p.reason
);
```

`SlashExecuted` nevertheless emits the requested proposal amounts rather than the amounts

actually removed:

```
emit SlashExecuted(
    proposalId,
    p.e3Id,
    p.operator,
    p.reason,
    p.ticketAmount,
    p.licenseAmount,
    true,
    lane
);
```

This is reachable whenever a configured penalty exceeds the operator's remaining collateral, including after prior slashes have depleted it. The event may report a full penalty even when the slash was partial or zero.

On-chain accounting and ticket-fund routing remain correct because they use actual amounts. BondingRegistry also emits actual balance deltas through `TicketBalanceUpdated` and `LicenseBondUpdated`. However, indexers and accounting or monitoring integrations that treat `SlashExecuted` as the authoritative execution result record incorrect values unless they reconcile multiple events.

`SlashProposed` already preserves the requested policy amounts, so reporting actual values in `SlashExecuted` does not lose proposal information.

Recommendations:

It is recommended to make `slashLicenseBond` return its actual slashed amount, capture both actual values in SlashingManager, and emit those values from `SlashExecuted`.

```
uint256 actualLicenseSlashed;

if (p.licenseAmount > 0) {
    actualLicenseSlashed = dependencies.bonding.slashLicenseBond(
        p.operator,
        p.licenseAmount,
        p.reason
    );
}

emit SlashExecuted(
    proposalId,
    p.e3Id,
    p.operator,
```

```
        p.reason,
        actualTicketSlashed,
        actualLicenseSlashed,
        true,
        lane
    );
```

Update `IBondingRegistry.slashLicenseBond` to return `uint256 actualAmount`. This does not change the function selector, although interfaces and callers must be recompiled.

Add tests covering ticket and license penalties against insufficient active collateral, insufficient active-plus-pending collateral, and zero remaining collateral.

Interfold: Resolved with `@83c66ec38f ...`

Zenith: Verified.

[L-16] Static registry writers cannot serve concurrent E3 registry generations

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L274-L283
- crates/events/src/interfold_event/e3_requested.rs#L14-L34
- crates/evm/src/ciphernode_registry/actor.rs#L55-L100
- crates/evm/src/ciphernode_registry/handlers.rs#L21-L107

Description:

Interfold snapshots the current ciphernode registry when it creates an E3. An E3 therefore remains bound to its original registry after governance configures a replacement.

The runtime `E3Requested` payload does not include that request-time registry. Each registry writer instead stores one static registry address and accepts relevant effects based only on their chain ID.

```
if self.provider.chain_id() == data.e3_id.chain_id() {
    ctx.notify(data);
}

// The effect is submitted to the writer's static registry.
submit_ticket_to_registry(provider, contract_address, e3_id,
    ticket_id).await;
```

Consequently, a node cannot safely process E3s assigned to old and new registry generations at the same time. For example:

1. Interfold creates E3 `x` through registry `R1`.
2. Governance rotates Interfold to registry `R2`.
3. E3 `x` remains bound to `R1`.
4. A node configured for `R2` sends remaining registry actions for E3 `x` to `R2`.
5. A node kept on `R1` sends actions for new E3s to `R1`.

The issue affects an old E3 only while it still requires a registry-dependent action. These actions include ticket submission, committee finalization, committee-event ingestion, and committee public-key publication. Calls sent to the wrong registry normally revert or fail their preflight checks.

Running separate `R1` and `R2` nodes does not resolve the issue under the normal configuration. Both nodes consume all `E3Requested` events from the same Interfold, and the events do not identify their assigned registry. The runtime has no supported generation filter. Duplicate same-chain configuration is also unsafe because multiple writers subscribe to the same chain-scoped effects.

The likely consequences are E3 timeouts, requester refunds, wasted operator work and gas, and reduced committee availability. No theft or proof-verification bypass was established.

Recommendations:

It is recommended to bind each runtime E3 to its request-time registry. Add the registry address to the on-chain `E3Requested` event or expose a public view that returns the snapshotted registry for an E3. Decode and persist this address as an `E3id → registry` route.

Keep readers and writers for old and new registries active until every E3 assigned to the old registry is terminal. Before processing an effect, each writer must verify that its registry address matches the E3's persisted route. Preserve the emitting contract address when EVM logs become typed events.

Reject duplicate resolved chain IDs at startup because duplicate chain entries are not a safe rotation mechanism.

If the protocol does not support hot rotation, enforce a drain-before-rotation process instead. Pause new requests, wait for all old E3s and slash routes to terminate, migrate membership, rotate the registry, update node configuration, and then resume requests. Update the dependency-rotation documentation and tests to state this restriction.

Interfold: Resolved with `@145014a499 ...`

Zenith: Verified.

[L-17] `isCiphernodeEligible` queries the global bonding registry instead of the per-E3 snapshot

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- CiphernodeRegistryOwnable

Description

`isCiphernodeEligible` queries the current global `bondingRegistry.isActive(node)`, but `submitTicket` enforces eligibility against the per-E3 snapshotted `_bondingFor(e3Id)`. After a `setBondingRegistry` call, off-chain systems using this view to predict `submitTicket` acceptance receive stale answers from the new registry while in-flight E3s still enforce the old one.

```
File: CiphernodeRegistryOwnable.sol
782:     function isCiphernodeEligible(address node)
    public view returns (bool) {
783:         if (!isEnabled(node)) return false;
784:
785:         require(
786:             address(bondingRegistry) != address(0),
787:             BondingRegistryNotSet()
788:         );
789:         return bondingRegistry.isActive(node);
790:     }
```

No on-chain exploit results, but any off-chain system (including the Rust ciphernode layer) relying on this function for E3-specific eligibility decisions will produce incorrect results after a registry rotation.

Recommendations

Add an `e3Id`-parameterized overload that queries `_committeeDependencies[e3Id].bonding.isActive(node)`, or document the global-only semantics.

Interfold: Resolved with PR-1768.

Zenith: Verified.

[L-18] Documented requester cancellation has no requester-accessible implementation

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Medium

Target

- docs/pages/requestor-guide.mdx#L126-L128
- packages/interfold-contracts/contracts/Interfold.sol#L803-L920
- packages/interfold-contracts/contracts/E3RefundManager.sol#L281-L347

Description:

The requester guide states that requesters can cancel an E3 before computation starts. It also states that cancellation uses `RequesterCancelled` and provides a partial refund.

However, `IInterfold`, `Interfold`, and the SDK expose no requester cancellation function. `markE3Failed()` does not provide this functionality. It requires an expired stage deadline and derives the failure reason from the current stage.

```
function markE3Failed(
    uint256 e3Id
) external returns (FailureReason reason) {
    E3Stage current = _e3Stages[e3Id];

    InterfoldPricing.validateMarkFailedStage(e3Id, uint8(current));

    bool canFail;
    uint256 deadline;
    (canFail, reason, deadline) = _checkFailureCondition(e3Id, current);
    if (!canFail) revert FailureConditionNotMet(e3Id);

    // ...

    _markE3FailedWithReason(e3Id, current, reason);
}
```

Although `onE3Failed()` accepts `RequesterCancelled`, only the E3's registry or slashing manager can call it. A requester cannot use that function. A configurable slash policy can

conditionally supply this reason, but that is not a requester-controlled cancellation path.

Consequently, a requester cannot stop an in-flight E3 through the protocol or obtain the documented cancellation refund. The remaining escrowed fee can be spent if the E3 continues successfully. The loss cannot exceed the fee that the requester already paid.

The failure is deterministic when a requester tries to cancel, but the frequency of cancellation attempts is unknown. No attacker gains additional authority. This is primarily a missing-functionality and documentation issue.

Recommendations:

It is recommended to decide whether requester cancellation is a supported protocol feature.

If cancellation is supported, add a function that:

- permits only the original requester;
- defines an exact cancellation stage and timestamp;
- uses the standard failed-state transition and terminal events;
- prevents repeated cancellation and races with output publication; and
- preserves permissionless failure processing or processes the refund atomically.

The existing refund logic always treats `RequesterCancelled` as a failure at `KeyPublished`. This is correct only if cancellation is restricted to `KeyPublished`, before `inputWindow[1]`. If cancellation is allowed during earlier stages, calculate the refund from the actual stage at which cancellation occurs.

If cancellation is not supported, remove the promise from the requester guide and mark `RequesterCancelled` as reserved or deprecated.

Interfold: Resolved with `@f98a28f690 ...`

Zenith: Verified.

[L-19] Unmigrated registry rotation desynchronizes operator membership and committee capacity

SEVERITY: Low	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/Interfold.sol#L347-L355
- interfold-contracts/contracts/registry/BondingRegistry.sol#L494-L575
- interfold-contracts/contracts/registry/BondingRegistry.sol#L1027-L1031
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L254-L288
- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L490-L560

Description:

`BondingRegistry.setRegistry()` replaces the ciphernode-registry pointer without migrating membership or changing existing operator records.

```
function setRegistry(ICiphernodeRegistry newRegistry) public onlyOwner {
    registry = newRegistry;
    emit RegistrySet(address(newRegistry));
}
```

After governance activates a fresh registry, existing operators remain registered and can remain active in BondingRegistry. `numActiveOperators` also retains its previous value.

The fresh registry trusts this count when it accepts a committee request, even if its own membership tree is empty.

```
uint256 activeCount = bondingRegistry.numActiveOperators();
require(
    threshold[1] <= activeCount,
    InsufficientCiphernodes(threshold[1], activeCount)
);

roots[e3Id] = root();
```

Interfold transfers the requester's fee before calling `requestCommittee()`. Because the stale

BondingRegistry count satisfies the capacity check, the request succeeds and the fee remains escrowed.

Ticket submission then rejects every unmigrated operator because `submitTicket()` requires both current registry membership and BondingRegistry activity.

```
require(
    isEnabled(msg.sender) && _bondingFor(e3Id).isActive(msg.sender),
    NodeNotEligible()
);
```

Operators cannot repair their membership by registering again because `_registerOperator()` reverts with `AlreadyRegistered`. Full deregistration also reverts because BondingRegistry calls `removeCiphernode()` on the fresh registry, where the operator is not enabled. EVM atomicity rolls back the preceding deregistration changes.

The registry owner can import operators with `addCiphernode()`. Governance can also atomically restore all affected pointers to the old registry. These actions repair operator lifecycle calls and future requests. They do not necessarily repair an E3 that already snapshotted the fresh registry's empty membership root.

The issue does not introduce a permanent collateral lock. Subject to the existing bond-owner, slash-proposal, and exit-delay checks, bond owners can separately remove ticket balances, unbond license tokens, and claim the resulting exits.

The primary impact is failed E3s, delayed requester refunds, blocked registration and deregistration, and protocol unavailability during an incomplete migration. No unprivileged caller can rotate the registry. However, permissionless requesters can submit after governance activates a fresh but unmigrated registry or during a non-atomic migration.

This issue remains reachable when all contract pointers form a coherent graph. The missing state is operator membership, not dependency addresses.

Recommendations:

It is recommended to enforce a complete membership migration before activating a new registry:

1. Use or add an enforceable request pause.
2. Deploy and wire the new registry.
3. Import every registered operator before activation.
4. Verify the complete migrated membership set.
5. Atomically update all reciprocal contract pointers.
6. Update ciphernode runtime routing.

7. Resume requests only after all consistency checks succeed.

Add a batch migration function that verifies each imported operator through `BondingRegistry.isRegistered()`. Because BondingRegistry does not expose an enumerable operator list, use a canonical migration manifest, sorted membership commitment, or enumerable canonical operator set to prove completeness. Count equality alone is insufficient.

The pointer activation should require reciprocal wiring and a completed migration version or commitment from the new registry.

The runtime migration must use one of two explicit models:

- Drain every E3 assigned to the old registry before changing runtime configuration; or
- Implement per-E3 runtime registry routing so old and new registry generations can operate concurrently.

Without one of these runtime models, the contracts can preserve old E3 dependencies while ciphernode processes send actions only to the newly configured registry.

Interfold: Resolved with `@145014a499 ...` and `@9f1bd3201e ...`

Zenith: Verified.

4.5 Informational

A total of 18 informational findings were identified.

[I-1] The inclusive failure-reason bound admits a non-classifiable sentinel

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/interfaces/IInterfold.sol#L44-L61
- packages/interfold-contracts/contracts/Interfold.sol#L818-L831
- packages/interfold-contracts/contracts/E3RefundManager.sol#L282-L308

Description:

`FailureReason._MAX_FAILURE_REASON` is a sentinel delimiting the enum rather than an actual failure reason:

```
enum FailureReason {
    None,
    CommitteeFormationTimeout,
    InsufficientCommitteeMembers,
    DKGTimeout,
    DKGInvalidShares,
    NoInputsReceived,
    ComputeTimeout,
    ComputeProviderExpired,
    ComputeProviderFailed,
    RequesterCancelled,
    DecryptionTimeout,
    DecryptionInvalidShares,
    VerificationFailed,
    _MAX_FAILURE_REASON
}
```

`Interfold.onE3Failed` uses an inclusive upper bound and therefore accepts the sentinel:

```
require(
    reason > 0 && reason <= uint8(FailureReason._MAX_FAILURE_REASON),
    "Invalid failure reason"
);
```

A snapshotted registry or slashing manager can consequently transition an E3 to `Failed` while storing `_MAX_FAILURE_REASON`.

When `processE3Failure` is subsequently called, `E3RefundManager.calculateRefund` passes the stored reason to `getFailurePayer`. That function correctly classifies only real failure reasons and reverts for the sentinel:

```
revert InvalidFailureReason(reason);
```

The processing transaction reverts atomically. The attempted fee transfer and prior `e3Payments[e3Id] = 0` write are rolled back, leaving the payment in Interfold while the E3 remains `Failed`. Every subsequent processing attempt encounters the same reason and reverts, preventing refunds and node or treasury settlement until the implementation is upgraded.

The condition is not reachable through normal current flows:

- permissionless `markE3Failed` derives only real timeout reasons;
- the registry currently passes `InsufficientCommitteeMembers`; and
- `SlashingManager.setSlashPolicy` already requires `failureReason < _MAX_FAILURE_REASON`.

Triggering it therefore requires a buggy, malicious, legacy, or incoherently configured authorized dependency. This is primarily a defensive-validation and integration issue rather than a current unprivileged exploit.

Recommendations:

It is recommended to centralize failure-reason validation and use a strict upper bound before converting the raw value to the enum:

```
error InvalidFailureReason(uint8 reason);

function _validateFailureReason(
    uint8 raw
) internal pure returns (FailureReason reason) {
    if (
        raw == uint8(FailureReason.None) ||
        raw >= uint8(FailureReason._MAX_FAILURE_REASON)
    ) {
```

```
            revert InvalidFailureReason(raw);
        }

        return FailureReason(raw);
    }
```

`onE3Failed` should store only the value returned by this helper. Add regression tests proving that authorized callers cannot use `None`, `_MAX_FAILURE_REASON`, or larger values, while expected reasons remain accepted in their valid lifecycle contexts.

Do not add a general administrative function for rewriting recorded failure reasons, as that would allow retroactive changes to refund attribution. If an affected deployed E3 is actually identified, handle it through a narrowly scoped migration whose replacement reason is derived from verifiable lifecycle state.

Interfold: Resolved with `@a61bbfd70a ...`

Zenith: Verified.

[I-2] The operator registry has a finite lifetime insertion budget

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L492-L530
- interfold-contracts/contracts/registry/BondingRegistry.sol#L399-L475

Description:

The ciphernode registry uses a depth-20 incremental Merkle tree with a lifetime capacity of `2^20`, or 1,048,576 insertions. Every newly enabled node is appended at `numberOfLeaves`. Removing a node replaces its leaf with zero but does not reduce `numberOfLeaves` or make the index available for reuse:

```
uint40 index = ciphernodes.numberOfLeaves;
require(
    uint256(index) < MAX_CIPHERNODE_LEAVES,
    CiphernodeTreeExhausted()
);
ciphernodes._insert(uint160(node));
```

```
uint40 index = ciphernodeTreeIndex[node];
ciphernodes._update(0, index);
ciphernodeEnabled[node] = false;
```

Registration and deregistration churn therefore consumes the finite insertion budget monotonically. Re-registering a previously removed address consumes another leaf. Once all `2^20` insertion slots have been used, subsequent registrations revert even if most earlier leaves have been zeroed. Reaching this state would prevent new operators and previously removed operators from joining. Existing operators and E3s would continue functioning, but membership could no longer be replenished as operators leave.

This is not a practically feasible Ethereum-mainnet attack under the reference deployment configuration:

- exhaustion requires 1,048,576 successful insertions;
- the deployment script configures a 100 FOLD license bond;

- the deployment script configures a seven-day exit delay; and
- each cycle also incurs bonding, registration, deregistration, Merkle hashing, and storage costs.

Exhausting the tree without recycling capital during one seven-day exit cycle would require approximately 104.9 million FOLD. Conversely, sequentially recycling one 100 FOLD bond at a seven-day delay would take approximately 20,000 years.

Both `licenseRequiredBond` and `exitDelay` are governance-configurable rather than protocol constants. The enforced exit-delay range is 1—90 days, so different future configurations or deployments may change the economics. Nevertheless, the number of required insertions remains very large for an operator registry.

The state is technically reachable because Ethereum does not impose a fixed per-contract storage-size limit and insertions can accumulate across blocks. It should therefore be treated as a long-term scalability and maintenance constraint, not a presently exploitable denial-of-service vulnerability. The registry is upgradeable, so governance can also migrate or modify the implementation before capacity is exhausted.

Recommendations:

It is recommended to monitor `treeSize()` against `MAX_CIPHERNODE_LEAVES`, alert governance as lifetime usage increases, and document a migration threshold well before capacity is reached. Given the expected operator scale, no immediate code change is necessary.

If substantial registration churn is expected, consider reusing zeroed indices:

```
if (_freeIndices.length != 0) {
    index = _freeIndices[_freeIndices.length - 1];
    _freeIndices.pop();
    ciphernodes._update(uint160(node), index);
} else {
    index = ciphernodes.numberOfLeaves;
    ciphernodes._insert(uint160(node));
}
```

Before introducing a free list, confirm that off-chain components and event consumers do not treat tree indices as permanent operator identities. Index reuse must also remain compatible with reconstruction of historical roots.

If stable append-only indices are required, use versioned tree epochs instead. New registrations can move to a fresh tree while old roots and tree versions remain available for in-flight E3s.

Interfold: Resolved with `@1914ef40a1 ...`

Zenith: Verified.

[I-3] Minting lifecycle documentation conflicts with the implementation

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L64-L68
- packages/interfold-contracts/contracts/token/InterfoldToken.sol#L115-L117
- packages/interfold-contracts/contracts/token/InterfoldToken.sol#L368-L394

Description:

The implementation and function comments permit minting during every pre-TGE phase:

```
if (phase() == Phase.Live) revert MintingClosed();
```

Consequently, `DEFAULT_ADMIN_ROLE` and `MINTER_ROLE` can mint during the CCA and cooldown. Cooldown lasts at least 40 days after `CCA_END` and continues until someone calls permissionless `tge()`.

Other repository documentation states that minting is Virtual-only and that supply is distributed before `CCA_START`. This includes the `MintingClosed` error documentation, role description, flow trace, and token rewrite summary.

The behavior appears intentional because both minting functions explicitly document pre-TGE availability. No unprivileged minting path exists, and supply remains capped at 1.2 billion FOLD. Additional minting also does not alter the CCA's fixed auction inventory, LP reserve, or price calculation.

The mainnet checklist verifies 150 million FOLD at sale deployment, but does not unambiguously define that amount as final supply. Therefore, this is a specification and operational-assumption inconsistency rather than a confirmed vulnerability.

Recommendations:

It is recommended to establish one authoritative minting lifecycle and update all conflicting comments, flow traces, checklists, tests, and investor-facing supply documentation.

If minting until TGE is intended, document the allocation schedule, supply authority, and

distinction between deployment-time and final supply.

If supply must instead freeze at `CCA_START`, enforce `phase() == Phase.Virtual`, add lifecycle-boundary tests, and consider a one-way mint-finalization mechanism or dedicated revocable minting role.

Interfold: Resolved with `@37c2aa6edd ...`

Zenith: Verified.

[I-4] Honest node refund share permanently stranded when per-node amount rounds to zero

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- E3RefundManager.calculateRefund()
- E3RefundManager.claimHonestNodeReward()

Description

calculateRefund() computes `perNodeAmount = honestNodeAmount / honestNodes.length`. When the numerator is smaller than the denominator, Solidity integer division truncates to zero. claimHonestNodeReward() then reverts with `NoRefundAvailable` because it requires `perNodeAmount > 0`. No honest node can claim. The dust-routing-to-treasury path (line 452) triggers only on the last successful claim — since no claim succeeds, it never fires. The `honestNodeAmount` sits in E3RefundManager permanently.

With the default 40% honest allocation and 2-decimal GUSD: a $5 fee with 256 nodes produces `honestNodeAmount = 200 cents`, `perNodeAmount = floor(200/256) = 0`. Two dollars permanently stranded.

With 6-decimal USDC, triggering requires fees under $0.00064 for a 256-node committee — not economically realistic.

Risk Analysis

- **Impact:** Low — Dust-level token amounts (up to `committeeSize - 1` smallest units per failed E3) are permanently locked in E3RefundManager with no recovery path. No attacker profit.
- **Likelihood:** Low — Requires `honestNodeAmount < honestNodeCount`, which demands either a very low-decimal fee token (e.g., 2-decimal GUSD), a tiny E3 fee, or a governance-configured allocation that assigns minimal BPS to pre-decryption work. Not realistic with standard 6+ decimal tokens at normal fee levels.

Recommendations

Consider documenting that fee tokens below 6 decimals are not supported, or add a fallback when `perNodeAmount == 0`: route the entire `honestNodeAmount` to treasury or back to the requester rather than leaving it stranded.

```
if (honestNodes.length > 0 && finalDist.perNodeAmount == 0 &&
    finalDist.honestNodeAmount > 0) {
    // Route dust to treasury instead of stranding
    _pendingTreasury[policyTreasury][paymentToken]
    += finalDist.honestNodeAmount;
    finalDist.honestNodeAmount = 0;
}
```

Interfold: Resolved with `@1570b33ad8 ...`

Zenith: Verified.

[I-5] Per-route rounding directs cumulative slash dust to the protocol and final committee address

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/E3RefundManager.sol#L484-L597
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L818-L889

Description:

Slashed funds received before E3 settlement becomes ready are accumulated by `(e3Id, token)` and split once. After settlement is ready, every subsequent slash route is immediately settled independently:

```
_pendingSlashedByToken[e3Id][token] += amount;

if (_distributions[e3Id].calculated || _successSettlementReady[e3Id]) {
    _settleSlashedFunds(e3Id, token);
}
```

For successful E3s, the node percentage is rounded down separately for each settlement:

```
uint256 toNodes =
    (amount * _successSlashedNodeBpsFor(e3Id)) / BPS_BASE;
uint256 toProtocol = amount - toNodes;
```

At a 50% node allocation, two independently settled routes of three base units allocate two units to nodes and four to the protocol. A single aggregated route of six units allocates three units to each.

For `R` separately settled routes, the cumulative difference from aggregate calculation is bounded by:

```
floor(sum(amounts) × bps / 10,000)−
 sum(floor(amount × bps / 10,000))≤
 R − 1 base units
```

The node portion is then divided using floor division, with every remainder assigned to the final active-node array member:

```
uint256 perNode = amount / nodes.length;

uint256 nodeAmount = i == nodes.length - 1
    ? amount - distributed
    : perNode;
```

The final member receives up to `nodes.length - 1` additional base units per settlement. Because the committee uses canonical address ordering, this repeatedly favors the final remaining address unless membership changes.

Percentage drift applies only to successful-E3 settlement. The final-node remainder behavior applies to both successful and failed-E3 slash distributions.

No value is lost, and an unprivileged caller cannot freely divide one penalty into unlimited routes. Route boundaries generally correspond to separate slash proposals with governance-configured penalties. With a conventional six-decimal stablecoin, the discrepancy is normally negligible.

Recommendations:

It is recommended to document the current rounding policy explicitly if directing percentage dust to the protocol and node-division dust to the final canonical address is intentional.

If exact cumulative percentage allocation is required, carry fractional numerators forward per `(e3Id, token)` for successful-E3 settlement:

```
uint256 bps = _successSlashedNodeBpsFor(e3Id);

uint256 toNodes = Math.mulDiv(amount, bps, BPS_BASE);
uint256 remainder =
    mulmod(amount, bps, BPS_BASE) +
    percentageRemainder[e3Id][token];

toNodes += remainder / BPS_BASE;
percentageRemainder[e3Id][token] = remainder % BPS_BASE;

uint256 toProtocol = amount - toNodes;
```

Apply this only when active recipients exist. If `nodes.length == 0`, route the entire amount to the protocol and clear any stored node-share remainder according to the documented terminal policy.

For node-division dust, maintain a cursor keyed by `(e3Id, token)` and distribute the re-

mainder beginning from successive active-node positions instead of always assigning it to the final member. If expulsions change the active set between settlements, document that positional rotation provides bounded bias rather than exact lifetime equality.

Add tests covering fragmented versus aggregated raw amounts, the `R - 1` bound, multiple tokens, zero active recipients, repeated node remainders, membership changes, and conservation across node and treasury credits.

Interfold: Resolved with `@26f32ced5c ...`

Zenith: Verified.

[I-6] Floor rounding can leave operators active with zero license collateral

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L525-L579
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L759-L785
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L1033-L1063

Description:

Active status requires the operator's license bond to satisfy a retained percentage of `licenseRequiredBond`. This threshold is calculated using floor division:

```
bool newActiveStatus = op.registered &&
    op.licenseBond >= _minLicenseBond() &&
    (ticketToken.balanceOf(operator) / ticketPrice >= minTicketBalance);

function _minLicenseBond() internal view returns (uint256) {
    return (licenseRequiredBond * licenseActiveBps) / BPS_BASE;
}
```

Although both configuration values must be positive, the derived threshold is zero whenever:

```
licenseRequiredBond × licenseActiveBps < 10,000
```

For example, `licenseRequiredBond = 1` and `licenseActiveBps = 8,000` produce `_minLicenseBond() = 0`.

An operator can bond one unit, register, and then unbond its entire license balance. `unbondLicense` queues the collateral and invokes `_updateOperatorStatus`, where the remaining zero balance satisfies `0 ≥ 0`. The operator therefore remains active if it still holds enough tickets.

The queued license remains slashable until claimed. After `exitDelay`, however, `claimExits` transfers the collateral without recomputing activity, leaving the registered operator active and committee-eligible with no license collateral. The withdrawn unit can eventually be

reused to register other identities.

The direct rounding difference is at most one smallest token unit per operator. Its effect is nevertheless categorical because zero collateral passes the activity predicate. Exploitation requires an owner-selected edge-case configuration; repository defaults and tracked deployment arguments use a `100e18` required bond and `8,000` bps, producing an unaffected `80e18` threshold.

Recommendations:

It is recommended to round retained collateral requirements upward using full-precision multiplication:

```
function _minLicenseBond() internal view returns (uint256) {
    return
        Math.mulDiv(
            licenseRequiredBond,
            licenseActiveBps,
            BPS_BASE,
            Math.Rounding.Ceil
        );
}
```

For positive inputs, this guarantees a threshold of at least one base unit. It changes non-integral thresholds by at most one base unit and avoids intermediate multiplication overflow.

Do not add a derived-threshold check mechanically to both existing setters. Initialization currently configures `licenseRequiredBond` before `licenseActiveBps`, so validating the incomplete pair could break deployment. Ceiling division requires no additional nonzero check.

If floor rounding must be retained, validate the complete configuration pair atomically and without direct multiplication. For example, require:

```
licenseRequiredBond >= Math.ceilDiv(BPS_BASE, licenseActiveBps);
```

Add tests covering `(1, 8_000)`, `(9_999, 1)`, `10_000` bps, very large required-bond values, full unbond followed by exit claim, and refresh of an existing zero-bond operator after a configuration update.

Interfold: Resolved with `@c80e407789 ...`

Zenith: Verified.

[I-7] Requests rely on ERC-20 allowances rather than an in-call fee bound

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L243-L264
- packages/interfold-contracts/contracts/Interfold.sol#L343-L347
- packages/interfold-contracts/contracts/Interfold.sol#L1041-L1071

Description:

`request` calculates the fee from the current pricing, timeout, committee-threshold, and registry configuration, then transfers it without accepting a caller-specified maximum fee or expected fee token.

```
function request(
    E3RequestParams calldata requestParams
) external returns (uint256 e3Id, E3 memory e3) {
    // ...
    uint256 e3Fee = getE3Quote(requestParams);
    // ...
    feeToken.safeTransferFrom(msg.sender, address(this), e3Fee);
}
```

A privileged configuration update can execute after the requester obtains a quote but before their transaction is included. The request then uses the updated fee and succeeds if the requester's balance and allowance are sufficient.

This is not permissionless price manipulation because the relevant configuration setters are owner-restricted. Furthermore, the repository's request helpers approve exactly the quoted amount, which causes a request to revert if its fee subsequently increases. The SDK also permits callers to approve an exact amount.

Exposure therefore primarily affects requesters who maintain an allowance larger than the observed quote, particularly an unlimited allowance. For these requesters, the ERC-20 allowance does not express the maximum fee intended for that individual request.

Recommendations:

It is recommended to add a protected request entry point that accepts both the expected fee token and a maximum fee. Update the SDK to use it by default, while deprecating the legacy entry point if ABI compatibility requires retaining it.

```
function request(
    E3RequestParams calldata requestParams,
    IERC20 expectedFeeToken,
    uint256 maxFee
) external returns (uint256 e3Id, E3 memory e3) {
    require(feeToken == expectedFeeToken, FeeTokenChanged());

    uint256 e3Fee = getE3Quote(requestParams);
    require(e3Fee <= maxFee, FeeExceedsMaximum(e3Fee, maxFee));

    // Continue processing the request.
}
```

Binding the token is important because a fee-token rotation can change both token identity and denomination. A separate quote deadline may be added for UX purposes but is not essential because the request's input window already limits when it can be submitted.

Interfold: Resolved with `@1914ef40a1 ...`

Zenith: Verified.

[I-8] Fail-open active-node lookup can misallocate refunds after an unexpected registry failure

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L109-L129
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L953-L973
- packages/interfold-contracts/contracts/E3RefundManager.sol#L188-L199

Description:

When a failed E3 is processed, `InterfoldPricing.honestNodes` obtains its honest recipients from the request-time registry. For committee-formation failures, returning no recipients is expected because no canonical committee exists. However, the helper also converts every unexpected registry revert into an empty node list.

```
try registry.getActiveCommitteeNodes(e3Id) returns (
    address[] memory nodes,
    uint256[] memory
) {
    return nodes;
} catch {
    return new address[](0);
}
```

`E3RefundManager.calculateRefund` cannot distinguish this fallback from a legitimate absence of honest nodes and reallocates the node work share to the requester.

```
if (honestNodes.length == 0 && honestNodeAmount > 0) {
    requesterAmount += honestNodeAmount;
    honestNodeAmount = 0;
}
```

For a requester-attributable failure under the default allocation, an unexpected lookup failure would change the distribution from 40% to honest nodes, 55% to the requester, and 5% to the protocol into 0%, 95%, and 5%, respectively.

The production registry lookup has no explicit revert condition, is bounded to at most 256 members, and exposes no requester-controlled input beyond `e3Id`. No ordinary attacker-controlled trigger was identified. The issue therefore requires an abnormal condition such as an incompatible trusted-registry upgrade or corrupted committee accounting and is a fail-open settlement risk rather than a presently exploitable path.

Recommendations:

It is recommended to make the registry getter explicitly handle the legitimate absence of a finalized committee, while allowing unexpected failures to revert failure processing.

```
function getActiveCommitteeNodes(
    uint256 e3Id
) external view returns (address[] memory nodes, uint256[] memory scores) {
    Committee storage c = committees[e3Id];

    if (c.stage ≠ ICiphernodeRegistry.CommitteeStage.Finalized) {
        return (new address[](0), new uint256[](0));
    }

    // Build and return the active finalized committee.
}
```

Then remove the fail-open `try/catch`:

```
(address[] memory nodes, ) = registry.getActiveCommitteeNodes(e3Id);
return nodes;
```

This approach uses actual committee state rather than assuming that a particular failure reason always means no committee exists. An unexpected registry failure will revert the complete transaction, restoring the cleared E3 payment and allowing processing to be re-tried.

Add tests confirming that pre-finalization failures legitimately return no recipients, while an unexpected registry revert rolls back payment clearing and refund calculation. If fail-open settlement is intentional for liveness, document the resulting economic reallocation explicitly.

Interfold: Resolved with `@1b11f24480 ...`

Zenith: Verified.

[I-9] Duplicate cap definitions can desynchronize exposed and enforced limits

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L42-L52
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L27-L30
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L237-L247
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L285-L292

Description:

`Interfold` exposes the maximum committee size, protocol share, and margin through public ABI-facing constants. Its validation logic does not consume these values, however; `InterfoldPricing` maintains independent internal copies used to enforce the limits.

```
// Interfold.sol — values exposed to integrations
uint32 public constant MAX_COMMITTEE_SIZE = 256;
uint16 public constant MAX_PROTOCOL_SHARE_BPS = 5_000;
uint16 public constant MAX_MARGIN_BPS = 5_000;

// InterfoldPricing.sol — values actually enforced
uint16 internal constant MAX_PROTOCOL_SHARE_BPS = 5_000;
uint16 internal constant MAX_MARGIN_BPS = 5_000;
uint32 internal constant MAX_COMMITTEE_SIZE = 256;
```

All definitions currently match, and the audited implementation correctly enforces the advertised limits. There is therefore no present validation bypass.

A future upgrade could nevertheless change only one copy, causing the limits exposed through the `Interfold` ABI to differ from those enforced by `InterfoldPricing`. This could mislead integrations or unintentionally relax the committee-size or fee protections. The issue is therefore a maintenance and upgrade-safety risk rather than a currently reachable attack.

Recommendations:

It is recommended to make the public Interfold constants the single source of truth. Pass them into the external library validators, as already done with `MAX_TIMEOUT_WINDOW`, and remove the corresponding internal copies from `InterfoldPricing`.

```
InterfoldPricing.validateCommitteeThresholds(
    threshold,
    pc.minCommitteeSize,
    pc.minThreshold,
    MAX_COMMITTEE_SIZE
);

InterfoldPricing.validatePricingConfig(
    config,
    MAX_MARGIN_BPS,
    MAX_PROTOCOL_SHARE_BPS
);
```

Update the validator signatures accordingly. Add invariant tests that read each public limit, confirm the limit itself is accepted, and confirm `limit + 1` is rejected.

Interfold: Resolved with `@20302b366b ...`

Zenith: Verified.

[I-10] Duplicate owner validation in `_bondLicense` obscures the authorization boundary

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L640-L645
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L1147-L1153
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L1179-L1183

Description:

`bondLicenseFor` applies `onlyBondOwner` before calling `_bondLicense`. The modifier reads `bondOwnerOf(operator)` and requires `msg.sender` to equal that owner.

`_bondLicense` then reads the owner again and checks only that the owner is nonzero.

```
function bondLicenseFor(
    address operator,
    uint256 amount
) external nonReentrant noExitInProgress(operator) onlyBondOwner(operator) {
    _bondLicense(operator, amount);
}

function _bondLicense(address operator, uint256 amount) internal {
    require(operator ≠ address(0), ZeroAddress());
    require(amount ≠ 0, ZeroAmount());

    address bondOwner = bondOwnerOf(operator);
    require(bondOwner ≠ address(0), NotBondOwner(msg.sender, operator));
}
```

If the operator has no owner, `onlyBondOwner` reverts because `msg.sender` cannot equal the zero address. The later zero-owner check therefore cannot be the first failure through the only current call path.

The internal check is not an authorization check. It does not compare `msg.sender` with `bondOwner`. A future internal caller that omits `onlyBondOwner` could bypass the intended authorization boundary whenever the operator has a configured owner.

The current implementation does not permit unauthorized bonding or asset loss. The duplicate lookup adds minor gas cost and makes the authorization boundary harder to review. Exploitation requires a future refactor that introduces an unprotected internal call path.

Recommendations:

It is recommended to enforce owner authorization once inside `_bondLicense` and remove `onlyBondOwner` from `bondLicenseFor`. This places authorization at the accounting boundary and protects every future internal call path.

```
function bondLicenseFor(
    address operator,
    uint256 amount
) external nonReentrant noExitInProgress(operator) {
    _bondLicense(operator, amount);
}

function _bondLicense(address operator, uint256 amount) internal {
    require(operator ≠ address(0), ZeroAddress());
    require(amount ≠ 0, ZeroAmount());

    address bondOwner = bondOwnerOf(operator);
    if (msg.sender ≠ bondOwner) {
        revert NotBondOwner(msg.sender, operator);
    }

    operators[operator].licenseBond += amount;
    _bondedByOwner[bondOwner] += amount;

    // Continue the existing transfer and accounting flow.
}
```

Add tests that call `bondLicenseFor` with the configured owner, a non-owner, an operator without an owner, and the zero operator address.

Interfold: Resolved with `@95e9bcd5c3 ...`

Zenith: Verified.

[I-11] `onlyBondingRegistry` modifier is defined but never used

`SEVERITY:` Informational	`IMPACT:` Informational
`STATUS:` Resolved	`LIKELIHOOD:` Low

Target

- CiphernodeRegistryOwnable.sol

Description

The modifier is defined but never applied to any function. `onlyOwnerOrBondingVault` subsumes its role for all bonding-registry-gated operations. Dead modifiers increase surface area for misuse during future development and bloat deployment bytecode.

Recommendations

Remove the modifier and its associated `OnlyBondingRegistry` error if no future use is planned.

Interfold: Resolved with PR-1768.

Zenith: Verified.

[I-12] `setAccusationVoteValidity` NatSpec contradicts its `require` guard

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- CiphernodeRegistryOwnable.sol

Description

The NatSpec comment on line 698 states "Zero is allowed and intentionally disables slashing submission." The code on line 704 rejects zero. The comment describes system-level behavior (zero IS reachable via the timelocked `proposeAccusationVoteValidity` → `commitAccusationVoteValidity` path) but is placed on the wrong function, creating a misleading contradiction for integrators reading the direct setter in isolation. The error name `AccusationVoteValidityZeroRequiresTimelock` correctly describes the intent.

Recommendations

Move the zero-allowed documentation to the propose/commit functions. Update the direct setter's NatSpec to state that zero is disallowed here and requires the timelocked path.

Interfold: Resolved with PR-1768.

Zenith: Verified.

[I-13] Near-zero `accusationVoteValidity` bypasses the timelock intent

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- CiphernodeRegistryOwnable.sol

Description:

The 2-day timelock on zeroing `accusationVoteValidity` exists so operators can observe and react before slashing is effectively disabled. The direct setter rejects zero with `AccusationVoteValidityZeroRequiresTimelock()`, but accepts any nonzero value immediately:

```
require(
    _accusationVoteValidity != 0,
    AccusationVoteValidityZeroRequiresTimelock()
);
```

`setAccusationVoteValidity(1)` passes the guard and produces a 1-second validity window. Under realistic mempool latency, no vote can be submitted on-chain within 1 second. The effect is identical to zeroing: slashing is disabled. Operators receive no 2-day observation window.

Recommendations:

Enforce a minimum floor on the direct setter (e.g., `≥ DEFAULT_ACCUSATION_VOTE_VALIDITY / 2`), or route all reductions below a defined threshold through the timelocked path.

Interfold: Resolved with `@76b9a68686 ...`

Zenith: Verified.

[I-14] The license-token setter accepts tokens that disable owner transfers for license-bonded positions

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/interfaces/ILockAwareLicenseToken.sol#L7-L11
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L461-L491
- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L987-L1012

Description:

`BondingRegistry` stores the license token as `IERC20`. For compatibility of the replacement token, `setLicenseToken` verifies only that its nonzero address contains code. It does not verify support for `lockedBalanceOf`.

```
function setLicenseToken(IERC20 newLicenseToken) public onlyOwner {
    address next = address(newLicenseToken);
    IERC20 current = licenseToken;

    if (next == address(0)) {
        if (address(current) != address(0)) {
            revert InvalidBondingAsset(next);
        }
    } else if (next.code.length == 0) {
        revert InvalidBondingAsset(next);
    }

    // Existing-token liability checks do not validate the new interface.

    licenseToken = newLicenseToken;
    emit LicenseTokenSet(next);
}
```

`acceptBondOwner` later treats the token as `ILockAwareLicenseToken` when the position has active or pending license collateral.

```
uint256 delegatedBond =
    operators[operator].licenseBond + pendingLicense;

if (delegatedBond ≠ 0) {
    uint256 lockedBalance = ILockAwareLicenseToken(
        address(licenseToken)
    ).lockedBalanceOf(previousOwner);

    // Continue the owner-transfer validation.
}
```

A plain ERC-20 normally reverts for the unknown selector or returns no valid ABI-encoded value. The funded owner-transfer transaction then reverts.

The issue affects only positions with active or pending license collateral. Ticket-only and unfunded positions do not execute the external call.

The intended `InterfoldToken` implements `lockedBalanceOf`, so the default deployment is safe. The issue requires trusted governance to configure an incompatible replacement token.

Collateral does not become permanently inaccessible. The current owner can unbond, wait for the exit delay, and withdraw it. The incompatibility disables funded owner rotation and can require position downtime.

Recommendations:

It is recommended to require every nonzero license token to implement `ILockAwareLicenseToken`.

During `setLicenseToken`, probe `lockedBalanceOf(address(this))`. Require the call to succeed and return exactly one ABI-encoded `uint256`. The initial zero placeholder can remain exempt.

```
error IncompatibleLicenseToken(address token);

function _requireLockAwareLicenseToken(IERC20 token) internal view {
    (bool success, bytes memory result) = address(token).staticcall(
        abi.encodeCall(
            ILockAwareLicenseToken.lockedBalanceOf,
            (address(this))
        )
    );

    if (!success || result.length ≠ 32) {
        revert IncompatibleLicenseToken(address(token));
```

```
        }

        abi.decode(result, (uint256));
    }
```

A successful probe confirms compatibility only for the tested call. `acceptBondOwner` must still surface a clear custom error if a later account-specific call fails.

Remove generic ERC-20 support unless the protocol has a concrete requirement for it. If plain-token support is required, define a separate explicit mode and document that such tokens have no protocol-recognized lock balance.

Add configuration and funded owner-transfer tests for `InterfoldToken`, a plain ERC-20, a reverting implementation, and a malformed-return implementation.

Interfold: Resolved with `@f752fe487d ...`

Zenith: Verified.

[I-15] `E3Program` reentrancy can emit regressive lifecycle events

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L360-L403
- packages/interfold-contracts/contracts/Interfold.sol#L407-L442
- crates/request/src/routing/workflow.rs#L86-L108

Description:

`publishCiphertextOutput()` stores the ciphertext and advances the E3 to `CiphertextReady` before calling the selected E3 program's non-view `verify()` function.

The contract emits the ciphertext lifecycle events only after the callback returns.

```
e3s[e3Id].ciphertextOutput = ciphertextOutputHash;
e3s[e3Id].ciphertextCommitment = ciphertextCommitment;
_e3Stages[e3Id] = E3Stage.CiphertextReady;

(success) = e3.e3Program.verify(e3Id, ciphertextOutputHash, proof);
require(success, InvalidOutput(ciphertextOutput));

emit CiphertextOutputPublished(
    e3Id,
    ciphertextOutput,
    ciphertextCommitment
);
emit E3StageChanged(
    e3Id,
    E3Stage.KeyPublished,
    E3Stage.CiphertextReady
);
```

A malicious E3 program can call `publishPlaintextOutput()` from its `verify()` callback. The call succeeds if it supplies a valid plaintext proof bound to:

- The exact ciphertext and commitment from the outer call.
- The E3 committee and public key.

- The Interfold deployment, chain, and E3 identifier.

With the production verifier, obtaining this proof before the ciphertext event normally requires threshold-committee cooperation or proof generation outside the standard event-driven runtime.

The reentrant call marks the E3 complete and distributes rewards before the outer call emits its ciphertext events.

```
e3s[e3Id].plaintextOutput = plaintextOutput;
_e3Stages[e3Id] = E3Stage.Complete;

_verifyPlaintext(e3Id, keccak256(plaintextOutput), proof);
_distributeRewards(e3Id);

emit PlaintextOutputPublished(e3Id, plaintextOutput, proof);
emit E3StageChanged(e3Id, E3Stage.CiphertextReady, E3Stage.Complete);
```

The relevant lifecycle and output events appear in this order:

1. `PlaintextOutputPublished`
2. `E3StageChanged(CiphertextReady, Complete)`
3. `CiphertextOutputPublished`
4. `E3StageChanged(KeyPublished, CiphertextReady)`

Reward events can also occur before `PlaintextOutputPublished`.

The final on-chain stage remains `Complete`, and reward distribution executes once. However, the last lifecycle event describes a transition back to `CiphertextReady`.

The Rust router can report the later ciphertext and stage events as unexpected if it processes `E3RequestComplete` first. Otherwise, the lifecycle coordinator preserves the terminal state and ignores the regression.

Recommendations:

It is recommended to apply one shared reentrancy guard to both `publishCiphertextOutput()` and `publishPlaintextOutput()`.

Use an upgrade-safe guard and initialize it through the appropriate initializer or reinitializer. After `e3Program.verify()` returns, also confirm that the E3 remains in `CiphertextReady` before emitting the ciphertext events.

If E3 programs do not require state changes during verification, consider making `IE3Program.verify()` view-only. This change causes Solidity to use `STATICCALL`, but it can break existing stateful programs and requires a compatibility review.

Do not only move `verify()` before the state updates. Without a reentrancy guard, that change permits recursive ciphertext-publication attempts while the E3 remains in `KeyPublished`.

Add a regression test that performs the callback and confirms that lifecycle events cannot appear in reverse order.

Interfold: Resolved with `@f62dae4950 ...`

Zenith: Verified.

[I-16] Ticket-token rotation does not verify reciprocal registry authorization

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/registry/BondingRegistry.sol#L965-L984
- packages/interfold-contracts/contracts/token/InterfoldTicketToken.sol#L150-L155
- packages/interfold-contracts/contracts/token/InterfoldTicketToken.sol#L283-L343

Description:

`BondingRegistry.setTicketToken()` checks that the replacement address contains code. It also requires the old token to have no supply or pending payouts.

The function does not verify that the replacement token authorizes the `BondingRegistry`.

```
address next = address(newTicketToken);
if (next.code.length == 0) revert InvalidBondingAsset(next);

// Only the old token's liabilities are checked.

ticketToken = newTicketToken;
emit TicketTokenSet(next);
```

`InterfoldTicketToken` restricts its state-changing collateral functions to its configured `registry`.

```
modifier onlyRegistry() {
    if (msg.sender ≠ registry) revert NotRegistry();
    _;
}
```

A genuine replacement token configured for another registry passes `setTicketToken()`. New ticket deposits then revert when `BondingRegistry` calls `depositFrom()`. Burns also revert if the replacement reports ticket balances that an operator tries to remove or deregister.

An arbitrary contract can also pass the code-length check. Such a contract can make

protocol operations fail and can prevent another rotation by reverting from `totalSupply()` or `payableBalance()`.

Only the trusted owner can create this condition. Governance can usually recover by correcting the token registry or restoring the old token. Therefore, this is a configuration-hardening issue rather than an unprivileged vulnerability.

Recommendations:

It is recommended to use a two-step ticket-token rotation.

Governance should configure the replacement token before activation. `activateTicketToken()` should then:

1. Recheck the old token's liabilities.
2. Confirm that the required replacement-token view calls succeed.
3. Require `newTicketToken.registry() == address(this)`.
4. Assign the replacement only after all checks pass.

```
error TicketTokenRegistryMismatch(address configured, address expected);

function _validateTicketToken(
    InterfoldTicketToken newTicketToken
) internal view {
    address configured;

    try newTicketToken.registry() returns (address value) {
        configured = value;
    } catch {
        revert InvalidBondingAsset(address(newTicketToken));
    }

    if (configured ≠ address(this)) {
        revert TicketTokenRegistryMismatch(configured, address(this));
    }
}
```

This validation does not prove that an arbitrary contract behaves correctly. If only canonical `InterfoldTicketToken` deployments are supported, governance should use an approved deployment source.

Initial deployment requires a separate finalization check because the current bootstrap flow temporarily uses a placeholder registry.

Interfold: Resolved with `@5fd7ea86bb ...`

Zenith: Verified.

[I-17] Fresh Interfold controllers collide with retained local E3-ID state

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L273-L282
- packages/interfold-contracts/contracts/E3RefundManager.sol#L688-L701
- packages/interfold-contracts/contracts/slashing/SlashingManager.sol#L310-L323
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L254-L271
- crates/events/src/e3id.rs#L12-L29

Description:

Each Interfold controller assigns E3 IDs using a local counter. A fresh deployment starts with `nexte3Id = 0`:

```
e3Id = nexte3Id;
nexte3Id++;

dependencies.refundManager.snapshotE3Policy(
    e3Id,
    address(dependencies.registry)
);
```

The refund and slashing managers key their retained dependency snapshots only by this numeric ID:

```
E3PolicySnapshot storage snapshot =
    _e3PolicySnapshots[e3Id];

require(
    !snapshot.initialized,
    "Policy already snapshotted"
);
```

```
E3Dependencies storage dependencies =
    _e3Dependencies[e3Id];

require(
    !dependencies.initialized,
    InvalidProposal()
);
```

Both managers can be administratively redirected to replacement protocol contracts while preserving snapshots needed by historical E3s. Consequently, reusing either manager with a fresh Interfold controller causes unrelated local E3 namespaces to collide.

For the refund manager:

1. The original Interfold creates E3 `0`, initializing `_e3PolicySnapshots[0]`.
2. Governance deploys a fresh Interfold whose counter starts at zero.
3. Governance points the retained refund manager at the fresh Interfold.
4. The fresh Interfold attempts its first request using E3 ID `0`.
5. `snapshotE3Policy(0, ...)` reverts because the old snapshot is initialized.

The entire transaction reverts, including the earlier `nexte3Id++`. The counter remains zero, so every unchanged retry selects ID `0` and fails identically.

The same issue occurs when only the slashing manager is reused. A fresh registry calls `snapshotE3Dependencies(0)` during committee creation, which rejects the retained snapshot and rolls back the request.

Reusing the ciphernode registry creates an independent collision because committees, roots, public keys, and DKG anchors are also indexed only by numeric `e3Id`.

The current initialization guards make these collisions fail closed. Removing or overwriting them would be unsafe because other state is also keyed by local ID, including:

- refund distributions and claim records;
- honest-node lists and slashed-fund ledgers;
- slashing dependencies and evidence replay protection;
- pending slash routes referencing an E3; and
- registry committee and DKG state.

Overwriting historical entries could therefore mix liabilities, routing, membership, or evidence between unrelated E3s.

The off-chain identity has the same assumption. Rust identifies an E3 using only the chain ID and numeric E3 ID:

```
pub struct E3id {
    id: String,
    chain_id: u64,
}
```

A replacement Interfold on the same chain restarting at zero can therefore collide with retained Rust persistence even if fresh on-chain lifecycle components are deployed.

This condition requires privileged deployment or migration configuration and is not a public attack. The failed request does not lose its fee because the complete transaction reverts. Requests remain unavailable only until governance repairs the configuration.

An in-place upgrade of an existing Interfold proxy does not trigger the issue because the controller address and counter are preserved.

Recommendations:

It is recommended to make E3 IDs globally unique across Interfold controllers.

A single globally unique `uint256` ID is preferable to independently re-keying every downstream mapping. For example, reserve 160 bits for the Interfold proxy address and 96 bits for its local counter:

```
uint256 localId = nexte3Id++;
require(
    localId <= type(uint96).max,
    E3IdSpaceExhausted()
);

uint256 e3Id =
    (uint256(uint160(address(this))) << 96) |
    localId;
```

Every existing E3-scoped component would then receive the globally unique value through its existing `uint256 e3Id` interface. Evidence keys and signed payloads that already include `e3Id` would inherit the namespace, while Rust's existing `(chainId, e3Id)` identity would also remain unique.

This changes the assumption that E3 IDs are small sequential values and therefore requires an explicit migration and compatibility review across contracts, events, SDKs, Rust persistence, scripts, tests, and external indexers.

If local numeric IDs must remain externally visible, namespace all E3-scoped state by `(Interfold address, localE3Id)` instead. The namespace must also be included in:

- signed accusation and proof payloads;

- evidence replay keys;
- emitted events and public queries;
- slash routes and dependency lookups; and
- Rust's persisted E3 identity.

Namespacing only dependency snapshots is insufficient.

Under the current design, the safest operational workaround is to deploy fresh refund, slashing, and registry components for a fresh Interfold controller while retaining the old components for historical obligations. Off-chain persistence must also be reset or migrated so the new local IDs cannot collide with retained rounds.

Do not clear or overwrite old state after an E3 reaches a terminal stage because refund claims, slash proposals, appeals, and pending routes can remain live.

Add migration tests proving that:

- an old controller successfully creates E3 `0`;
- components are redirected to a fresh controller;
- the fresh controller's first request fails under the current design;
- its counter remains zero after the revert;
- retrying fails identically; and
- the selected remediation permits the new request without modifying historical obligations.

Interfold: Resolved with `@1914ef40a1 ...`

Zenith: Verified.

[I-18] Expelled members retain restricted grace-period failure authority

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- packages/interfold-contracts/contracts/Interfold.sol#L168-L174
- packages/interfold-contracts/contracts/Interfold.sol#L839-L868
- packages/interfold-contracts/contracts/lib/InterfoldPricing.sol#L180-L197
- packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol#L879-L930

Description:

After a stage deadline, `markE3Failed()` optionally restricts callers for a configured grace period. The documented authorized parties are the requester, owner, and active committee members.

The implementation instead authorizes anyone for whom `isCommitteeMember()` returns true:

```
if (
    block.timestamp < graceEnds &&
    caller ≠ requester &&
    caller ≠ contractOwner &&
    !ICiphernodeRegistry(registry).isCommitteeMember(e3Id, caller)
) revert IInterfold.MarkE3FailedInGracePeriod(e3Id, graceEnds);
```

Expelling a committee member changes its status from `Active` to `Expelled`:

```
c.memberStatus[node] =
    ICiphernodeRegistry.MemberStatus.Expelled;
c.activeCount--;
```

However, `isCommitteeMember()` intentionally preserves historical membership and returns true for every status other than `None`:

```
return
    committees[e3Id].memberStatus[node] ≠
```

```
    ICiphernodeRegistry.MemberStatus.None;
```

An expelled member therefore remains authorized during the restricted interval:

```
deadline < block.timestamp < deadline + markFailedGracePeriod
```

A reachable sequence is:

1. The owner configures a grace period long enough to contain a restricted post-deadline timestamp.
2. An E3 finalizes its committee.
3. A member is expelled while the remaining active committee stays viable.
4. A later stage deadline expires.
5. Before the grace period ends, the expelled address calls `markE3Failed()`.

The failure reason is derived from the current stage and deadline, so the expelled member cannot select an arbitrary reason or fail an E3 before its failure condition is satisfied. The defect instead allows a removed participant to finalize the timeout earlier than unrelated public callers.

The clearest practical consequence is compositional with late DKG publication. The current registry accepts committee publication after the DKG deadline. An expelled member can race that otherwise accepted publication during the grace interval and move the E3 irreversibly to `Failed`. If DKG publication is changed to enforce its deadline strictly, the remaining impact is primarily violation of the intended grace-period authorization policy.

The default grace period is zero. In that configuration, caller validation returns immediately and the defect has no effect because failure marking is already permissionless. Once any configured grace interval expires, every address may call regardless of committee status.

Recommendations:

It is recommended to authorize only finalized, currently active committee members during the restricted grace interval.

Preserve `isCommitteeMember()` for slashing and other flows that intentionally require historical membership. Instead, correct `isCommitteeMemberActive()` so it also requires committee finalization:

```
function isCommitteeMemberActive(
    uint256 e3Id,
    address node
) external view returns (bool) {
```

```
    Committee storage c = committees[e3Id];

    return
        c.stage ==
            ICiphernodeRegistry.CommitteeStage.Finalized &&
        c.memberStatus[node] ==
            ICiphernodeRegistry.MemberStatus.Active;
}
```

Then use that predicate in grace-period validation:

```
if (
    block.timestamp < graceEnds &&
    caller ≠ requester &&
    caller ≠ contractOwner &&
    !ICiphernodeRegistry(registry)
        .isCommitteeMemberActive(e3Id, caller)
) {
    revert IInterfold.MarkE3FailedInGracePeriod(
        e3Id,
        graceEnds
    );
}
```

The finalized-stage condition prevents provisional sortition candidates from being treated as active committee members without adding a new public interface function.

Add regression tests covering requester, owner, active finalized member, expelled member, provisional candidate, and unrelated caller behavior both during and after the grace interval. Also test zero grace and a grace value too short to contain a restricted post-deadline timestamp.

Interfold: Resolved with `@4d32ff77d3 ...`

Zenith: Verified.